



UNIVERSIDAD DE VALLADOLID

ESCUELA TÉCNICA SUPERIOR

INGENIEROS DE TELECOMUNICACIÓN

TALLER DE PROYECTOS II

MÁSTER EN TECNOLOGÍAS DE TELECOMUNICACIÓN

Vehículo conectado capaz de detectar señales en tiempo real

Autores:

**D. Jesús Alonso García, D. Alejandro Peláez Fernández
D. Ángel Alonso García**

Tutores:

**Dr. D. Alfonso Bahillo, Dr. D. Javier Aguiar,
Dr. D. Juan Carlos Aguado**

Valladolid, Junio 2024

Agradecimientos

Queríamos agradecer en primer lugar a los Profesores de la asignatura: Dr. D. Alfonso Bahillo, Dr. D. Javier Aguiar y Dr. D. Juan Carlos Aguado por habernos guiado durante el proyecto, dándonos las pautas básicas para orientarnos a lo largo del proyecto.

También queríamos agradecer a los expertos que han venido a darnos charlas en la materia sobre las distintas partes que componen nuestro proyecto. A David Castaño del Grupo Renault, que nos aconsejó y orientó sobre metodología ágil para la planificación del proyecto, a Ángel Martín que nos informó sobre la infraestructura del 4G, a Eduardo Onieva que nos introdujo en la inteligencia artificial y a Ángel Alves que nos explicó la evolución hacia el 5G y sus muchas aplicaciones.

Y por último, también queríamos agradecer a nuestros compañeros de clase, con los que nos hemos ayudado mutuamente y nos han sido de gran ayuda en ciertos momentos en los que teníamos dificultades para avanzar.

Muchas gracias a todos.

Resumen

En este documento presentamos un informe técnico económico acerca de la viabilidad de realizar el despliegue de una red 4G a lo largo de la autovía A-62 para ofrecer un servicio de vehículo conectado.

En primer lugar, estudiamos la cobertura a lo largo de toda la autovía mediante la simulación con Xirio para después desplegar las estaciones base necesarias para tener una cobertura óptima en todos los puntos de la misma.

Después realizamos un estudio acerca de la arquitectura 4G y diseñamos y buscamos los componentes hardware más adecuados para la estructura del core y de las estaciones base para nuestro caso. Además, estudiamos el despliegue de fibra óptica para unir las dos partes de la red. Una vez hecho esto realizamos la valoración económica de todo el despliegue.

A continuación, estudiamos las alternativas disponibles para la parte de inteligencia artificial, estudiando varios modelos de detección de imágenes y eligiendo el modelo más adecuado a nuestros requerimientos.

Por último, desarrollamos un demostrador sobre el que aplicar todos los conocimientos adquiridos durante el estudio y desplegamos el servicio a pequeña escala utilizando un vehículo Amazon DeepRacer, una BladeRF 2.0 xa9 y una tarjeta SIM programable.

En el demostrador ponemos en marcha el core de la red en un sistema operativo Ubuntu 20.04, mediante el software srsRAN y utilizamos la Blade como estación base. Registramos la tarjeta SIM en el core y la conectamos al coche mediante un módem Huawei.

Hemos grabado y creado un dataset para entrenar un modelo con YOLOv9, para aplicarlo en el core y detectar las imágenes. Una vez conectado todo, grabamos imágenes con el coche y las enviamos a través de la red al core, donde se ejecuta ese modelo y se detectan las señales. Después de esto, enviamos de vuelta al coche el resultado de la detección y mostramos por pantalla el resultado.

Palabras Clave

4G, LTE, eNodeB, EPC, UE, antena, core, cobertura, Xirio, vehículo conectado, YOLOv9, Inteligencia Artificial

Abstract

In this document, we present a technical-economic report on the feasibility of deploying a 4G network along the A-62 highway to offer connected vehicle services.

First, we study the coverage along the entire highway through simulation with Xirio to then deploy the necessary base stations to achieve optimal coverage at all points of the highway.

Next, we conduct a study on the 4G architecture, designing and selecting the most suitable hardware components for the core structure and base stations for our case. Additionally, we study the deployment of optical fiber to connect the two parts of the network. Once this is done, we conduct the economic evaluation of the entire deployment.

Then, we explore the available alternatives for the artificial intelligence part, studying several image detection models and choosing the most suitable model for our requirements.

Finally, we develop a demonstrator to apply all the knowledge acquired during the study and deploy the service on a small scale using an Amazon DeepRacer vehicle, a BladeRF 2.0 xa9, and a programmable SIM card.

In the demonstrator, we set up the network core on an Ubuntu 20.04 operating system using srsRAN software and use the Blade as a base station. We register the SIM card in the core and connect it to the car via a Huawei modem.

We have recorded and created a dataset to train a model with yolov9, to apply it in the core and detect images. Once everything is connected, we record images with the car and send them through the network to the core, where this model runs and detects the signals. After this, we send the detection result back to the car and display the result on the screen.

Keywords

4G, LTE, eNodeB, EPC, UE, antenna, core, coverage, Xirio, connected vehicle, YOLOv9, Artificial Intelligence

Índice general

1. Introducción	1
1.1. Contexto y motivación	1
1.2. Objetivos	2
1.3. Fases y métodos	2
1.4. Estructura de la memoria	3
2. Arquitectura 4G	4
2.1. Evolución de las redes móviles	4
2.2. Red de Acceso por Radio (RAN)	5
2.3. Red principal (Core Network)	5
2.4. Planos de control y usuario	6
2.5. Interfaces de red	6
2.6. Optimización y eficiencia	8
2.7. Impacto de 4G LTE	8
3. Infraestructura 4G	10
3.1. Diseño de la red de acceso	10
3.2. Diseño del core de la red 4G	15
3.3. Capacidad de una celda 4G	20
3.4. Estudio del tráfico de la carretera	22
3.5. Medición de las características de la red	24
4. Inteligencia artificial (IA)	30
4.1. Inteligencia artificial en vehículos autónomos	30
4.1.1. Componentes clave de la IA en vehículos autónomos	30
4.1.2. Técnicas de IA utilizadas	31
4.1.3. Aplicaciones de la IA en vehículos autónomos	31
4.2. Ética y seguridad en vehículos autónomos	32
4.2.1. Ética en la toma de decisiones	32
4.2.2. Seguridad y fiabilidad	32
4.3. YOLOv9: You Only Look Once	33
4.3.1. Evolución de YOLO	33
4.3.2. Componentes clave de YOLOv9	34
4.3.3. Funcionamiento de YOLOv9	35
4.4. Innovaciones y futuro de la IA en vehículos autónomos	36
4.4.1. Tecnologías emergentes	36

<i>ÍNDICE GENERAL</i>	v
4.4.2. Impacto social y económico	36
5. Informe económico	37
5.1. Costes	37
6. Conclusiones y líneas futuras	40
6.1. Conclusiones	40
6.2. Líneas futuras	41
Anexo I. Desarrollo del demostrador	43
Paso 0. Materiales necesarios	43
Paso 1. Instalación y desarrollo del ENodeB y core de la red 4G	44
A. UHD	48
B. SoapySDR	49
C. BladeRF	49
D. SoapyBladeRF	50
E. ZeroMQ	51
F. Software srsRAN 4G	51
Paso 2. Configuración de la SDR	52
Paso 3. Conexión con el coche y configuración	55
Paso 4. Elección del algoritmo de IA y entrenamiento	58
A. Escoger algoritmo de identificación de señales	58
B. Etiquetado de Imágenes	59
C. Entrenamiento del modelo	59
D. Resultados del entrenamiento	61
E. DAFO Yolov9	62
Paso 5. Implementación del algoritmo de IA en la red 4G	63
Paso 6. Funcionamiento del demostrador	65
A. Archivo <i>detect.py</i>	65
B. Mandar órdenes al coche	67
Anexo II. Simulador Xirio-Online	68
A. Banda de frecuencias	69
B. Extremos: Definición de los sectores	70
C. Extremos: Configuración del terminal	72
D. Parámetros de cálculo	73
E. Área de cálculo y rangos de colores	73
F. Estudio multitransmisor: Simulación	74
G. Extra: Análisis de los UE	75

Índice de figuras

1.	Arquitectura LTE RAN	5
2.	Arquitectura EPC	6
3.	Arquitectura 4G mediante la interfaz X2	7
4.	Arquitectura 4G mediante la interfaz S1	7
5.	Estaciones base ya desplegadas (Infoantenas).	11
6.	Cobertura de las antenas instaladas en las estaciones ya desplegadas.	11
7.	Mapa de todas las estaciones base (en rojo las nuevas).	12
8.	Mapa de antenas desplegadas.	13
9.	Cobertura total en el tramo de autovía.	13
10.	Alcance de cobertura de un UE.	14
11.	Posibles arquitecturas para las estaciones base: distribuida (izquierda), centralizada (centro) y virtualizada (derecha).	16
12.	Fibra oscura de Reintel cerca de la A-62	18
13.	Mapa del despliegue final con los elementos más importantes	19
14.	Tabla relación CQI con modulación empleada.	23
15.	Parámetros de medida en la interfaz web del módem.	24
16.	Niveles de señal para los distintos indicadores.	25
17.	Niveles de señal con visión directa y visión no directa.	26
18.	Señal en tiempo real en Netmonitor.	26
19.	Medición de latencia y jitter con ping	27
20.	Medida del throughput de subida y de bajada	28
21.	jitter y pérdidas para un flujo de 30Mbps	29
22.	Medida del jitter en la subida	29
23.	Ecuación del cuello de botella informativo	34
24.	Arquitectura PGI	35
25.	DAFO software srsRAN.	45
26.	DAFO software Open5Gs.	45
27.	DAFO software OpenAirInterface.	45
28.	Árbol de decisión del tipo de instalación srsRAN [29].	46
29.	Detección de la bladeRF.	50
30.	Configuración del fichero <i>epc.conf</i>	53
31.	Configuración del fichero <i>user_db.csv</i>	53
32.	Configuración del fichero <i>enb.conf</i> (parte 1).	54

33.	Configuración del fichero <i>enb.conf</i> (parte 2).	54
34.	Funcionamiento eNodeB.	55
35.	Funcionamiento EPC.	56
36.	Fichero de configuración <i>vehicle.conf</i> .	57
37.	Vehículo del laboratorio conectado a la red.	57
38.	MQTT explorer.	58
39.	Detección de objetos con YOLO	59
40.	1ª Parte del Script	60
41.	2ª Parte del Script	60
42.	3ª Parte del Script	60
43.	4ª Parte del Script	60
44.	5ª Parte del Script	61
45.	Matriz de confusión	61
46.	Importaciones y configuración inicial	63
47.	Servidor y funciones auxiliares	64
48.	Bloque principal del código	64
49.	Bucle principal de procesamiento	65
50.	Resultado de la detección de señales	65
51.	Resultados en formato json	66
52.	Escribir resultados de la detección	66
53.	Imprimir resultados de la detección	66
54.	Función para enviar los controles al coche	67
55.	Recogemos la señal detectada	67
56.	Establecemos los controles y lo enviamos al coche	67
57.	Creación de una nueva cobertura individual.	68
58.	Menú de configuración de un nuevo estudio de cobertura individual.	69
59.	Menú de configuración de la banda de frecuencias.	70
60.	Menú de configuración de un sector.	71
61.	Tabla de especificaciones de los modelos de antenas Ubunquiti [33].	71
62.	Menú de configuración de la antena en Xirio-Online.	72
63.	Azimut vertical y horizontal de la antena.	72
64.	Menú de configuración del terminal.	73
65.	Sistema de colores de las coberturas.	73
66.	Menú de configuración del estudio de cobertura multitrasmisor.	74
67.	Alcance de las antenas de los UE en distintos puntos de la autovía.	78

Índice de tablas

1.	Ancho de banda por usuario según el tipo de modulación.	22
2.	Desarrollo de los gastos asociados a las estaciones base	38
3.	Desarrollo de los gastos asociados al core	38
4.	Desarrollo de los gastos asociados a la red de distribución	39
5.	Azimet estaciones base.	75
6.	Resultados RSRP usando 5MHz de ancho de banda.	76
7.	Resultados RSRP usando 1.4MHz de ancho de banda.	77
8.	Resultados mejor servidor y solapamiento con 5MHz de ancho de banda. . . .	77

Capítulo 1

Introducción

Este primer capítulo pretende servir al lector como una introducción al proyecto que se ha desarrollado y el cuál se describe en este documento a continuación.

1.1. Contexto y motivación

La evolución de la tecnología es cada día más rápida, siendo el último avance la Inteligencia Artificial (IA), la cual se ha extendido por todo el mundo en los últimos años. Hoy en día, las personas utilizan la Inteligencia Artificial como una herramienta más para casi cualquier ámbito de su vida, tanto laboral como personal.

El sector del automóvil también ha apostado por esta tecnología, siendo la conducción totalmente autónoma uno de los objetivos a corto y medio plazo de la mayoría de las empresas automovilísticas. Para poder conseguir este objetivo, la Inteligencia Artificial cobra un gran papel, ya que se utiliza con el objetivo de analizar todo el entorno que rodea a un vehículo y tomar decisiones, como por ejemplo minimizar el número de incidentes en la carretera, los costes de los accidentes o brindar de comodidad al usuario final mientras éste se desplaza a otro lugar.

Por otro lado, esto no es tarea fácil debido a que al entrenar dicha tecnología se puede pecar de introducir un sesgo, distinto en cada cultura. ¿Tiene que ser una conducción más segura para el conductor o para el resto de los transeúntes de la vía? ¿Cómo de rápido es capaz de reaccionar una máquina? En caso de fallo, ¿quién es el culpable? Estas son algunas de las preguntas que hoy en día no tienen respuesta dado que se tratan de una cuestión moral.

La IA en conducción autónoma puede realizarse de 2 maneras principalmente: mediante Computación en la Nube o Cloud Computing, o mediante Computación en el Borde o Edge Computing. Ambas tienen sus ventajas y sus desventajas (latencia, mejora, actualización de algoritmos, optimización...), siendo el Edge Computing la última tendencia.

El objetivo final del proyecto será presentar la razón por la que se ha escogido un tipo de

CAPÍTULO 1. INTRODUCCIÓN

computación u otro, así como la tecnología de telecomunicaciones que se ha empleado para implementarla.

1.2. Objetivos

EL objetivo de este proyecto consiste en la propuesta de una infraestructura real en el tramo de autovía A-62 entre los kilómetros 158 y 231 para poder desplegar un servicio de vehículo conectado a los usuarios de la vía, y desarrollar un pequeño demostrador en el entorno de laboratorio.

Con respecto a la aplicación de servicio de vehículo conectado, se desplegará un servicio en tiempo real de vídeos con detección de imágenes. Dicho video será grabado por el vehículo que transite por la carretera, se enviará a través de la red hasta el core a 20FPS, en donde se encuentra el sistema de detección de señales hecho mediante inteligencia artificial, y el core devolverá el video al coche indicando dónde se encuentran las señales y qué tipo de señales son.

Con respecto a la arquitectura e infraestructura, se explorará la tecnología 4G LTE y se justificará el diseño de toda la red basándose en el uso de la de las bandas de frecuencia entre otras cosas.

Por último, con respecto al demostrador, se desplegará una pequeña red 4G definida por software en el entorno de laboratorio de la Escuela Técnica superior de Ingenieros de Telecomunicación en Valladolid, en la que se conectará un vehículo teledirigido con una cámara integrada. El vehículo grabará un vídeo que será enviado al core a través de dicha red, para posteriormente el core analizar una de cada 5 frames del vídeo que es enviado y mostrará por pantalla el resultado. Una vez obtenido el resultado, enviará el coche la instrucción correspondiente al significado de la señal detecta (p.ej. si es una señal de giro obligatorio a la derecha, entonces el coche girará a la derecha).

1.3. Fases y métodos

Para el desarrollo de este proyecto, se han seguido los siguientes pasos:

1. Toma de requisitos que debe cumplir el proyecto, exigidos por el enunciado de la licitación planteada.
2. Análisis de las distintas herramientas y tecnologías actuales que se pueden utilizar tanto como para el demostrador como para el tramo de carretera.
3. Diseño de las funcionalidades que es necesario implementar en ambos casos.
4. Desarrollo del demostrador en el entorno de laboratorio y mejoras gracias a aportaciones de nuevas ideas.

5. Desarrollo y entrenamiento del sistema de inteligencia artificial para la detección de señales.
6. Prueba del demostrador por parte de los desarrolladores y evaluación de su rendimiento y funcionamiento acorde a los requisitos inicialmente planteados, mediante herramientas de medición adecuadas.
7. Desarrollo y justificación de toda la infraestructura 4G a desarrollar para hacer viable el servicio que se desea desplegar.
8. Elaboración de la memoria del proyecto.

A lo largo de esta memoria se encuentra una descripción en detalle de cada uno de los pasos seguidos y los métodos que se emplean para poder cumplir con el objetivo del proyecto.

1.4. Estructura de la memoria

A continuación, se indica la estructura de esta memoria correspondiente al proyecto desarrollado. En dicha memoria, se podrán encontrar cinco capítulos, siendo el primero una pequeña introducción y descripción del proyecto que se ha llevado a cabo.

En el **capítulo 2** se presenta la arquitectura de la tecnología que se va a utilizar en el proyecto, en este caso 4G LTE. Se describe en un primer lugar a modo introductorio la evolución de la redes móviles, seguido de un análisis más en profundidad de la arquitectura, para finalmente analizar el impacto de la tecnología 4G LTE.

En el **capítulo 3** se exponen todos los componentes software y hardware necesarios que la tecnología 4G LTE necesita para poder ser desplegada y utilizada. Además, se presenta una propuesta de diseño de la infraestructura que se prevee desplegar para brindar servicio a los usuarios.

En el **capítulo 4** se explica como está aplicada la tecnología de la Inteligencia Artificial en el sector de los coches autónomos, y la evolución que ha ido teniendo a lo largo del tiempo. También, se expone el modelo YOLO para el reconocimiento y detección de objetos, además de aspectos como, la ética de los vehículos autónomos o la mejora y líneas futuras de esta innovación tecnológica.

En el **capítulo 5** se presenta el informe económico que supone el desarrollo y puesta en marcha del proyecto. Asimismo, se incluyen todos los precios de los alquileres de parcelas, elementos hardware y mano de obra final.

Finalmente, en el **capítulo 6** se hace un pequeño resumen de todo lo que se ha mencionado en la memoria y se realiza una conclusión final sobre el proyecto. También se incluyen líneas futuras para la seguir con la evolución de la aplicación en un futura, y el cierre final junto a las referencias bibliográficas al final de la memoria.

Capítulo 2

Arquitectura 4G

La tecnología de comunicaciones móviles ha experimentado una evolución significativa a lo largo de las últimas décadas, pasando por varias generaciones que han transformado radicalmente la manera en que nos comunicamos y accedemos a la información. La Cuarta Generación (4G) de tecnologías móviles, y en particular la Long-Term Evolution (LTE), representa un salto cualitativo en términos de velocidad, eficiencia y capacidad en comparación con sus predecesores.

2.1. Evolución de las redes móviles

Antes de sumergirnos en la arquitectura específica de 4G LTE, es útil considerar brevemente la evolución de las redes móviles [1]:

- **1G (Primera Generación):** fue lanzada por NTT en Japón en 1979, introdujo la telefonía móvil analógica, pero con malas comunicaciones de voz y ninguna seguridad ya que las llamadas de voz se reproducían en las torres de radio.
- **2G (Segunda Generación):** fue introducida en el año 1992, y es la que se encargó de hacer la transición a comunicaciones digitales, permitiendo servicios como SMS, roaming internacional y mayor calidad de voz.
- **3G (Tercera Generación):** surgió en el año 2000, y proporcionó acceso a Internet y servicios multimedia, a través de un aumento de las tasas de datos, mayor capacidad de voz y datos y soporte a diversas aplicaciones a bajo coste. Los datos se enviaban a través de la tecnología de una tecnología llamada Packet Switching y las llamadas de voz se traducían mediante conmutación de circuitos.
- **4G LTE (Cuarta Generación):** está basado totalmente en IP y surge como una respuesta a la creciente demanda de datos y servicios de alta velocidad, habilitando una amplia gama de aplicaciones que van desde la transmisión de video en alta definición hasta la realidad aumentada y virtual.

2.2. Red de Acceso por Radio (RAN)

La RAN es responsable de la comunicación inalámbrica entre el dispositivo móvil y la red principal. En 4G LTE, la RAN se ha rediseñado para mejorar la eficiencia y la velocidad de transmisión de datos. Los componentes clave de la RAN son [2]:

- **User Equipment (UE):** es el módem 4G LTE UE implementado íntegramente en software, que funciona como una aplicación en un sistema operativo estándar basado en Linux y se conecta a cualquier red LTE, proporcionando una interfaz de red estándar con conectividad móvil de alta velocidad.
- **evolved Node B (eNodeB):** es la estación base LTE eNodeB implementada íntegramente en software, responsable de la transmisión y recepción de señales de radio, control de recursos de radio, y la ejecución de funciones de gestión de movilidad. Al ejecutarse como una aplicación en un sistema operativo estándar basado en Linux, eNodeB se conecta a cualquier red central LTE (EPC) y crea una célula LTE local. A diferencia de las estaciones base en tecnologías anteriores, el eNodeB en LTE integra tanto las funciones de la estación base como las de los controladores de radio.

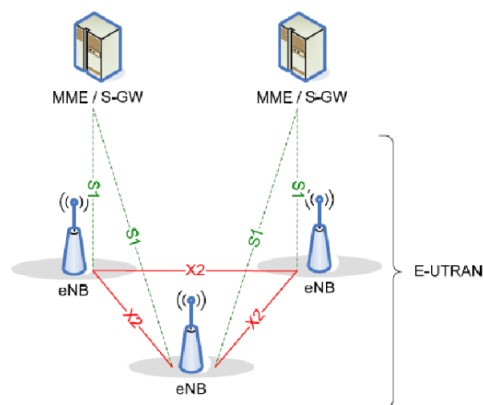


Figura 1: Arquitectura LTE RAN

2.3. Red principal (Core Network)

La red principal en 4G LTE se denomina Evolved Packet Core (EPC) y es una implementación ligera de una red central LTE completa, que está diseñada para proporcionar conectividad IP de extremo a extremo. Los componentes fundamentales de la EPC incluyen [3]:

- **Home Subscriber Server (HSS):** base de datos que almacena información sobre los suscriptores, incluyendo sus identificadores de usuario, la clave o los límites de uso, entre otras muchas cosas. Es el encargado de autenticar y autorizar el acceso del usuario a la red.

CAPÍTULO 2. ARQUITECTURA 4G

- **Mobility Management Entity (MME):** es el principal elemento de control de la red. Gestiona la movilidad y el control de las conexiones. Es responsable de avisar a los UE en modo inactivo y del establecimiento, mantenimiento y liberación de las sesiones de conexión.
- **Serving Gateway (SGW):** es la principal pasarela del plano de datos, ya que actúa como el punto de anclaje para la movilidad de la capa de usuario dentro de la red de acceso. Funciona como un router IP y ayuda a establecer sesiones GTP entre el eNodeB y el P-GW.
- **Packet Data Network Gateway (PGW):** es el punto de contacto de la red LTE con las redes de datos externas como Internet. Realiza funciones de asignación de los parámetros de calidad de servicio (QoS) a las sesiones de los abonados.

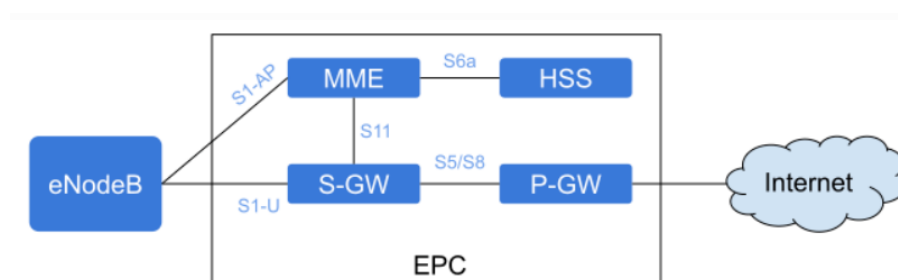


Figura 2: Arquitectura EPC

2.4. Planos de control y usuario

La arquitectura de 4G LTE separa el plano de control y el plano de usuario mediante interfaces abiertas [4]:

- **Plano de Control:** maneja las interfaces y los protocolos de señalización previos necesarios para la gestión de la red y las sesiones de comunicación, como la autenticación, la configuración de sesiones y la gestión de movilidad, entre otras cosas para establecer los túneles GTP-U.
- **Plano de Usuario:** encargado de generar y transportar los datos del usuario, como tráfico de Internet, llamadas VoIP y transmisión de video entre el eNodeB, el S-GW y el P-GW, a través de túneles GTP-U en las interfaces S1-U y S5.

2.5. Interfaces de red

Para facilitar la comunicación entre los diferentes componentes, LTE utiliza varias interfaces estandarizadas [4]:

- **X2 Interface:** interfaz entre eNodeBs que permite la comunicación directa entre ellos, y que facilita la gestión de movilidad y la transferencia de datos sin interrupciones durante el handover, transfiriendo los paquetes de usuario pendientes sin la necesidad de intervención del MME.

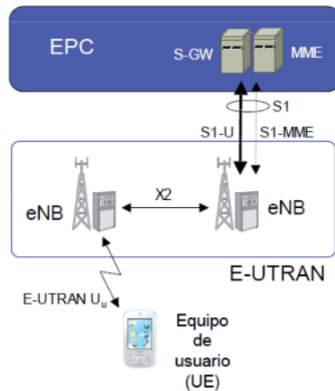


Figura 3: Arquitectura 4G mediante la interfaz X2

- **S1 Interface:** interfaz que conecta E-UTRAN (eNodeB) con la EPC (MME para el control de mensajes, o S-GW para el control de paquetes de usuario) para completar el handover, comunicando la celda de servicio con la celda de destino a través del MME y así, poder controlar el handover. Se compone de dos subinterfases: S1-MME, que transporta la información del plano de control (autenticación, la configuración del portador, el traspaso y la paginación) y S1-U, que transporta la información del plano del usuario (paquetes de datos, voz y vídeo). Esta interfaz se utiliza en el caso de que no exista conexión X2 entre los eNodeBs origen y destino, exista conexión X2, pero no se le permita realizar Handovers o la preparación del Handover falla entre ambas celdas.

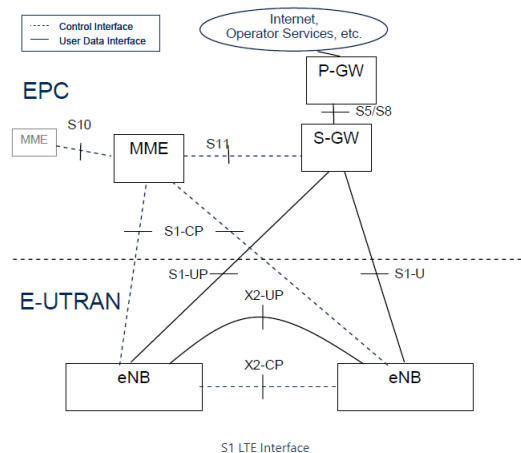


Figura 4: Arquitectura 4G mediante la interfaz S1

2.6. Optimización y eficiencia

LTE ha sido diseñado para optimizar la eficiencia del espectro y la latencia. Algunos de los fundamentos más importantes de nivel físico implementados para alcanzar mayores niveles de capacidad y eficiencia son los siguientes [5]:

- **OFDMA (Orthogonal Frequency Division Multiple Access):** utilizada en el enlace descendente del sistema LTE, ofrece la posibilidad de que los diferentes símbolos modulados sobre las subportadoras pertenezcan a usuarios diferentes. De esta manera, permite la transmisión simultánea de múltiples señales en diferentes frecuencias, mejorando la eficiencia del espectro. Esta técnica se consigue dividiendo el canal en un conjunto de subportadoras que se reparten en grupos en función de la necesidad de cada uno de los usuarios, consiguiendo diversidad multiusuario, diversidad frecuencial, robustez frente al multitrayecto, flexibilidad en la banda asignada y elevada granularidad en los recursos asignables.
- **SC-FDMA (Single Carrier Frequency Division Multiple Access):** técnica de modulación híbrida utilizada en el enlace ascendente, que combina la robustez frente a la propagación multicamino y flexibilidad de ubicación de las subportadoras propia de los sistemas OFDM con menor PAPR propia de las modulaciones con portadora única. Reduce el consumo de energía de los dispositivos móviles y el nivel de los picos de potencia en el dominio del tiempo pero aumenta la potencia radiada fuera de banda en el dominio frecuencial.
- **MIMO (Multiple Input Multiple Output):** sistema que utiliza múltiples antenas para la transmisión y recepción de datos a una tasa elevada, dividida en varias tramas más reducidas. Cada una de esas tramas se modula y se transmite a través de una antena diferente en un momento determinado, permitiendo la mejora de la capacidad y la cobertura.

2.7. Impacto de 4G LTE

La adopción de 4G LTE ha tenido un impacto profundo en términos de capacidad y cobertura:

- **Velocidades de Datos:** ha permitido velocidades de datos significativamente más altas, facilitando servicios como streaming de video en alta definición y juegos en línea.
- **Latencia:** uno de los requisitos fundamentales de la arquitectura del sistema LTE es la reducción de la latencia para mejorar la experiencia del usuario en aplicaciones sensibles al tiempo, como videollamadas y aplicaciones de realidad aumentada. Esto se consigue con la arquitectura plana basada en el protocolo IP operando completamente en modo paquete.
- **Internet de las Cosas (IoT):** la capacidad y la eficiencia de LTE han habilitado la conectividad de una masiva gama de dispositivos IoT, desde sensores industriales hasta dispositivos domésticos inteligentes.

CAPÍTULO 2. ARQUITECTURA 4G

En conclusión, la arquitectura de 4G LTE representa un avance significativo en la tecnología de comunicaciones móviles, ofreciendo mejoras en velocidad, eficiencia y capacidad. A través de una combinación de componentes innovadores y optimizaciones de red, LTE ha transformado la forma en que nos comunicamos y accedemos a la información, estableciendo las bases para futuras generaciones de tecnología móvil. La comprensión detallada de su arquitectura es crucial para apreciar las capacidades y el impacto de esta tecnología en nuestra vida cotidiana.

Capítulo 3

Infraestructura 4G

Vista la arquitectura de una red 4G, continuamos con el análisis y descripción de la infraestructura de toda la red que se va a desplegar para el despliegue del servicio. Para comenzar, es necesario saber que los componentes esenciales que necesita una red 4G para que sea completamente operativa y funcional. Posteriormente se realiza un pequeño estudio básico sobre la capacidad de una celda 4G para extrapolarlo al resto de la red, seguido de la realización un estudio sobre el tráfico que circula por la carretera y las mediciones reales que se deben hacer para saber el rendimiento de la red. Por último, se presenta un diseño de infraestructura orientado al servicio de análisis de video que se va a desplegar.

3.1. Diseño de la red de acceso

Dado que nuestro objetivo es el despliegue de nuestra aplicación en el tramo de autovía, primero debemos averiguar la infraestructura de telecomunicaciones ya desplegada en esa área. Para ello, hemos visitado la página web del Ministerio para la Transformación Digital y de la Función Pública llamada “Infoantenas” [6]. En esta página web se muestran todas las estaciones base que están desplegadas hasta la actualidad, así como las antenas que se encuentran instaladas en dichas estaciones base, el rango de frecuencias a las que están configuradas y el grado de dirección a la que apuntan con respecto al Azimut. Una vez ya sabemos dónde se ubican todas las estaciones base en el área deseada (Figura 5), debemos decidir si vamos a emplear dichas estaciones o, por el contrario, desplegar las nuestras. Para ello, debemos realizar un estudio exhaustivo del terreno, así como elegir una buena antena de radiación con una buena ganancia en nuestra frecuencia de trabajo y el área que dé cobertura que abarca cada una.

Para realizar las simulaciones de nuestra área de cobertura, hemos decidido utilizar la versión gratuita y de navegador de Xirio-Online. Xirio-Online es una aplicación de software que se utiliza para la planificación y simulación de redes de comunicaciones inalámbricas. La configuración del entorno de simulación puede observarse en el Anexo II de la memoria.

CAPÍTULO 3. INFRAESTRUCTURA 4G

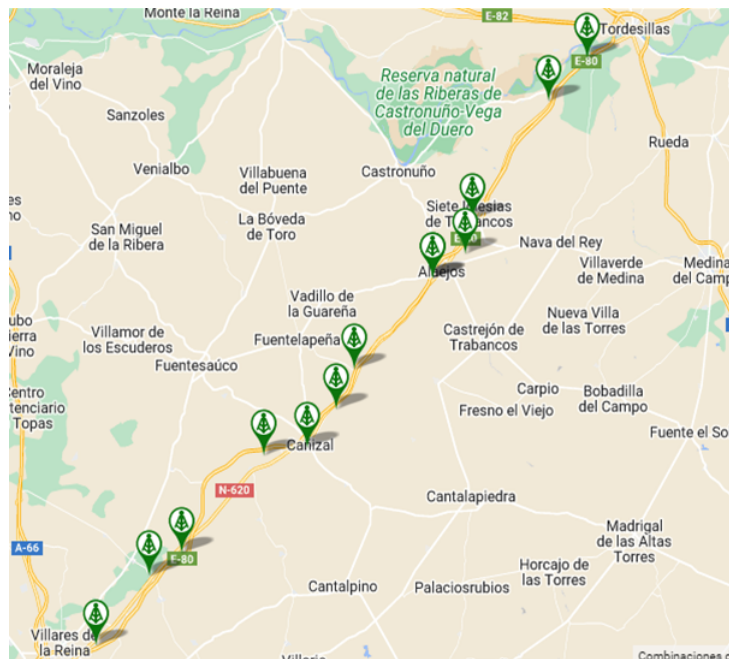


Figura 5: Estaciones base ya desplegadas (Infoantenas).

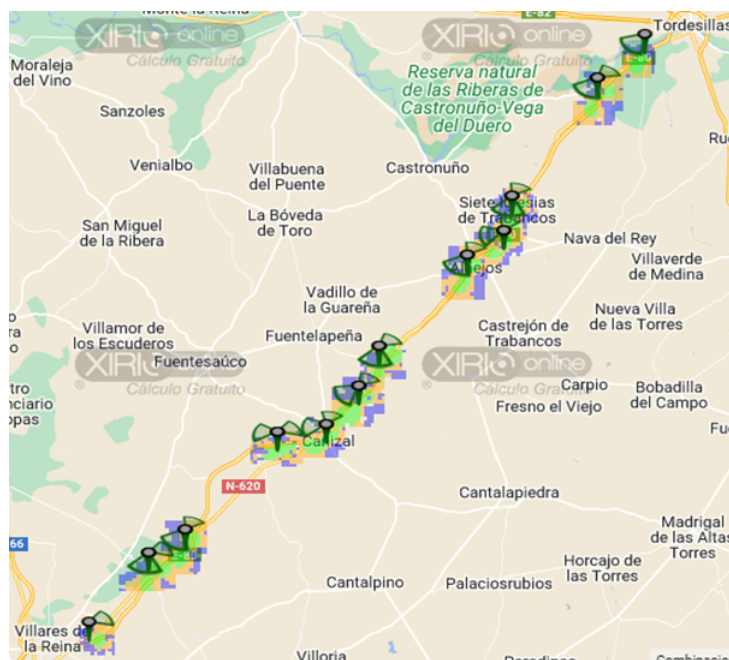


Figura 6: Cobertura de las antenas instaladas en las estaciones ya desplegadas.

Analizada la situación, nos dimos cuenta que simplemente haciendo uso de las estaciones que ya están desplegadas (es posible alquilar el uso de cada una e instalar nuestras antenas a una cierta altura) no era suficiente. Dado que tenemos que desplegar el servicio en la autovía A-62 entre los kilómetros 158 y 231, quiere decir que debemos cubrir un total de 73Km. Dado que tenemos a nuestra disposición un rango de frecuencias de banda 7, muy elevado para la tecnología LTE, y el entorno es externo y no interno, existía la duda sobre hasta qué distancia llegaría la señal

CAPÍTULO 3. INFRAESTRUCTURA 4G

con suficiente potencia para que el receptor pudiese recuperar la información transmitida. Ante esta situación, decididos averiguarlo consultando a un experto de la empresa Movistar sobre el tema. El experto, Ángel Alves [7] nos indicó que las antenas que emiten a dicha frecuencia (22670.5MHz) no alcanzan más de 1,5km en condiciones buenas y al exterior sin una ganancia muy alta, por lo que el factor limitante serán las antenas instaladas en los vehículos. Dicha teoría es respaldada con numerosos estudios como por ejemplo: “Comparison of LTE Coverage Areas in Three Frequency Bands” en el que se concluye con que este tipo de frecuencias son ideales para trabajar con ellas en centros comerciales debido a que sus celdas tienen un radio de aproximadamente 1km [8]. Sin embargo, en dicho estudio también menciona el alcance de otras frecuencias más utilizadas, como la banda de 900MHz, la cual tiene un alcance de hasta 3km, siendo necesarias la mitad de antenas.

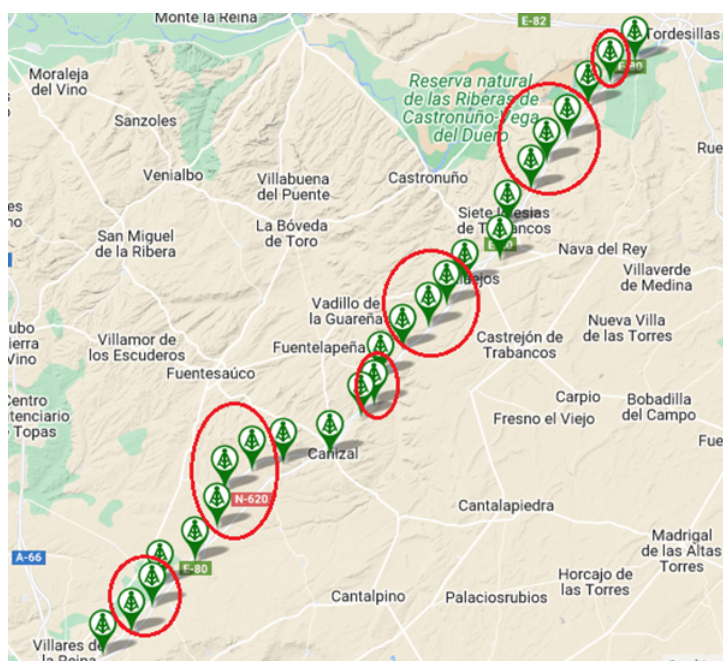


Figura 7: Mapa de todas las estaciones base (en rojo las nuevas).

Como consecuencia de este comportamiento, cada antena debe situarse a no más de 1,5km de distancia. Por lo tanto, las estaciones base en donde se instalarían las antenas deberían estar a unos 3km de distancia aproximadamente, con un par de antenas direccionales la mayoría de ellas apuntando a la carretera de manera que su haz principal abarque la mayor parte del espacio. Dado que al instalar las antenas en las estaciones base ya desplegadas no se cubre con una calidad de señal suficiente toda la superficie deseada (Figura 6), se decidió que la mejor situación era aprovechar gran parte de la infraestructura existente y, a mayores, instalar y desplegar una estación base cada 3km aproximadamente. Como consecuencia directa de esta decisión, son necesarias un total de 13 estaciones base a mayores para cubrir los 73km de autovía. Las ubicaciones de las estaciones base que hemos decidido junto a las ya disponibles pueden verse en la Figura 7 (las nuestras están rodeadas con un círculo rojo).

Las nuevas estaciones base posicionadas estratégicamente en función del terreno, la distancia, la altura y los edificios de los alrededores. En total, suman un total de 25 estaciones base

CAPÍTULO 3. INFRAESTRUCTURA 4G

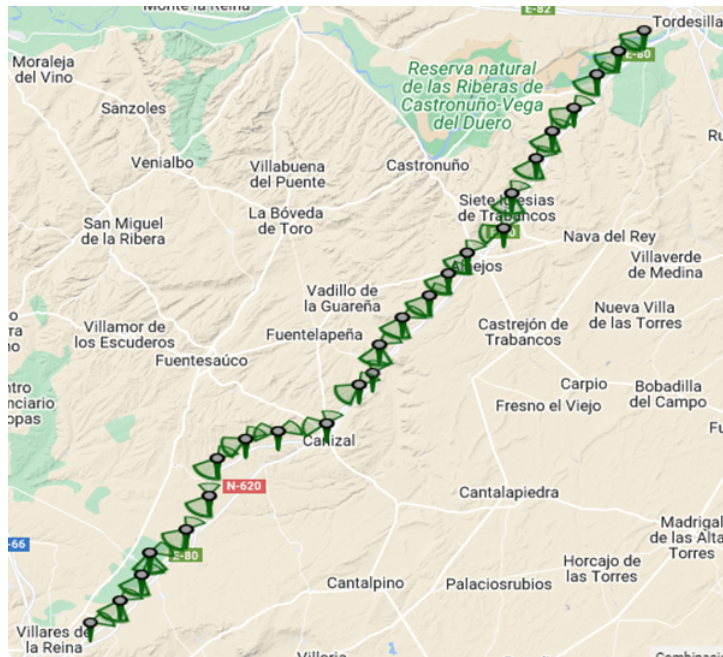


Figura 8: Mapa de antenas desplegadas.

disponibles para instalar antenas, lo que se traduce en un total de 47 antenas directivas, cada una de ellas con su celda correspondiente (modulación en la mayoría del tramo a 16-QAM o 64-QAM).

La simulación llevada a cabo en Xirio que muestra los resultados de los estudios de cobertura de cada antena puede verse en el Anexo II. Si analizamos los resultados de la señal RSRP (ver Figura 9), es posible observar cómo queda cubierto toda el área de carretera con una calidad de señal superior a -80dB, asegurando una calidad de servicio muy buena. Por otro lado, si analizamos la cobertura que tienen las antenas de los receptores en varios puntos del tramo de

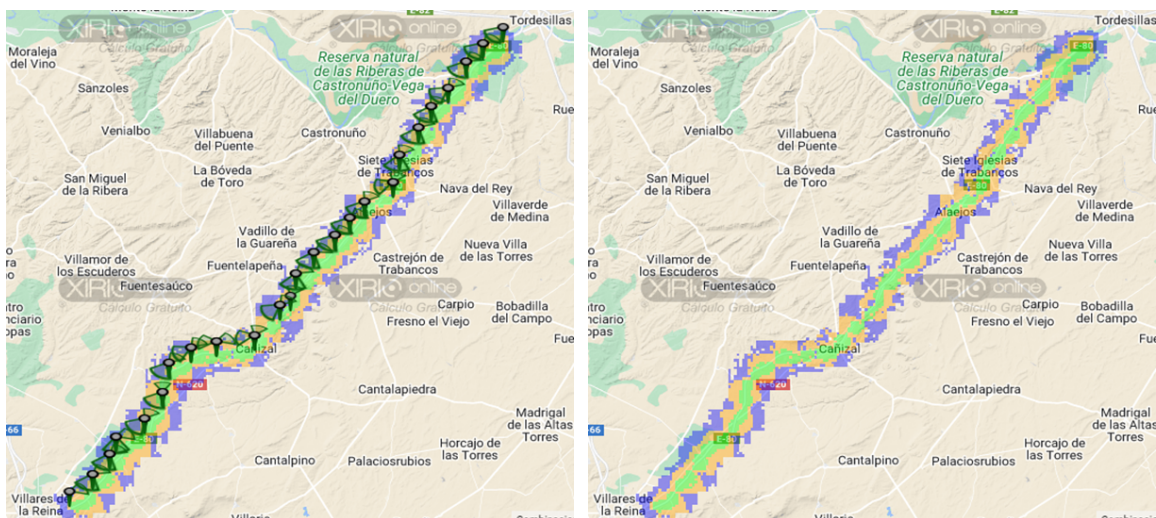


Figura 9: Cobertura total en el tramo de autovía.

CAPÍTULO 3. INFRAESTRUCTURA 4G

autovía (Figura 10), es fácil percatarse que al tratarse de antenas omnidireccionales de 2dBi de ganancia y trabajar a frecuencias de banda 7, su alcance es muchísimo menor, siendo este de aproximadamente 1,5km (tal y como se ha indicado anteriormente). Por lo tanto, podemos concluir que el factor limitante no es tanto la potencia de las antenas que conforman la red de acceso, sino de las antenas de los equipos de usuario, ya que no son tan potentes ni se encuentran a grandes alturas.

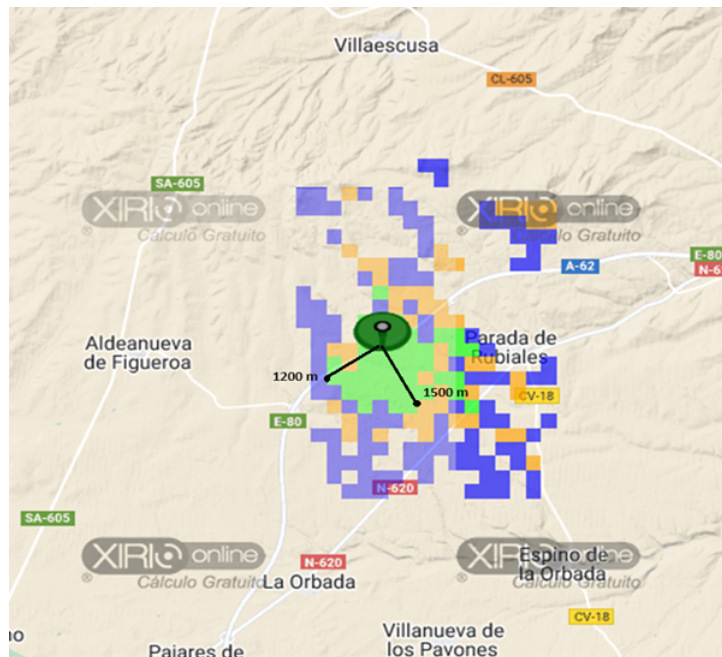


Figura 10: Alcance de cobertura de un UE.

Analizada la situación en la que nos encontramos, podemos resaltar que con la estructura que se ha planteado la red ofrece una muy buena calidad de servicio a cada usuario cuando las condiciones son ideales o casi ideales.

Para finalizar con el diseño de la infraestructura, nos preguntamos cual es el máximo de usuarios a los que podemos dar servicio. Tal y como se va a exponer en la sección 3.3, en cada celda hay una capacidad máxima de 22 usuarios y mínima de 7 por celda. Por lo tanto, como tenemos 47 antenas en total, suponiendo que se encuentra una gran cantidad de vehículos en todo el tramo de autovía, como máximo puede darse servicio a 1.034 usuarios al mismo tiempo teniendo una calidad de señal muy buena y a un mínimo de 329 usuarios con una calidad bastante peor.

Cabe mencionar que los usuarios no se encontrarán estáticos al utilizar la red, sino que se desplazarán a lo largo de la misma y, por tanto, deberán de cambiar de conexión entre un eNodeB y otro conforme vaya empeorando la calidad de señal. Dicho procedimiento se denomina handover. El handover es un proceso que asegura la continuidad del servicio una sesión de datos cuando un usuario se mueve de una célula a otra. Este procedimiento es esencial para mantener la calidad de la experiencia del usuario en la tecnología 4G LTE. El procedimiento de handover es el siguiente:

1. **Mediciones de la Calidad de Señal:** El UE mide continuamente la calidad de la señal de su célula actual y de las células vecinas. Estas mediciones incluyen parámetros como la potencia de la señal recibida (RSRP) y la calidad de la señal recibida (RSRQ).
2. **Decisión de Handover:** Basado en las mediciones reportadas por el UE, el eNodeB decide cuándo es necesario realizar un handover. Esta decisión se basa en varios factores, como la degradación de la señal actual o la mejora de la señal de una célula vecina.
3. **Preparación del Handover:** El eNodeB fuente prepara el handover enviando un mensaje al eNodeB destino. Este mensaje incluye la identidad del UE y otros parámetros necesarios para la transferencia.
4. **Conexión con el eNodeB Destino:** El UE establece una nueva conexión con el eNodeB destino y empieza a comunicarse con él. El eNodeB destino confirma la recepción del UE y establece el contexto necesario para la continuidad de la sesión.

3.2. Diseño del core de la red 4G

El despliegue de la infraestructura de toda la red parte del estudio realizado en el Anexo II, en el que realizamos la simulación de varias antenas colocadas estratégicamente para dar cobertura en todo el tramo de la autovía. El resultado de la simulación nos ha dado como resultado la colocación de 47 antenas en 25 estaciones base, 13 de las cuales las colocaremos a mayores y las otras 12 aprovecharemos las que ya estaban montadas a lo largo de la autovía. Esto ayudará a abaratar los costes.

En el despliegue de la red 4G intervienen principalmente tres partes bien diferenciadas. Estas tres partes son, en primer lugar, todas y cada una de las estaciones base, en segundo lugar, el núcleo de la red y, por último, la red de conexión entre ambos.

A continuación, detallaremos las posibles opciones para el despliegue de cada una de las partes y elegiremos la que consideremos más adecuadas para nuestro proyecto.

Comenzaremos por las estaciones base. La arquitectura de las estaciones base ha evolucionado considerablemente durante los últimos años. Comenzó siendo una arquitectura distribuida con las primeras tecnologías móviles, después con el aumento de las estaciones de las redes de telefonía, evolucionó a arquitecturas centralizadas, y en los últimos años, los proveedores están moviéndose cada vez más hacia la arquitectura conocida como vRAN.

En la RAN distribuida, la RRU y la BBU están ubicadas en el mismo lugar. La RAN distribuida es el tipo más tradicional de estación base que generalmente se despliega para proporcionar cobertura de área amplia junto con el sistema de antenas montado en una torre. La ventaja de esta configuración es que la distancia relativamente corta entre la RRU y la BBU no plantea grandes desafíos para el fronthaul y simplifica la coordinación de las operaciones de la RRU y la BBU. Por otro lado, la desventaja es que cada estación base necesita estar equipada con ambas unidades, lo que incrementa el costo del despliegue. Esta arquitectura se utilizaba tradicionalmente en las estaciones 2G, ya que entonces se empleaban las bandas de frecuencia

CAPÍTULO 3. INFRAESTRUCTURA 4G

más bajas para transmitir principalmente tráfico de voz. Por lo tanto, era posible establecer estaciones base que pudieran cubrir un amplio rango con estas bandas de espectro más bajas, lo que requería menos antenas debido a su mayor cobertura. Sin embargo, con el uso de frecuencias más altas en la tecnología 4G, surge la necesidad de instalar muchas más antenas para cubrir todo el tramo de la autovía. Esto implica que colocar ambos equipos en cada una de las estaciones base aumente considerablemente los costes. Por lo que nosotros utilizaremos la otra alternativa, la centralizada [9].

En la RAN centralizada, la RRU y la BBU no están ubicadas en el mismo lugar. En cambio, las RRU se despliegan de manera remota junto al sistema de antenas y se conectan mediante una interfaz de fronthaul a una BBU que procesa las señales de radio de varias RRU pertenecientes a una o varias estaciones base. La desventaja de esta configuración es que es relativamente complejo coordinar la RRU y la BBU separadas, y que la conexión desde la RRU a la BBU, que en una estación base RAN distribuida puede ser de unos pocos metros, ahora puede ser de varios kilómetros y requiere un ancho de banda relativamente alto, a menudo solo disponible con una conexión de fibra óptica [9].

La última opción y más reciente sigue la estructura de la centralizada, pero la BBU está virtualizada y se proporciona a través de la infraestructura de la nube. Esta configuración ayuda a aprovechar la infraestructura en la nube con economías de escala y a beneficiarse de un despliegue más flexible de las funciones de la red. A esta arquitectura se la conoce como RAN virtualizada (vRAN) [9].

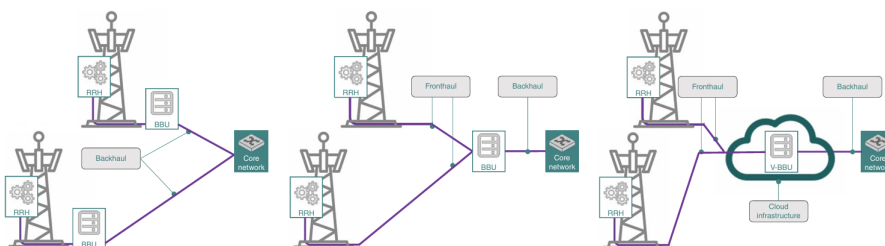


Figura 11: Posibles arquitecturas para las estaciones base: distribuida (izquierda), centralizada (centro) y virtualizada (derecha).

Dentro de estas opciones que se han expuesto y debido al gran número de antenas que hemos decidido desplegar para dar cobertura en todos los puntos, que eleva considerablemente los costes, hemos decidido implantar la arquitectura centralizada de estaciones base, situando una BBU central. Puesto que la distancia entre los RRU y BBU no puede superar los 38 kilómetros, hemos decidido poner dos BBU en dos puntos de la autovía. Estos dos BBU estarán situados por la zona de la Aldeanueva de Figueroa y en Alaejos, y cada una se conectaría a las RRU de las estaciones base más próximos.

El siguiente punto del despliegue es determinar la estructura del core. De una manera similar a las estaciones base, la disposición del core de una red 4G puede ser tanto centralizada como distribuida, y la elección entre estos dos enfoques depende de varios factores, incluyendo la escala de la red, el rendimiento esperado y los costos. A continuación, se detallan las ventajas e inconvenientes de cada enfoque.

CAPÍTULO 3. INFRAESTRUCTURA 4G

En primer lugar, los costes de despliegue de un core centralizado son menores. Tener un core centralizado implica menores gastos en construcción, despliegue, equipos y personal, además de requerir menos recursos para mantener la infraestructura. La gestión y el mantenimiento también son más sencillos, ya que toda la infraestructura está concentrada en un solo lugar. Sin embargo, es importante tener en cuenta los problemas de latencia a lo largo de toda la red. Si los requisitos de latencia son muy restrictivos y la red es muy grande, puede que no se cumplan estos requisitos, lo que haría inviable el despliegue del servicio. En el capítulo 3.3 se ha discutido este punto.

Por otro lado, un core centralizado sería más apto para este punto de la latencia, ya que, al tener más puntos de control en puntos intermedio de la red, se reducen estos tiempos. Además, en un core distribuido se puede optimizar el enrutamiento del tráfico y balancear la carga de manera más eficiente, haciendo que sea más difícil saturar el servicio en condiciones de mucha demanda. Los inconvenientes del core distribuido son el aumento de los costes y el aumento de la complejidad de la gestión.

Puesto que la longitud del tramo de autovía no es excesivamente larga, los criterios de latencia no se ven excesivamente afectados respecto a poner un core distribuido, por lo que hemos decidido desplegar un core centralizado para así reducir los costes y reducir la complejidad de la gestión. Además, la demanda de usuarios generalmente no será muy elevada, salvo condiciones excepcionales, como pueden ser atascos, por lo que tener más de un core puede ser un gasto innecesario.

El tercer punto en el despliegue es determinar cómo se van a comunicar los elementos de la red anteriormente descritos. Hay dos puntos en los que desplegar la red, el conocido como backhaul, que es la parte de la red que va desde el core hasta la BBU y el conocido como fronthaul, que es la conexión entre el BBU y el RRU.

La red fronthaul está relacionada con una arquitectura centralizada en la que las funciones de procesamiento de la banda base están centralizadas. Esto no solo simplifica la operación de las RRU, sino que también permite una cooperación mejorada entre las estaciones base.

La transmisión de señales entre las BBU y las RRU en las arquitecturas centralizadas, se basa principalmente en la tecnología de radio digital sobre fibra (D-RoF). Esto se hace para garantizar la transparencia de la forma de onda y la rentabilidad. Además, la conexión entre las BBUs y las RRU se basa principalmente en la interfaz de radio pública común (CPRI) [10].

El puerto CPRI es un puerto que soporta el protocolo CPRI (Common Public Radio Interface). Conectando el puerto CPRI de la BBU-RRU, se establece entre ellos el canal de transmisión de los datos de control de CPRI y los datos de usuario de CPRI. Entre ellos, el formato de datos del plano de usuario es I/Q, y cada celda debe ocupar una gran cantidad de ancho de banda CPRI para transmitir los datos en el puerto CPRI [11].

La red backhaul entre el core y los BBU será también de fibra óptica. El despliegue de la fibra óptica en el backhaul supone la utilización de distintos tipos de dispositivos destinados a la comunicación en fibra óptica. Estos dispositivos son los ONT (terminal de red óptica) y los OLT (terminal de línea óptica). Estos dispositivos son utilizados en redes PON (red óptica pasiva) de fibra óptica. El terminal de línea óptica (OLT) es el punto de partida de la red óptica

CAPÍTULO 3. INFRAESTRUCTURA 4G

pasiva. La función principal del OLT es convertir, entramar y transmitir señales para la red PON y coordinar la multiplexación del terminal de red óptica (ONT) para una transmisión ascendente compartida. El ONT es el dispositivo eléctrico del sistema de red óptica pasiva en el lado opuesto (el del usuario) de la red e incluye puertos Ethernet para la conectividad de red o dispositivos domésticos [12].

Visto lo anteriormente explicado, para nuestro despliegue de la red serán necesarios un OLT que estará situado en el core y un par de ONT, uno para cada BBU. Para la conexión de fronthaul a través de fibra, será necesario que los equipos RRU y BBU tengan la interfaz CPRI.

Para el despliegue de la fibra óptica hemos revisado las redes de fibra óptica oscura en la zona de Valladolid, Zamora y Salamanca de Reintel. Reintel es el principal operador neutro de infraestructuras de telecomunicaciones de fibra óptica en España.

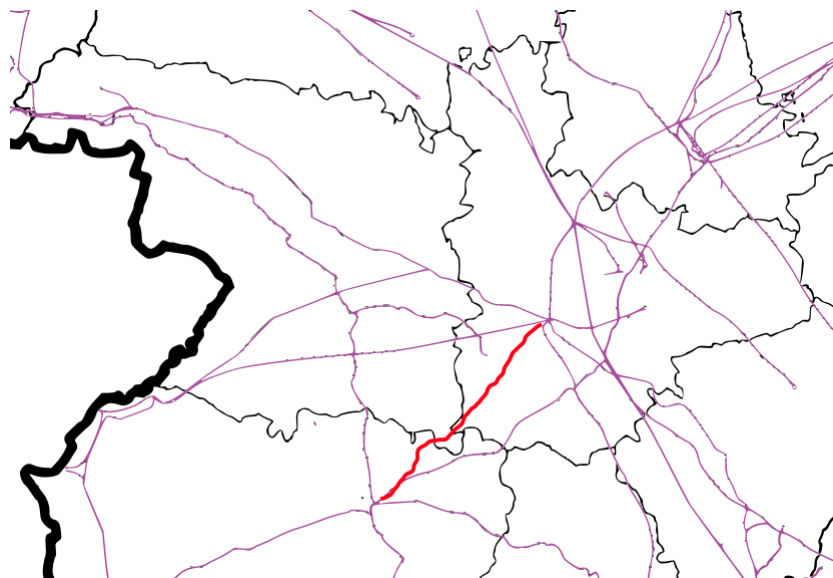


Figura 12: Fibra oscura de Reintel cerca de la A-62

Debido a que dicha fibra no está cerca de la autovía, no resulta interesante utilizarla, por lo que será necesario desplegarla a lo largo de la carretera.

En la Figura 13 se muestra el despliegue de toda la red y de todos sus elementos.

Una vez vista la estructura principal de las tres partes que componen el despliegue de la red vamos a explicar los elementos que hemos decidido usar para cada una de las partes y las características principales:

Las estaciones base tiene tres elementos principales, que son las antenas, el RRU y el BBU.

En cada estación base se han colocado antenas de la marca Ubitiqui modelo am2g-16- 90. En las 23 estaciones base centrales del tramo de autovía se han colocado dos antenas y en las dos estaciones de los extremos se ha colocado solo una. Estas antenas son antenas directivas para que den cobertura a lo largo de la carretera. Tienen un ancho de haz de 90° y una ganancia de 17PerdB. Puede trabajar en el rango de frecuencias de la banda 7 de 2600MHz.

CAPÍTULO 3. INFRAESTRUCTURA 4G

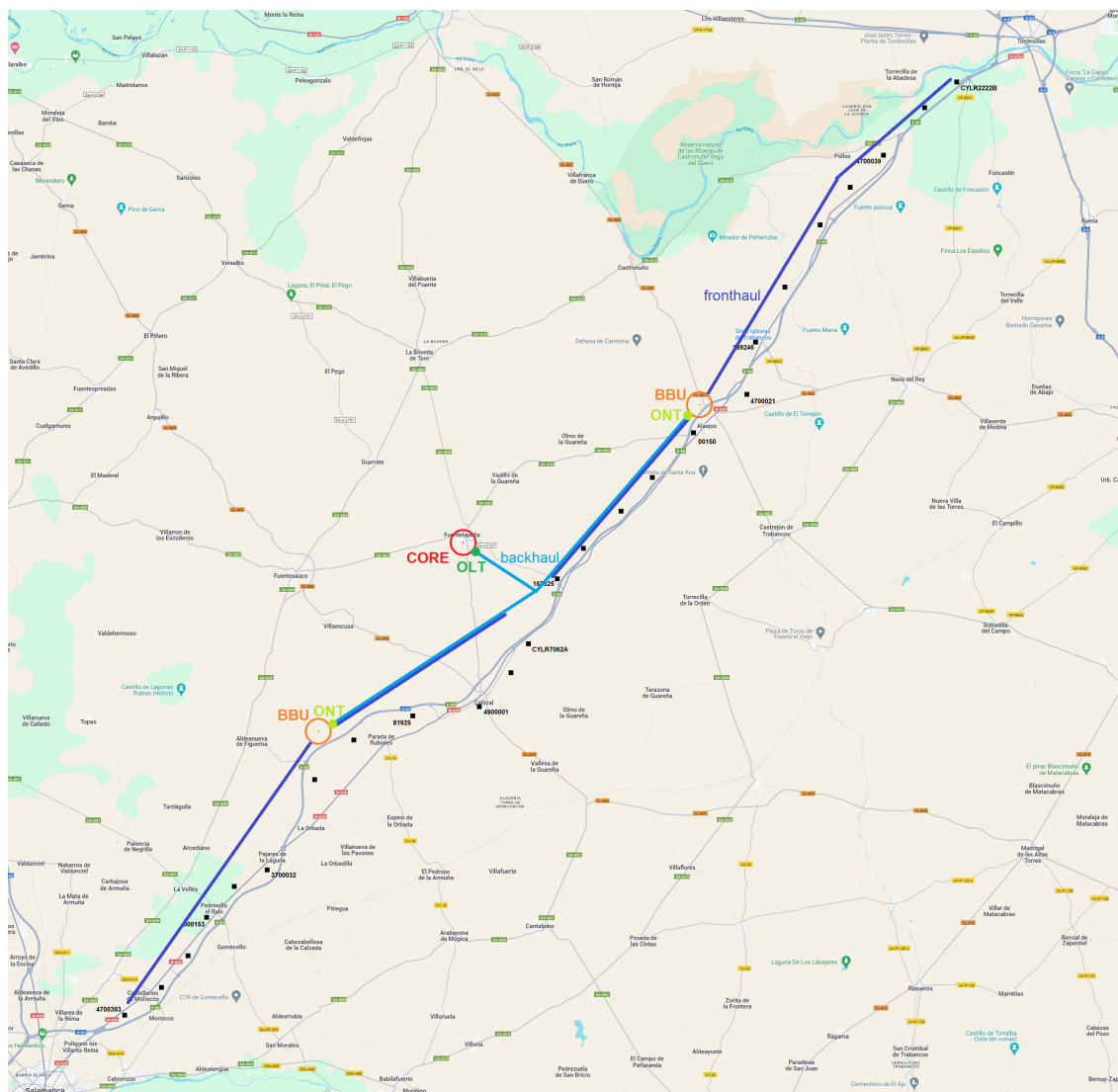


Figura 13: Mapa del despliegue final con los elementos más importantes

En la misma torre de la estación base se colocan los equipos RRU. En cada estación base se colocan tantos RRU como antenas haya. En nuestro caso utilizamos el modelo Huawei RRU5301. Este RRU procesa señales en el rango de frecuencia que vamos a usar y tiene puertos CPRI para conectarla a la BBU centralizada con velocidades superiores a Gbps. Es una unidad de radio remota que ofrece alta capacidad y cobertura, compatible con las soluciones LTE de Huawei.

El modelo de BBU que vamos a usar es también de la marca Huawei. Es el modelo BBU3900. Gestiona de forma centralizada todo el sistema de la estación base, incluida la operación y el mantenimiento, el procesamiento de señales y el reloj del sistema. Procesa datos de banda base de enlace ascendente y descendente y proporciona puertos CPRI para comunicarse con los módulos de la estación base [13]. La capacidad exacta de la BBU 3900 de Huawei para conectarse a RRU puede variar dependiendo de la configuración específica. Generalmente, puede soportar entre 12 y 36 RRUs, dependiendo de la configuración específica y los módulos utilizados. En nuestro caso cada BBU dará servicio a la mitad de las RRU, es decir unas 24.

CAPÍTULO 3. INFRAESTRUCTURA 4G

En el núcleo de la red se encuentran el core de la red 4G y todos los módulos necesarios para realizar el procesamiento de las imágenes.

Para la parte de 4G se va a utilizar el core modelo IPLOOK IKEPC 510. EL IPLOOK IKEPC 500 combina los elementos de red MME, SGW, PGW, HSS, PCRF, IMS y DRA en una plataforma X86 COTS (comercial fuera de estantería) que es 100 % compatible con 3GPP. Cada uno de estos elementos puede soportar todas las interfaces estándar definidas por 3GPP.

Dentro del core debe haber también el módulo que realice todo el procesamiento relacionado con inteligencia artificial para la detección de imágenes. Para ello se ha seleccionado la serie NCasT4 v3 de Azure que usa la tecnología de las GPU Nvidia Tesla T4 y las CPU AMD EPYC 7V12 (Rome). Cuenta con hasta 4 GPU NVIDIA T4 con 16 GB de memoria cada una, hasta 64 núcleos de procesador AMD EPYC 7V12 (Rome) sin multiproceso (frecuencia base de 2,45 GHz, frecuencia máxima de todos los núcleos de 3,1 GHz y frecuencia máxima de un solo núcleo de 3,3 GHz) y 440 GiB de memoria del sistema. Estos módulos son idóneos para implementar servicios de inteligencia artificial, como la inferencia en tiempo real de solicitudes generadas por el usuario, o bien para cargas de trabajo de visualización y gráficos interactivos mediante el controlador GRID de NVIDIA y tecnología de GPU [14].

3.3. Capacidad de una celda 4G

Una parte fundamental para el diseño de una infraestructura 4G es el análisis y estudio de la capacidad que ofrece a los usuarios cada una de las celdas 4G. Una celda se refiere a un área geográfica que está cubierta por una antena base (eNodeB) específica en una red LTE y es la unidad fundamental de cobertura y servicio de la red.

Lo primero que se debe averiguar es la calidad mínima que debe tener de las imágenes de vídeo para que se pueda llevar a cabo la detección de señales de con una tasa de acierto elevada y con un grado de confianza alto. Tras barajar diversas opciones sobre cuál debería ser, se concluyó que la calidad mínima del vídeo que cada coche debería enviar al core a través de la red debe tener dimensiones 640x380 píxeles, suponiendo que las imágenes (frames) que componen el vídeo tienen un tamaño de media 170KB, y grabados con una tasa de refresco de imagen de 30fps.

Una vez hecho esto, hay que comprimir las imágenes antes de enviarlas para poder optimizar el uso de la red. Actualmente, en el mercado se utilizan dos sistemas distintos de conversión de vídeo:

- **H.264 (MPEG-4 AVC):** Estándar de compresión de vídeo digital con pérdidas que utiliza técnicas de predicción de movimiento para reducir la redundancia temporal entre los frames de vídeo sucesivos y es altamente eficiente. La tasa de compresión es de media un 80 % aproximadamente. Este sistema es el más extendido actualmente.
- **H.265:** También conocido como High Efficiency Video Coding (HEVC), es una evolución del estándar de compresión de vídeo H.264 y representa un avance significativo en la

CAPÍTULO 3. INFRAESTRUCTURA 4G

eficiencia de compresión y calidad de vídeo. Como consecuencia, la tasa de compresión es de media un 85 % aproximadamente. Actualmente no es el más utilizado, pero es esperable que en un futuro sea el estándar predominante en el mercado del automóvil.

La compresión de ambos estándares es bastante eficiente, ocupando cada imagen de media 34KB si se utiliza el primero de los dos (H.264), y una media de 25,5KB si, en cambio, se utiliza el estándar H.265. Tal y como se va a trabajar a partir de ahora y en próximos análisis de la infraestructura, se va a suponer en la mayoría de los casos la peor situación posible de la red o del estado de las celdas. Por lo tanto, se va a suponer en este caso que la gran mayoría de vehículos que harán uso de este servicio disponen del estándar H.264 para la compresión previa al envío de vídeo.

Sabiendo la calidad mínima que necesita el servicio que se va a desplegar, es posible averiguar cuál es la tasa de bits que necesita cada equipo de usuario (UE) para comunicarse con la red troncal y, por tanto, el ancho de banda que necesita cada uno para que la comunicación sea posible. A partir de aquí, se continúa calculando la tasa de bits que un usuario necesita para poder enviar el vídeo sin ningún tipo de error o interferencia. Teniendo en cuenta que las cámaras de vídeo graban a 30fps, esto supone una sobrecarga de trabajo para el core en el análisis de vídeo. Por este motivo, cada usuario en vez de enviar los 30fps, envía dos de cada tres imágenes que se graban, descartando aquellas que sean múltiples de tres, siendo una tasa de refresco final del vídeo de 20fps. Dada esta situación, entonces:

$$Carga_usuario = 34 \text{ KB/frame} \times 20 \text{ frames/s} \times 8\text{bits} = 5,44 \text{ Mbps} \quad (1)$$

La tasa binaria final que necesita cada usuario es de 5,44 Mbps para realizar el envío de vídeo. Además, hay que tener en cuenta que el enlace va a tener aproximadamente un 75 % de carga útil, quedando el restante para información de control de tráfico, encaminamiento, seguridad... (25 %). En total, la capacidad que necesita cada usuario es de 6,8 Mbps:

$$Carga_usuario_total = 5,44 \text{ Mbps} + (5,44 \text{ Mbps} \times 0,25) = 6,8 \text{ Mbps} \quad (2)$$

Conocida la capacidad que necesita un usuario para el envío de vídeo, hay que continuar con el análisis y cálculo de la celda para averiguar el ancho de banda que necesita cada uno. No es posible calcular directamente el ancho de banda sin antes averiguar qué tipo de modulaciones se emplean en 4G LTE para las comunicaciones inalámbricas. Según la release 14 de 3GPP [15], la tecnología 4G emplea tres tipos de modulaciones en función del CQI (Channel Quality Indicator). El CQI indica la calidad del canal de comunicación entre el dispositivo móvil (UE, User Equipment) y la estación base (eNodeB). Este indicador es indicado por el dispositivo móvil a la estación base, y se utiliza para adaptar de manera dinámica los parámetros de transmisión con el objetivo de optimizar el rendimiento de la red. Recibido un CQI desde el usuario, se escoge una modulación soportada por la tecnología, dichas modulaciones son QPSK, 16-QAM y 64-QAM cuyas eficiencias espectrales son:

CAPÍTULO 3. INFRAESTRUCTURA 4G

Modulación	Bits/símbolo	Capacidad del canal
QPSK	2	$2 \text{ bits/Hz} \times 10 \text{ MHz} \times 2 = 40 \text{ Mbps}$
16-QAM	4	$4 \text{ bits/Hz} \times 10 \text{ MHz} \times 2 = 80 \text{ Mbps}$
64-QAM	6	$6 \text{ bits/Hz} \times 10 \text{ MHz} \times 2 = 120 \text{ Mbps}$

Tabla 1: Ancho de banda por usuario según el tipo de modulación.

La relación entre el CQI que recibe el eNodeB de cada UE y el tipo de modulación que se elige para las comunicaciones entre ambos están estandarizadas y se pueden ver en la Figura 14. Como consecuencia, dependiendo del CQI del canal de conexión de usuario, la celda tiene una capacidad asignada a cada uno dependiente de la modulación:

- Con un CQI bajo (4, 5 ó 6): Se emplea una modulación QPSK en las comunicaciones. Si el ancho de banda tanto de subida como de bajada son de 10MHz en cada celda, es posible dar servicio a un total de 3 usuarios en cada una.

$$usuarios_T = \frac{40 \text{ Mbps}}{5,44 \text{ Mbps}} = 7,35 = 7 \text{ usuarios} \quad (3)$$

- Con un CQI medio (7, 8 ó 9): Se emplea una modulación 16-QAM en las comunicaciones. Si el ancho de banda tanto de subida como de bajada son de 10MHz en cada celda, es posible dar servicio a un total de 7 usuarios en cada una.

$$usuarios_T = \frac{80 \text{ Mbps}}{5,44 \text{ Mbps}} = 14,70 = 14 \text{ usuarios} \quad (4)$$

- Con un CQI alto (valores entre 10 y 15): Se emplea una modulación 64-QAM en las comunicaciones. Si el ancho de banda tanto de subida como de bajada son de 10MHz en cada celda, es posible dar servicio a un total de 10 usuarios en cada una.

$$usuarios_T = \frac{120 \text{ Mbps}}{5,44 \text{ Mbps}} = 22,05 = 22 \text{ usuarios} \quad (5)$$

Tras analizar dichos resultados, podemos concluir que cada celda puede dar servicio a un mínimo de siete usuarios de manera simultánea y a un máximo de veintidos. En la sección 3.1 estos cálculos se han tenido en cuenta para el diseño de toda la infraestructura de la red a desplegar.

3.4. Estudio del tráfico de la carretera

Para dar servicio a los usuarios que circulen por la autovía A-62 es necesario realizar un estudio de los vehículos que circulan por ella diariamente y realizar una estimación de los posibles usuarios que pueden utilizar nuestro servicio. Para ello accedemos a los datos oficiales

CAPÍTULO 3. INFRAESTRUCTURA 4G

CQI index	modulation	code rate x 1024	efficiency
0	out of range		
1	QPSK	78	0.1523
2	QPSK	120	0.2344
3	QPSK	193	0.377
4	QPSK	308	0.6016
5	QPSK	449	0.877
6	QPSK	602	1.1758
7	16 QAM	378	1.4766
8	16 QAM	490	1.9141
9	16 QAM	616	2.4063
10	64 QAM	466	2.7305
11	64 QAM	567	3.3223
12	64 QAM	666	3.9023
13	64 QAM	772	4.5234
14	64 QAM	873	5.1152
15	64 QAM	948	5.5547

Figura 14: Tabla relación CQI con modulación empleada.

del gobierno del tráfico monitorizado por dos estaciones permanentes dentro de la zona para la que daremos servicio[16]. Estas estaciones permanentes están situadas en los puntos kilométricos 179.61 y 223.74 de la autovía A-62. Los datos obtenidos pertenecen al año 2023, y se recogen como la media diaria de vehículos ligeros, pesados y totales que pasan por esos dos puntos kilométricos. Estos datos se muestran también a lo largo de los meses del año, siendo los periodos máximos en los meses de verano principalmente. En el punto kilométrico 179.61 se recoge un tráfico medio a lo largo del año de 14826 vehículos diarios, incluyendo vehículos pesados y vehículos ligeros. En el punto kilométrico 223.74 se recogen 16173 vehículos de media al día. Entre estos datos hay una pequeña diferencia de tráfico que puede deberse a los desvíos de los coches hacia carreteras secundarias, pero nos da una idea del volumen de tráfico medio diario que soporta la autovía.

Para estimar el número de usuarios que podría utilizar nuestro servicio, referenciamos un artículo publicado por el profesor de la Universidad Politécnica de Madrid en el año 2017 en el que realiza una encuesta a distintos usuarios para conocer la disponibilidad de estos a acoger tecnologías aplicadas a la conducción como puede ser la introducción del vehículo autónomo, el vehículo conectado y el vehículo compartido[17]. Centrándonos en el vehículo conectado, que es la parte que nos interesa, los resultados muestran que el 56 % de los encuestados reconocerían que esta tecnología sería útil y valiosa, pues aportaría información y servicios útiles para los usuarios. Sin embargo, solo el 16 % de estos estarían interesados en pagar por disponer de servicios de este tipo.

Aunque este estudio es del año 2017 y queda ya un poco lejos, lo tomamos como referencia para estimar los potenciales usuarios que podrían requerir de nuestro servicio. Es previsible que estos datos hayan aumentado respecto al año en el que se hizo el estudio, pero como de nuevo no tenemos datos, tomaremos ese valor como cierto. Sería conveniente volver a realizar una encuesta a la población con el objetivo de actualizar estos datos.

Para calcular el número de usuarios que utilizarán nuestro servicio en el tramo de autovía multiplicamos el porcentaje de personas dispuestas a pagar por el servicio por el número de vehículos que circulan por la autovía. Considerando el volumen mayor entre los dos puntos

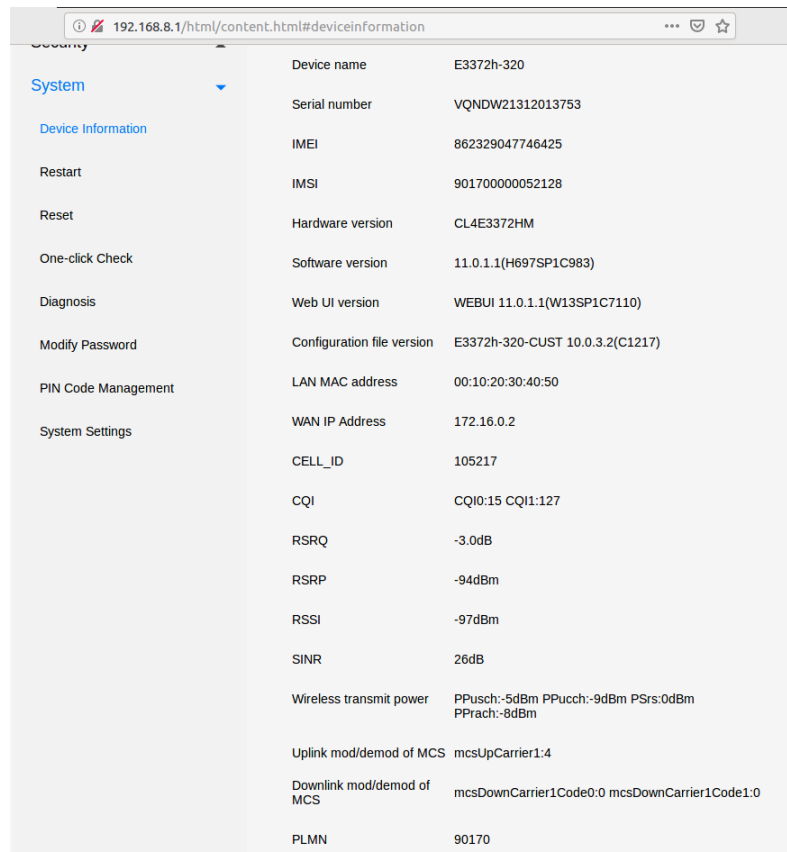
CAPÍTULO 3. INFRAESTRUCTURA 4G

kilométricos, que es 16000, esta multiplicación da como resultado 2560 vehículos utilizando el servicio de media al día.

3.5. Medición de las características de la red

Para comprobar que la red funciona correctamente es necesario llevar a cabo medidas de distintos parámetros para asegurar la calidad del servicio para todos los usuarios.

Algunos de estos parámetros son la velocidad de la red, la latencia, el jitter, el ancho de banda, la estabilidad, la disponibilidad y la tasa de errores. Además, de todos estos parámetros, hay que medir la cobertura de la red, esto incluye medir las zonas donde el nivel de señal sea bajo, las zonas de sobrealcances, y las zonas de intercambio entre antenas. (Esto en nuestro caso no será necesario pues solo tenemos una antena y un usuario). También es importante medir la capacidad de la red, esto es, el número de usuarios que permite simultáneamente. Para medir estas características hemos utilizado distintas herramientas. Para medir los parámetros físicos de la señal recibida se ha utilizado la interfaz web del módem, que nos ofrece parámetros de calidad y de potencia de la intensidad de señal en un momento dado.



The image shows a web browser window displaying the configuration page of a modem. The address bar shows the URL '192.168.8.1/html/content.html#deviceinformation'. On the left, there is a sidebar menu with options: System, Device Information, Restart, Reset, One-click Check, Diagnosis, Modify Password, PIN Code Management, and System Settings. The main content area displays various parameters in a table-like format.

Device name	E3372h-320
Serial number	VQNDW21312013753
IMEI	862329047746425
IMSI	901700000052128
Hardware version	CL4E3372HM
Software version	11.0.1.1(H697SP1C983)
Web UI version	WEBUI 11.0.1.1(W13SP1C7110)
Configuration file version	E3372h-320-CUST 10.0.3.2(C1217)
LAN MAC address	00:10:20:30:40:50
WAN IP Address	172.16.0.2
CELL_ID	105217
CQI	CQI0:15 CQI1:127
RSRQ	-3.0dB
RSRP	-94dBm
RSSI	-97dBm
SINR	26dB
Wireless transmit power	PPusch:-5dBm PPucch:-9dBm PSrs:0dBm PPrach:-8dBm
Uplink mod/demod of MCS	mcsUpCarrier1:4
Downlink mod/demod of MCS	mcsDownCarrier1Code0:0 mcsDownCarrier1Code1:0
PLMN	90170

Figura 15: Parámetros de medida en la interfaz web del módem.

En la interfaz del módem, además de la dirección IP, la dirección MAC, el número de célula

CAPÍTULO 3. INFRAESTRUCTURA 4G

y otros datos se muestran los siguientes parámetros de calidad: El CQI es el indicador de calidad del canal. Contiene información sobre la calidad del canal entre el UE y el eNB. En LTE, hay 15 valores diferentes de CQI que van desde 0 hasta 15, donde 15 indica la mejor calidad del canal y 0 indica la peor calidad del canal. Dependiendo del valor que informe el UE, la red transmite datos con diferentes tamaños de bloque de transporte, es decir, distintas modulaciones. Si la red obtiene un valor de CQI alto del UE, transmite los datos con un tamaño de bloque de transporte mayor y viceversa.

Esta variabilidad en las modulaciones ayuda a mejorar la eficiencia. Si la red enviara un tamaño de bloque de transporte grande sin considerar el valor de CQI cuando la calidad del canal es baja, sería probable que el UE no pudiera decodificarlo, obligando a la red a retransmitirlo, lo que a su vez provocaría un desperdicio de recursos de radio.

RSRQ (Calidad recibida de la señal de referencia) describe la calidad de las señales piloto recibidas. El valor RSRQ se mide en dB.

RSRP (potencia recibida de la señal de referencia) es el valor de potencia promedio de las señales piloto recibidas (señal de referencia) o el nivel de señal recibida desde la estación base. El valor RSRP se mide en dBm. Con RSRP = -120 dBm y menos, la conexión LTE puede ser inestable o no estar establecida.

El SINR es la relación señal a interferencia más ruido. Un valor SINR positivo significa que hay una señal más efectiva que el ruido. El valor mínimo aceptable para el funcionamiento estable de la red es de 10 dB. Cuanto mayor sea el valor SINR, mejor será la calidad de la señal. Un valor de SINR negativo significará más ruido en la señal recibida que en la señal efectiva. Con valores negativos o cercanos a cero será imposible establecer una conexión LTE, o será deficiente en velocidad y calidad.

RSSI (Indicador de intensidad de la señal recibida) es un indicador del nivel de potencia de la señal recibida por el módem. El valor se mide en dBm. El valor mínimo aceptable para el funcionamiento de la red es de -85 dBm. Los parámetros SINR y RSSI no están directamente relacionados entre sí, es decir, puede haber casos en los que uno de los valores sea alto y otro bajo.

Calidad de señal	RSRP (dBm)	RSRQ (dB)	SINR (dB)	RSSI (dBm)
Excelente	>-80	>-10	>20	>-65
Bueno	De -80 a -90	De -10 a -15	De 13 a 20	De -65 a -75
Malo	De -90 a -100	De -15 a -20	De 0 a 13	De -75 a -85
Muy malo	<-100	<-20	<0	<-85

Figura 16: Niveles de señal para los distintos indicadores.

A continuación, se muestra una tabla con la calidad de la señal en función de los parámetros anteriores (Figura 16). Hay que tener en cuenta que la calidad de la conexión LTE depende no sólo de los indicadores en cuestión, sino también de otros factores (de la carga de trabajo de la

CAPÍTULO 3. INFRAESTRUCTURA 4G

estación base, de la calidad del equipo de la estación base, etc.). La estimación de parámetros es aproximada y subjetiva y se basa en la experiencia práctica de los usuarios.

Se han realizado mediciones en distintos puntos del laboratorio para ilustrar la variación de estos parámetros. En el laboratorio hay muchos obstáculos y la intensidad de la señal varía bastante si no hay visión directa entre el eNB y el UE. Cuando hay visión directa entre el eNB y el UE, la calidad de señal mejora, siendo mejor cuanto más cerca y cuando apunta más directamente. Esto se muestra en la siguiente imagen. En la imagen se muestran los valores de la interfaz del módem cuando la visión entre la antena y el coche es directa, a la izquierda, y cuando no lo es, a la derecha.

Visión directa		Visión no directa	
CQI	CQI0:15 CQI1:127	CQI	CQI0:15 CQI1:127
RSRQ	-3.0dB	RSRQ	-3.0dB
RSRP	-86dBm	RSRP	-92dBm
RSSI	-79dBm	RSSI	-83dBm
SINR	>=30dB	SINR	18dB

Figura 17: Niveles de señal con visión directa y visión no directa.

Para ilustrar esto también se utilizó una aplicación de Android en la que se media la señal en tiempo real. Esta aplicación es Netmonitor. Se introdujo la tarjeta SIM en un teléfono y en la aplicación se mostraban algunos de los datos de intensidad y calidad de señal anteriormente mencionados pero esta vez se mostraba la evolución en tiempo real, según nos movíamos por el laboratorio.

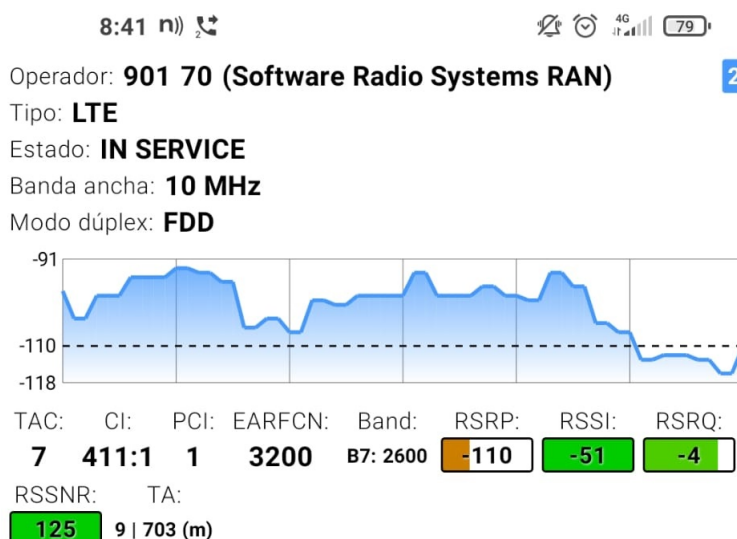


Figura 18: Señal en tiempo real en Netmonitor.

CAPÍTULO 3. INFRAESTRUCTURA 4G

En la interfaz web del módem también aparecen otros datos como PPusch, PPucch, PSrs y PPrach que están relacionados con la comunicación de UE y el eNB en el enlace ascendente. Estos parámetros se utilizan para realizar la sincronización y el control de potencia.

Por otro lado, además de las medidas de intensidad de la señal se han realizado medidas de jitter, de la latencia y de la velocidad de la red (throughput). Para ello se ha utilizado las herramientas iperf3 y ping.

Ping es una utilidad de software de administración de redes informáticas que se utiliza para probar la accesibilidad de un host en una red de TCP/IP. En nuestro caso nos resulta útil porque nos ofrece información de la latencia, mide el tiempo de ida y vuelta de los mensajes enviados desde el host de origen a un host de destino.

```
deepracer@amss-zp4t:~$ ping 172.16.0.1
PING 172.16.0.1 (172.16.0.1) 56(84) bytes of data.
64 bytes from 172.16.0.1: icmp_seq=1 ttl=63 time=37.7 ms
64 bytes from 172.16.0.1: icmp_seq=2 ttl=63 time=36.5 ms
64 bytes from 172.16.0.1: icmp_seq=3 ttl=63 time=34.6 ms
64 bytes from 172.16.0.1: icmp_seq=4 ttl=63 time=32.7 ms
64 bytes from 172.16.0.1: icmp_seq=5 ttl=63 time=30.6 ms
64 bytes from 172.16.0.1: icmp_seq=6 ttl=63 time=48.8 ms
64 bytes from 172.16.0.1: icmp_seq=7 ttl=63 time=47.6 ms
64 bytes from 172.16.0.1: icmp_seq=8 ttl=63 time=25.6 ms
64 bytes from 172.16.0.1: icmp_seq=9 ttl=63 time=43.8 ms
64 bytes from 172.16.0.1: icmp_seq=10 ttl=63 time=43.0 ms
64 bytes from 172.16.0.1: icmp_seq=11 ttl=63 time=41.6 ms
64 bytes from 172.16.0.1: icmp_seq=12 ttl=63 time=39.6 ms
64 bytes from 172.16.0.1: icmp_seq=13 ttl=63 time=37.6 ms
64 bytes from 172.16.0.1: icmp_seq=14 ttl=63 time=35.4 ms
64 bytes from 172.16.0.1: icmp_seq=15 ttl=63 time=33.6 ms
64 bytes from 172.16.0.1: icmp_seq=16 ttl=63 time=31.6 ms
64 bytes from 172.16.0.1: icmp_seq=17 ttl=63 time=49.5 ms
64 bytes from 172.16.0.1: icmp_seq=18 ttl=63 time=47.7 ms
64 bytes from 172.16.0.1: icmp_seq=19 ttl=63 time=45.6 ms
64 bytes from 172.16.0.1: icmp_seq=20 ttl=63 time=43.6 ms
64 bytes from 172.16.0.1: icmp_seq=21 ttl=63 time=41.7 ms
^C
--- 172.16.0.1 ping statistics ---
21 packets transmitted, 21 received, 0% packet loss, time 20036ms
rtt min/avg/max/mdev = 25.673/39.504/49.583/6.492 ms
deepracer@amss-zp4t:~$
```

Figura 19: Medición de latencia y jitter con ping

Al hacer un ping a la estación base durante varios intentos vemos que la latencia media es de unos 40ms. Además, se muestra el tiempo mínimo y el máximo y la desviación estándar de esos intentos, que sería una medida del jitter. En este caso de 6ms.

Iperf3 es una herramienta más compleja que permite simular condiciones específicas en la red para realizar medidas. Permite elegir si el tráfico que se envía es TCP o UDP y permite controlar la tasa de envío de datos para poder detectar y evitar saturaciones en la red, entre otras cosas. Iperf3 se utiliza lanzando un servidor iperf3 en un lado de la conexión, en este caso en el eNB mediante el comando:

```
$ iperf3 -s
```

Esto pondrá al equipo en modo escucha en el puerto 5201 (por defecto). Después se realizan las peticiones desde el equipo que actuará como cliente, en este caso el coche. Para lanzar la orden del cliente se usa la opción -c. Con la opción -p se selecciona el puerto en escucha en el servidor. Con la opción -u se indica el protocolo utilizado. Si se pone indica que se usa UDP,

CAPÍTULO 3. INFRAESTRUCTURA 4G

si no, se usa TCP. Se usa la opción -R para enviar tráfico del servidor al cliente, de lo contrario se enviaría del cliente al servidor. Esto es útil para medir la capacidad de la red tanto en subida como en bajada. Por último, se usa la opción -b para indicar el volumen de tráfico enviado.

En primer lugar, utilizamos iperf3 enviando tráfico TCP para medir el throughput. TCP ajusta la velocidad de transmisión en función de la congestión de la red, proporcionando una medida realista del throughput en condiciones típicas de red.

```
deepracer@anss-zp4t:~$ iperf3 -c 172.16.0.1 -p 5201 -R
Connecting to host 172.16.0.1, port 5201
Reverse mode, remote host 172.16.0.1 is sending
[ 4] local 192.168.8.105 port 53648 connected to 172.16.0.1 port 5201
[ ID] Interval      Transfer    Bandwidth
[ 4] 0.00-1.00 sec  3.98 MBytes 33.3 Mbits/sec
[ 4] 1.00-2.00 sec  4.20 MBytes 35.2 Mbits/sec
[ 4] 2.00-3.00 sec  4.20 MBytes 35.2 Mbits/sec
[ 4] 3.00-4.00 sec  4.20 MBytes 35.2 Mbits/sec
[ 4] 4.00-5.00 sec  4.20 MBytes 35.2 Mbits/sec
[ 4] 5.00-6.00 sec  4.19 MBytes 35.2 Mbits/sec
[ 4] 6.00-7.00 sec  4.19 MBytes 35.2 Mbits/sec
[ 4] 7.00-8.00 sec  4.19 MBytes 35.1 Mbits/sec
[ 4] 8.00-9.00 sec  4.19 MBytes 35.1 Mbits/sec
[ 4] 9.00-10.00 sec 4.20 MBytes 35.2 Mbits/sec
- - - - -
[ ID] Interval      Transfer    Bandwidth    Retr
[ 4] 0.00-10.00 sec 43.9 MBytes 36.8 Mbits/sec 11
[ 4] 0.00-10.00 sec 42.1 MBytes 35.3 Mbits/sec
- - - - -
iperf Done.
deepracer@anss-zp4t:~$ iperf3 -c 172.16.0.1 -p 5201
Connecting to host 172.16.0.1, port 5201
[ 4] local 192.168.8.105 port 53652 connected to 172.16.0.1 port 5201
[ ID] Interval      Transfer    Bandwidth    Retr  Cwnd
[ 4] 0.00-1.00 sec  2.08 MBytes 17.5 Mbits/sec  0   158 KBytes
[ 4] 1.00-2.00 sec  2.18 MBytes 18.3 Mbits/sec  0   259 KBytes
[ 4] 2.00-3.00 sec  2.01 MBytes 16.9 Mbits/sec  0   361 KBytes
[ 4] 3.00-4.00 sec  2.13 MBytes 17.8 Mbits/sec  0   461 KBytes
[ 4] 4.00-5.00 sec  2.33 MBytes 19.6 Mbits/sec  0   561 KBytes
[ 4] 5.00-6.00 sec  2.01 MBytes 16.9 Mbits/sec  0   663 KBytes
[ 4] 6.00-7.00 sec  1.77 MBytes 14.9 Mbits/sec  0   764 KBytes
[ 4] 7.00-8.00 sec  2.22 MBytes 18.7 Mbits/sec  0   864 KBytes
[ 4] 8.00-9.00 sec  1.78 MBytes 14.9 Mbits/sec  0   966 KBytes
[ 4] 9.00-10.00 sec 2.22 MBytes 18.7 Mbits/sec  0  1.04 MBytes
- - - - -
[ ID] Interval      Transfer    Bandwidth    Retr
[ 4] 0.00-10.00 sec 20.7 MBytes 17.4 Mbits/sec  0
[ 4] 0.00-10.00 sec 20.0 MBytes 16.8 Mbits/sec
- - - - -
iperf Done.
deepracer@anss-zp4t:~$
```

Figura 20: Medida del throughput de subida y de bajada

En la imagen vemos que tenemos un throughput de bajada de 36Mbps y uno de subida de 16Mbps.

Una vez visto el throughput, simulamos tráfico UDP para ver el jitter y las pérdidas de paquetes. Se utiliza UDP pues es un protocolo sin conexión que envía paquetes (datagramas) sin establecer una conexión previa y sin garantizar la entrega, el orden o la integridad de los datos. Debido a estas características, el jitter se puede medir de manera efectiva, ya que los paquetes pueden llegar en diferentes intervalos de tiempo, permitiendo observar la variación en los tiempos de llegada.

Primero simulamos un tráfico de 30Mbps para ver si el canal de bajada y de subida experimentan pérdidas.

Vemos que el canal de bajada no experimenta pérdidas. Sin embargo, el de subida sí. Esto es debido al throughput calculado antes. El de bajada era superior a 30Mbps, mientras que el de

CAPÍTULO 3. INFRAESTRUCTURA 4G

```
deepracer@amss-zp4t:~$ iperf3 -c 172.16.0.1 -p 5201 -u -R -b 30M
Connecting to host 172.16.0.1, port 5201
Reverse mode, remote host 172.16.0.1 is sending
[ 4] local 192.168.8.105 port 54713 connected to 172.16.0.1 port 5201
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagrams
[ 4] 0.00-1.00 sec  3.58 MBytes  30.0 Mbits/sec  2.130 ms  2/460 (0.43%)
[ 4] 1.00-2.00 sec  3.61 MBytes  30.3 Mbits/sec  2.809 ms  0/462 (0%)
[ 4] 2.00-3.00 sec  3.57 MBytes  29.9 Mbits/sec  2.771 ms  0/457 (0%)
[ 4] 3.00-4.00 sec  3.58 MBytes  30.0 Mbits/sec  2.700 ms  0/458 (0%)
[ 4] 4.00-5.00 sec  3.57 MBytes  30.0 Mbits/sec  3.198 ms  0/457 (0%)
[ 4] 5.00-6.00 sec  3.55 MBytes  29.8 Mbits/sec  3.007 ms  0/455 (0%)
[ 4] 6.00-7.00 sec  3.58 MBytes  30.0 Mbits/sec  3.133 ms  0/458 (0%)
[ 4] 7.00-8.00 sec  3.59 MBytes  30.1 Mbits/sec  3.049 ms  0/459 (0%)
[ 4] 8.00-9.00 sec  3.59 MBytes  30.1 Mbits/sec  2.840 ms  0/459 (0%)
[ 4] 9.00-10.00 sec 3.55 MBytes  29.8 Mbits/sec  3.096 ms  0/454 (0%)
-- -- -- -- --
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagrams
[ 4] 0.00-10.00 sec 36.0 MBytes  30.2 Mbits/sec  2.791 ms  2/4604 (0.043%)
[ 4] Sent 4604 datagrams

iperf Done.
deepracer@amss-zp4t:~$ iperf3 -c 172.16.0.1 -p 5201 -u -b 30M
Connecting to host 172.16.0.1, port 5201
[ 4] local 192.168.8.105 port 47978 connected to 172.16.0.1 port 5201
[ ID] Interval      Transfer      Bandwidth      Total Datagrams
[ 4] 0.00-1.00 sec  3.26 MBytes  27.3 Mbits/sec  417
[ 4] 1.00-2.00 sec  3.59 MBytes  30.1 Mbits/sec  460
[ 4] 2.00-3.00 sec  3.62 MBytes  30.4 Mbits/sec  464
[ 4] 3.00-4.00 sec  3.57 MBytes  30.0 Mbits/sec  457
[ 4] 4.00-5.00 sec  3.51 MBytes  29.4 Mbits/sec  449
[ 4] 5.00-6.00 sec  3.57 MBytes  30.0 Mbits/sec  457
[ 4] 6.00-7.00 sec  3.66 MBytes  30.7 Mbits/sec  469
[ 4] 7.00-8.00 sec  3.54 MBytes  29.7 Mbits/sec  453
[ 4] 8.00-9.00 sec  3.61 MBytes  30.3 Mbits/sec  462
[ 4] 9.00-10.00 sec 3.66 MBytes  30.7 Mbits/sec  468
-- -- -- -- --
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagrams
[ 4] 0.00-10.00 sec 35.6 MBytes  29.9 Mbits/sec  10.219 ms  2685/3733 (72%)
[ 4] Sent 3733 datagrams

iperf Done.
deepracer@amss-zp4t:~$
```

Figura 21: jitter y pérdidas para un flujo de 30Mbps

subida no llegaba a 20Mbps. El jitter del canal de bajada es de unos 2 ms. Volvemos a medir el tráfico de subida con un tráfico de 10Mbps.

```
deepracer@amss-zp4t:~$ iperf3 -c 172.16.0.1 -p 5201 -u -b 10M
Connecting to host 172.16.0.1, port 5201
[ 4] local 192.168.8.105 port 52417 connected to 172.16.0.1 port 5201
[ ID] Interval      Transfer      Bandwidth      Total Datagrams
[ 4] 0.00-1.00 sec  1.08 MBytes  9.04 Mbits/sec  138
[ 4] 1.00-2.00 sec  1.20 MBytes  10.0 Mbits/sec  153
[ 4] 2.00-3.00 sec  1.19 MBytes  9.96 Mbits/sec  152
[ 4] 3.00-4.00 sec  1.20 MBytes  10.0 Mbits/sec  153
[ 4] 4.00-5.00 sec  1.20 MBytes  10.0 Mbits/sec  153
[ 4] 5.00-6.00 sec  1.19 MBytes  9.96 Mbits/sec  152
[ 4] 6.00-7.00 sec  1.20 MBytes  10.0 Mbits/sec  153
[ 4] 7.00-8.00 sec  1.19 MBytes  9.96 Mbits/sec  152
[ 4] 8.00-9.00 sec  1.20 MBytes  10.0 Mbits/sec  153
[ 4] 9.00-10.00 sec 1.20 MBytes  10.0 Mbits/sec  153
-- -- -- -- --
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagrams
[ 4] 0.00-10.00 sec 11.8 MBytes  9.91 Mbits/sec  7.270 ms  0/1512 (0%)
[ 4] Sent 1512 datagrams

iperf Done.
```

Figura 22: Medida del jitter en la subida

Ahora no se producen pérdidas de paquetes. El jitter es de 7 ms.

Capítulo 4

Inteligencia artificial (IA)

La Inteligencia Artificial (IA) ha emergido como una tecnología revolucionaria, impactando en una multitud de sectores industriales, científicos y sociales. Uno de los campos donde la IA ha tenido un impacto significativo es en el desarrollo de vehículos autónomos. Este capítulo explora el papel crucial de la IA en los vehículos autónomos y se centra en un componente esencial de la visión artificial: YOLOv9.

4.1. Inteligencia artificial en vehículos autónomos

La inteligencia artificial se utiliza en los vehículos autónomos para el uso de algoritmos y técnicas avanzadas que permitan al vehículo percibir su entorno, tomar decisiones y actuar sin intervención humana. El objetivo principal es crear sistemas de conducción que sean seguros, eficientes y capaces de manejar una variedad de situaciones complejas en carretera.

4.1.1. Componentes clave de la IA en vehículos autónomos

- **Percepción:** capacidad del vehículo para recopilar y procesar información del entorno a través de sensores, como cámaras, radares, LiDAR y sistemas de ultrasonido. Estos sensores permiten al vehículo identificar peatones, otros vehículos, señales de tráfico y obstáculos en su camino.
- **Localización y mapeo:** la localización es el proceso mediante el cual el vehículo determina su posición precisa en un mapa, utilizando tecnologías como el GPS, junto con el mapeo simultáneo y localización, para crear mapas detallados del entorno en tiempo real.
- **Planeación y toma de decisiones:** uso de algoritmos de planificación que permiten al vehículo determinar la mejor ruta a seguir, empleando técnicas de IA, como aprendizaje por refuerzo y redes neuronales, para seleccionar las acciones adecuadas en función de la situación del tráfico, las señales de tráfico y las normas de conducción.

- **Control:** ejecución de las decisiones tomadas por el sistema de planificación, ya sea regulación de la velocidad, cambio de dirección, aceleración o frenado del vehículo. Se implementan algoritmos de control avanzados, como los controladores predictivos basados en modelos (MPC), para asegurar que el vehículo siga la trayectoria planificada con precisión y seguridad.

4.1.2. Técnicas de IA utilizadas

- **Aprendizaje automático (machine learning):** fundamental con el uso de algoritmos que permiten al vehículo aprender de grandes volúmenes de datos, identificar patrones y mejorar su rendimiento con el tiempo, procesándolos para que el vehículo tome decisiones en tiempo real [18].
- **Redes neuronales profundas (Deep Neural Networks, DNNs):** base principal en la investigación de las nuevas tecnologías de IA, para mejorar la conducción autónoma. Las DNNs, y en particular las redes neuronales convolucionales (CNNs) permiten el reconocimiento y clasificación de objetos en la carretera en tiempo real con alta precisión [18].
- **Aprendizaje por refuerzo (reinforcement learning):** permite que el vehículo aprenda mediante la experimentación y la retroalimentación, siendo especialmente útil para la toma de decisiones en entornos dinámicos y complejos, donde el vehículo debe adaptarse continuamente a nuevas situaciones.

4.1.3. Aplicaciones de la IA en vehículos autónomos

- **Reconocimiento de señales de tráfico:** utilizando técnicas de visión por computadora y redes neuronales, los vehículos pueden identificar y responder adecuadamente a las señales de tráfico. Es la principal aplicación de los coches autónomos, ya que permitirá al vehículo seguir el carril de forma adecuada sin la necesidad de la atención del conductor a la carretera.
- **Detección de obstáculos:** permite la detección y clasificación de obstáculos en la carretera, incluyendo vehículos, peatones y animales, garantizando una conducción segura. Es el gran reto de implementar a la perfección en los vehículos autónomos, ya que sería un gran avance para la total prevención de accidentes de tráfico.
- **Navegación y control de ruta:** gracias al análisis de datos de tráfico en tiempo real, existen sistemas de navegación que implementan algoritmos de planificación de rutas óptimas para evitar atascos o incluso gestionar el flujo de tráfico en áreas urbanas y optimizar la coordinación de semáforos, mejorando así la eficiencia y la movilidad en las ciudades [19].
- **Dashcams:** se incorporan cámaras de salpicadero en el vehículo para detectar comportamientos de riesgo por parte del conductor, como puede ser el uso del móvil durante la conducción o señales de fatiga, para poder alertar al conductor del riesgo que conlleva su comportamiento y evitar posibles accidentes.

4.2. Ética y seguridad en vehículos autónomos

Las decisiones que tiene que llegar a tomar un vehículo autónomo en una situación dada, puede ser elegir entre la vida o la muerte de un peatón o incluso del propio conductor del vehículo. Estas situaciones en las que una muerte es inevitable, son extremadamente complejas, por eso debe existir una normativa común que solucione todos los posibles dilemas que se puedan dar en la conducción autónoma [20].

4.2.1. Ética en la toma de decisiones

En Europa, la Comisión Europea está trabajando para permitir el uso de vehículos totalmente autónomos, estipulando un reglamento sobre la seguridad general de los vehículos formado por un conjunto normativo mínimo de sistemas avanzados de asistencia a la conducción (SAAC) necesario en todos los vehículos de carretera, además de los requisitos técnicos para vehículos totalmente autónomos de nivel 4 y 5 [20].

- **Algoritmos de decisión:** discutir cómo los algoritmos toman decisiones en situaciones de dilemas éticos, como en accidentes inevitables. Se deben dar respuesta a dilemas, tales como ¿atropellar a un peatón o chocar contra una terraza?, ¿atropellar a un bebé o a un anciano?, ¿poner en riesgo al conductor para evitar un accidente?...
- **Transparencia y explicabilidad:** lo más importante es diseñar un sistema transparente y cuya lógica de decisiones pueda ser explicada a los usuarios y reguladores. Uno de los aspectos más controvertidos es el decidir quién es el culpable en un accidente de un vehículo autónomo, ya que puede haber más de un responsable, habría que valorar el papel del fabricante, el desarrollador de los algoritmos, los fabricantes de los sensores o, incluso, el conductor humano de algún otro vehículo.

4.2.2. Seguridad y fiabilidad

Una vez aclarada la ética de los vehículos autónomos, uno de los retos más importantes es que, la conducción autónoma sea una tecnología segura y fiable, para que la gente no dude de si comprar un coche que incorpore esta innovación o no.

- **Ciberseguridad:** es vital proteger los sistemas autónomos de ataques cibernéticos que podrían comprometer la seguridad del vehículo, incluso provocando accidentes de manera intencional.
- **Pruebas y validación:** establecer estándares para probar y validar la fiabilidad de los sistemas autónomos antes de su despliegue comercial.
- **Redundancia y tolerancia a fallos:** diseñar sistemas con múltiples capas de seguridad y redundancia para minimizar el riesgo de fallos.

4.3. YOLOv9: You Only Look Once

YOLO (You Only Look Once) es una técnica de detección de objetos en tiempo real basada en redes neuronales convolucionales. Desarrollada por Joseph Redmon y sus colaboradores, YOLO ha revolucionado el campo de la visión por computadora al permitir la detección de objetos con alta precisión y velocidad [21].

4.3.1. Evolución de YOLO

Desde su introducción, YOLO ha pasado por varias versiones, mejorando la precisión y la eficiencia, y adaptada cada una a tareas específicas como la detección de objetos, la segmentación de instancias, la clasificación de imágenes, la estimación de poses o el seguimiento multiobjeto [21]:

- **YOLOv1:** introdujo la idea de detección de objetos en un solo paso, donde una sola red neuronal predice las clases y las ubicaciones de los objetos en una sola evaluación.
- **YOLOv2:** mejoró la precisión mediante la introducción de técnicas como la agrupación de anclajes y una arquitectura de red mejorada.
- **YOLOv3:** introdujo la detección de objetos a múltiples escalas y una estructura de red más profunda, mejorando la detección de objetos pequeños. Fue original de Joseph Redmon y es conocida principalmente por su eficaz capacidad de detección de objetos en tiempo real.
- **YOLOv4 y YOLOv5:** tanto la primera lanzada por Alexey Bochkovskiy en 2020, como la segunda lanzada en 2022 por Ultralytics, continuaron mejorando la precisión y la eficiencia mediante el uso de técnicas avanzadas de preprocesamiento de datos y optimización de red, ofreciendo mejores prestaciones y compensaciones de velocidad en comparación con las versiones anteriores.
- **YOLOv6 y YOLOv7:** introdujeron mejoras en la estructura de la red y técnicas de post-procesamiento, aumentando aún más la precisión y la velocidad. YOLOv6 fue lanzado en 2022 por Meituan y, se utiliza principalmente en muchos de los robots de reparto autónomos de empresas.
- **YOLOv8:** continuó la evolución con técnicas de entrenamiento mejoradas y optimizaciones específicas de hardware, incorporando funciones mejoradas como la segmentación de instancias, la estimación de pose o puntos clave y la clasificación.
- **YOLOv9:** incorpora avances en arquitectura de redes neuronales, técnicas de regularización y optimización, y mejora en la eficiencia del procesamiento, permitiendo una detección de objetos aún más precisa y rápida. Su desarrollo está basado en el código base de Ultralytics YOLOv5, pero implementando a mayores la Información de Gradiente Programable (PGI).

4.3.2. Componentes clave de YOLOv9

YOLOv9 introduce un enfoque innovador para abordar los principales desafíos en la detección de objetos a través del aprendizaje profundo, centrándose principalmente en los problemas de pérdida de información y eficiencia en la arquitectura de red. Este enfoque tiene cuatro componentes clave [22]:

- **The information bottleneck principle:** el principio del cuello de botella informativo indica cómo se produce la pérdida de información a medida que los datos se transforman dentro de una red neuronal, lo que supone un reto fundamental en el aprendizaje profundo. La ecuación del cuello de botella informativo representa la pérdida de información mutua entre los datos originales y los transformados a medida que los datos pasan por capas sucesivas de una red.

$$I(X, X) \geq I(X, f_{\theta}(X)) \geq I(X, g_{\phi}(f_{\theta}(X)))$$

Figura 23: Ecuación del cuello de botella informativo

Esta pérdida de información puede dar lugar a gradientes poco fiables y a una mala convergencia del modelo. Una de las soluciones es aumentar el tamaño del modelo para mejorar su capacidad de transformación de datos, reteniendo así más información, aunque esto no resuelve totalmente el problema. Por esta razón, YOLOv9 implementan la Información de Gradiente Programable (PGI), que ayuda a preservar los datos esenciales a través de la profundidad de la red, garantizando una generación de gradiente más fiable y, en consecuencia, una mejor convergencia y rendimiento del modelo.

- **Reversible Functions:** una función se considera reversible si puede invertirse sin pérdida de información. Esta propiedad permite a la red conservar un flujo de información completo, permitiendo así actualizaciones más precisas de los parámetros del modelo. YOLOv9 incorpora funciones reversibles en su arquitectura para mitigar el riesgo de degradación de la información y garantizar que no haya pérdida de información durante la transformación de los datos. Las funciones reversibles permiten a las redes mantener toda la información de entrada a través de todas las capas ya que los datos de entrada originales pueden reconstruirse perfectamente a partir de las salidas de la red.
- **Programmable Gradient Information (PGI):** la información de gradiente programable es un concepto novedoso introducido en YOLOv9 para combatir el problema del cuello de botella de la información, garantizando la conservación de los datos esenciales en las capas profundas de la red. PGI se desarrolla integrando tres componentes, cada uno de los cuales cumple una función distinta pero interrelacionada dentro de la arquitectura del modelo [22].

La rama principal garantiza que el modelo siga siendo ágil y eficiente durante esta fase crítica de interferencias sin sobrecarga computacional adicional. La rama auxiliar garantiza la generación de gradientes fiables y facilita la actualización precisa de los parámetros, aprovechando las funciones reversibles, ya que permite conservar y utilizar todos los datos

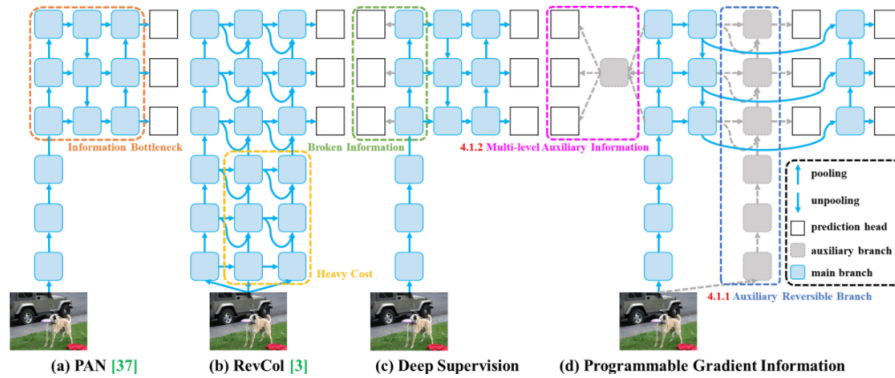


Figura 24: Arquitectura PGI

completos para el aprendizaje. La información auxiliar multinivel se utiliza para combinar la información de gradiente en todas las capas del modelo, para solucionar el problema de la pérdida de información en los modelos de supervisión profunda, y garantizando que el modelo comprenda plenamente los datos.

- **Generalized Efficient Layer Aggregate Network (GELAN):** GELAN representa un diseño único que se adapta al marco PGI con una arquitectura aún más refinada que permite a YOLOv9 lograr una utilización de parámetros y una eficiencia computacional superiores, permitiendo la integración flexible de varios bloques computacionales, lo que hace que YOLOv9 se adapte a una amplia gama de aplicaciones sin sacrificar la velocidad ni la precisión [23].

4.3.3. Funcionamiento de YOLOv9

El proceso de detección de objetos en YOLOv9 se puede resumir en los siguientes pasos:

- **División de la imagen:** la imagen de entrada se divide en una cuadrícula de celdas.
- **Predicción de celdas:** cada celda de la cuadrícula predice una caja delimitadora y una probabilidad de que la caja contenga un objeto de una clase específica, previamente introducidas para el entrenamiento.
- **Ajuste de anclas:** utiliza anclas predefinidas para ajustar mejor las cajas delimitadoras a los objetos detectados.
- **Filtrado de predicciones:** se aplican umbrales de confianza y técnicas de supresión de no-máximos para filtrar las predicciones redundantes y mantener solo las más relevantes.

En la búsqueda de una detección óptima de objetos en tiempo real, YOLOv9 destaca por su enfoque innovador para superar los retos de pérdida de información inherentes a las redes neuronales profundas. Al integrar PGI y la versátil arquitectura GELAN, YOLOv9 no sólo mejora la capacidad de aprendizaje del modelo, sino que también garantiza la retención de información crucial a lo largo del proceso de detección, logrando así una precisión y un rendimiento excepcionales.

CAPÍTULO 4. INTELIGENCIA ARTIFICIAL (IA)

Los avances de YOLOv9 están profundamente arraigados en la resolución de los retos que plantea la pérdida de información en las redes neuronales profundas. El principio del cuello de botella de información y el uso innovador de funciones reversibles son fundamentales para su diseño, lo que garantiza que YOLOv9 mantenga una alta eficiencia y precisión.[23].

4.4. Innovaciones y futuro de la IA en vehículos autónomos

4.4.1. Tecnologías emergentes

- **5G y V2X (Vehicle-to-Everything):** la conectividad avanzada y la comunicación entre vehículos e infraestructura pueden mejorar la seguridad y eficiencia de los vehículos autónomos.
- **Edge computing e IA distribuida:** el papel del edge computing en el procesamiento de datos en tiempo real y la mejora de la capacidad de respuesta de los sistemas autónomos, puede suponer un gran avance para la conducción autónoma, ya que reducirá la latencia y los tiempos de respuesta de manera considerable

4.4.2. Impacto social y económico

- **Impacto en el empleo:** un aspecto importante a tener en cuenta en un futuro es analizar cómo la automatización del transporte afectará a los trabajos relacionados con la conducción y qué medidas se pueden tomar para mitigar estos efectos.
- **Beneficios ambientales:** evaluar cómo los vehículos autónomos pueden contribuir a la reducción de emisiones y a la mejora de la eficiencia energética en el transporte.
- **Accesibilidad y movilidad:** los vehículos autónomos pueden servir de gran ayuda para personas con movilidad reducida o con discapacidades o personas que residan en áreas rurales o de difícil acceso, ya que facilitarán la conducción de una manera desmedida.

En conclusión, la integración de la inteligencia artificial en los vehículos autónomos ha abierto nuevas posibilidades en el ámbito del transporte, mejorando la seguridad, la eficiencia y la comodidad de los viajes. YOLOv9 representa un avance fundamental en la detección de objetos en tiempo real, ya que ofrece mejoras significativas en términos de eficacia, precisión y adaptabilidad, permitiendo a los vehículos percibir y reaccionar a su entorno con una precisión sin precedentes. La continua evolución de estas tecnologías, y abordando retos críticos mediante soluciones innovadoras como PGI y GELAN, YOLOv9 promete transformar aún más nuestro mundo, llevando la autonomía vehicular a nuevos niveles de sofisticación y fiabilidad.

Capítulo 5

Informe económico

En este capítulo vamos a realizar la valoración económica de todo el despliegue de la red explicado en el capítulo 3.2 en el que se han comentado los elementos principales de las tres partes que componen la red: las estaciones base, el core y la red de conexión entre ambas. Además de los elementos que se han mencionado hay otros elementos que componen estas partes y que hay que tener en cuenta.

5.1. Costes

Comenzamos con la valoración económica de los elementos de las estaciones base. La torre será el lugar donde estén montadas las antenas de cada estación base. Se ha realizado el estudio de cobertura con las antenas colocadas a 20 metros de altura, por lo que cada torre deberá tener como mínimo esta altura. Una torre de telefonía móvil de unos 20 metros tiene un precio aproximado de unos 1.500€ [24]. Será necesario construir 13 torres, las otras 12 se aprovechan las que ya están montadas a lo largo de la autovía.

En cada torre irán montadas las antenas del modelo Ubitiqui AM-2G16-90 [25], que tienen un precio de 140€ cada una y las RRU del modelo Huawei RRU5301 que tienen un precio de 112€ [26]. En cada torre se montarán dos de cada, salvo en las torres de los extremos que solo será necesario una antena y una RRU.

Para todas las estaciones base habrá dos BBU Huawei BBU3910 que tienen un precio de 750€ cada una.

Además de estos elementos principales hay otros elementos montados en cada estación base que incluyen conectores, cables de conexión entre la antena y el RRU, aisladores, jumpers, pararrayos para cada torre que se valorarán en conjunto como 1.000€ por estación base.

Dentro de los gastos de las estaciones base también se deben incluir los gastos debidos a la construcción y compra de terrenos. Una parcela de la una hectárea en la zona de Valladolid ronda

CAPÍTULO 5. INFORME ECONÓMICO

los 8000€ en promedio. Para construir una estación base no se necesita tanto terreno, basta con unos 200-300m² más las servidumbres. Tasaremos la compra en unos 1.000-2.000 € y unos gastos de construcción e instalación de las torres de 130.000€ [27]. Las BBU se construirán en un terreno compartido junto a dos de las torres por lo que no hará falta incluir gastos de compra de terrenos adicionales. Sí será necesario una caseta para guardar todos los equipos.

Elementos	Características	Precio (€)	Cantidad	Precio total (€)
Torre	20m	1.500	13	19.500
Antena	Ubitiqui AM-2G16-90	140	47	6.580
RRU	Huawei RRU5301	112	47	5.264
BBU	Huawei BBU3910	750	2	1.500
Caseta BBU		1.000	2	2.000
Otros elementos	conectores, aisladores, jumpers, cables, para-rayos ...	1.000	25	25.000
Compra de terrenos	200m ²	2.000	13	26.000
Obra de construcción e instalación		130.000	13	1.690.000
Total estaciones base				1.775.844

Tabla 2: Desarrollo de los gastos asociados a las estaciones base

La parte de despliegue del core se compone principalmente del equipo que provee el core de la red 4G y del servidor para la inteligencia artificial. El equipo para el core 4G del modelo IPLOOK IKEPC 510 tiene un precio de 50.000€ [28] y el equipo para la inteligencia artificial del modelo Azure NCasT4 v3, de 3.500€. A parte de estos equipos, se necesitan otros elementos adicionales para que todo funcione correctamente como pueden ser baterías, sistemas de alimentación, sistemas de control, sistemas de seguridad, cableado adicional y conectores, etc. Esto estará valorado en 10.000€.

Elementos	Características	Precio (€)	Cantidad	Precio total (€)
Servidor core 4G	IPLOOK IKEPC510	50.000	1	50.000
Servidor IA	Azure NCasT4 v3	3.500	1	3.500
Otros equipos	Sistemas de alimentación, baterías, sistemas de control, cableado, conectores	10.000	1	10.000
Terreno		2.000	1	2.000
Total core				60.500

Tabla 3: Desarrollo de los gastos asociados al core

Para el despliegue de la red de distribución es necesario calcular los kilómetros de fibra para el backhaul y para el fronthaul. La conexión entre el core y la BBU de Alaejos es de 15 km y la conexión entre el core y la BBU de Aldeanueva de Figueroa es de otros 15 kilómetros. Esta es la parte de backhaul, que también requiere de los OLT y ONT. Estos están valorados en 350

CAPÍTULO 5. INFORME ECONÓMICO

y 50€ respectivamente. Para el fronthaul son necesarias las conexiones desde las BBU hasta cada torre. Estas conexiones requerirán unos 100km de fibra, que será una longitud un poco mayor a la longitud de la autovía. El despliegue de la fibra se muestra en la figura 13. El coste del kilómetro de fibra es de unos 5.000 euros y la obra para el despliegue de la fibra supone un gasto de 3.000.000€, basados en otras obras de similar magnitud. Esto incluye la maquinaria, la mano de obra, los conductos y demás elementos.

Elementos	Características	Precio (€)	Cantidad	Precio total (€)
ONT	Huawei EG8245H	50	2	100
OLT	Huawei MA5800-X2	350	1	350
Fibra		5.000€/km	130	650.000
Despliegue fibra		3.000.000	1	3.000.000
Total red distribución				3.650.450

Tabla 4: Desarrollo de los gastos asociados a la red de distribución

El coste total del despliegue de todas las estaciones base, del core y de la red de distribución es de:

$$Costetotal = 1,775,844 \text{ €} + 60,500 \text{ €} + 3,650,450 \text{ €} = 5,486,794 \text{ €} \quad (6)$$

Capítulo 6

Conclusiones y líneas futuras

6.1. Conclusiones

En este apartado se van a recoger las principales conclusiones que se han obtenido con el desarrollo del proyecto acerca de cada uno de los puntos que se han llevado a cabo, incluyendo la planificación, la elección de elementos hardware y software para el desarrollo del demostrador, las conclusiones que se han obtenido de la red 4G que se va a desplegar y de la inteligencia artificial. Las conclusiones son las siguientes:

Es importante hacer una planificación adecuada del proyecto, realizando un análisis de las posibles tareas que hay que llevar a cabo. Es importante hacer un seguimiento semanal entre los miembros del equipo de trabajo para ver los avances y las posibles mejoras, dando cada miembro su punto de vista. Conforme avanza el proyecto es importante determinar los posibles cambios que se pueden hacer antes de que sea demasiado tarde y no tengan remedio.

Antes de iniciar todo el proceso es importante elegir los medios hardware y software más adecuados para llevar a cabo el proyecto. Es importante estudiar que el hardware de todos los elementos es compatible entre sí, que el software que se va a usar es compatible con ese hardware y que va seguir recibiendo actualizaciones por parte de los desarrolladores para futuras actualizaciones del producto.

Respecto al despliegue de la red, es interesante realizar simulaciones con algún software para estudiar la cobertura, y determinar la situación óptima para cada una de las antenas.

La red 4G tiene limitaciones de latencia que pueden ser un problema para algunas aplicaciones en tiempo real como puede ser el servicio de vehículo conectado desarrollado en este proyecto.

Utilizar la tecnología de 4G a las frecuencias que se nos han dado como disponibles hace que se requieran un gran número de estaciones base para dar una cobertura óptima para el servicio en todos los puntos de la autovía. Poner un gran número de estaciones base eleva considerablemente los costes del despliegue.

CAPÍTULO 6. CONCLUSIONES Y LÍNEAS FUTURAS

El despliegue de toda la red requiere de una inversión inicial muy grande, y es necesario realizar un estudio previo del retorno que tendrá la aplicación en cuanto la disponibilidad de los usuarios de utilizar el servicio.

En lo que se refiere a la inteligencia artificial, se necesita un dataset muy grande, completo y detallado con gran cantidad de imágenes y con todo tipo de señales para que el servicio funcione correctamente, de lo contrario, habrá muchas ocasiones en las que el sistema fallará, lo que es peligroso para la aplicación para la que lo estamos desarrollando. Tener un dataset amplio y completo es una tarea difícil, que requiero mucho tiempo y muchas pruebas de tomar imágenes en todas las posibles situaciones. Además, escoger el modelo adecuado para cada aplicación es una parte fundamental.

Para llevar a cabo las tareas de detecciones de imágenes mediante inteligencia artificial también es importante tener un hardware adecuado que pueda procesar en tiempo real las imágenes. Esto ha quedado demostrado en el demostrador realizado en el que la ejecución del modelo es lenta, lo que hace que sea inviable aplicarlo en tiempo real.

6.2. Líneas futuras

La principal línea futura de este proyecto sería continuar con el desarrollo de un sistema de detección de imágenes más complejo, en el que además de señales detecte más elementos de la carretera como pueden ser las líneas de la carretera, otros vehículos, personas e informe al conductor en todo momento de la situación en la carretera: cuando detenerse, cuando parar, si debe tener cuidado al cambiar de carril al estar rodeado de muchos coches, etc.

Otro posible punto de desarrollo de la aplicación puede ser ofrecer otro tipo de información a través de la red. Sería interesante utilizar la red como medio de comunicación entre todos los coches y que entre todos ellos se compartan información sobre el estado de la carretera, el estado del tráfico, el estado de los semáforos, información acerca de aparcamientos libres, etc.

Otra posible línea futura sería modificar la arquitectura desarrollada en la que el procesamiento se realiza en el core de la red para adaptarla al paradigma de edge-computing, ya que permitiría disminuir considerablemente la latencia del servicio.

Otro punto de desarrollo puede ser la implantación de una red 5G en vez de 4G, que permitiría mejores tiempos de latencia, mejores capacidades en la red y permitiría un uso mayor y más eficiente de los servicios.

Y por último el paso final sería pasar del vehículo conectado al vehículo autónomo. Esto requeriría, además de la visión artificial, el uso de posicionamiento GPS y el uso de otros múltiples sensores, para monitorizar en todo momento la posición respecto a otros vehículos y/u objetos. Este salto sería un paso al que todavía quedaría mucho para llegar aunque podríamos ir avanzando a través de los distintos niveles de conducción autónoma.

CAPÍTULO 6. CONCLUSIONES Y LÍNEAS FUTURAS

Este campo de desarrollo tiene muchas posibilidades y permite muchas aplicaciones que pueden ser de mucha utilidad a los usuarios.

Anexo I

Desarrollo del demostrador

En esta sección de la memoria se describen los pasos necesarios que se han seguido para el desarrollo y despliegue de un pequeño demostrador cuyo objetivo es enseñar a los clientes el correcto funcionamiento a pequeña escala del servicio 4G LTE que se desplegaría en el tramo de autovía A-62 indicado anteriormente. Para ello, han sido necesarios una serie de materiales para poder desarrollarlo, así como piezas de software fundamentales que lo han hecho posible.

El objetivo del demostrador consiste en la configuración de una red 4G LTE para que un vehículo en miniatura se conecte a la red a través de una SIM y pueda hacer uso de un servicio desplegado en el núcleo de la red (EPC). El servicio que se ofrece en el demostrador es el análisis de imágenes individuales cada cierto intervalo de tiempo, captadas por la cámara del vehículo y enviados haciendo uso de la red ofrecida. Dicho servicio es distinto al que se ofrecerá en el tramo de autovía debido a las limitaciones que presentan los componentes hardware empleados.

Paso 0. Materiales necesarios

Antes de comenzar con el desarrollo del demostrador, se debe analizar el objetivo final del proyecto y escoger los materiales hardware necesarios para llevarlo a cabo. Dado que se trata de un servicio en una red SDR (Software Defined Radio, Red Definida por Software) de vehículo conectado, nos dimos cuenta de la variedad de elementos que necesitamos:

- Un ordenador de sobremesa en donde correrán los softwares relacionados con la SDR (el software del eNodeB y del EPC), el servicio basado en IA (Inteligencia Artificial) que se encarga de la detección y clasificación de señales entrenado con señales de juguete y, por último, el servidor mosquito que permite la conexión interactiva en tiempo real con el vehículo teledirigido.
- Una memoria USB de 20 GB de capacidad empleada como almacenamiento extra para nuestro EPC y para la instalación del SO del core.

ANEXO I. DESARROLLO DEL DEMOSTRADOR

- Un sistema SDR (Software Defined Radio) BladeRF 2.0 xa9. Dicho sistema será utilizado para poder establecer conexiones inalámbricas entre los UE y el eNodeB de la red mediante el uso de antenas.
- Un vehículo Amazon DeepRacer Evo dirigido por control remoto con una cámara conectada. El vehículo será el dispositivo que va a hacer uso de la red para poder conectarse al servicio de detección de señales en vídeo.
- Un módem 4G LTE de Huawei y una tarjeta SIM de Vodafone S.A.U. ambos programables para que el vehículo pueda conectarse al eNodeB y, por lo tanto, hacer uso del servicio de detección de señales.
- Frecuencias de banda 7 concedidas por la empresa Vodafone a utilizar en nuestro entorno de laboratorio. Dichas bandas de frecuencias van de 2540 a 2550 MHz y de 2660 a 2670 MHz.

Junto a todos estos elementos hardware escogidos se encuentran todos los sistemas software de código abierto utilizados para la configuración y funcionamiento coordinado de todos los dispositivos. Dichos softwares fueron elegidos a medida que se desarrolló el prototipo y sus descripciones detalladas se encuentran más adelante en este Anexo.

Paso 1. Instalación y desarrollo del ENodeB y core de la red 4G

El primer paso en el desarrollo del prototipo es la configuración del ordenador a nuestra disposición en donde funcionará el software relacionado con el EPC y el ENodeB. Para comenzar, fue necesario escoger un sistema operativo y el software de nuestra red SDR compatibles entre ellos.

Debido a nuestra necesidad de escoger un SO de código abierto, nuestras opciones fueron reducidas, quedando solamente tres opciones finales: Debian, Fedora o Ubuntu. Al mismo tiempo, se barajaron distintos softwares SDR de licencia gratuita, siendo las opciones barajadas open5Gs, OpenAirInterface y srsRan. Para elegir ambos softwares, creamos varios DAFOs (Debilidades, Amenazas, Fortalezas y Oportunidades) de cada una de las opciones con el objetivo de valorar las ventajas y desventajas de cada una (Figuras 25, 26 y 27).

Si se analizan los DAFOs en profundidad, nos percatamos que las tres opciones son más o menos similares. Sin embargo, debíamos elegir también un SO compatible con dichos softwares. Finalmente, tras barajar todas las opciones posibles, se decidió el software srsRAN (srsRAN_4G versión 23.11) junto a la instalación de Ubuntu (versión 20.04.6 en nuestro caso) como SO compatible. Esta elección es la más razonable debido a que son softwares compatibles entre sí en dichas versiones, y existe un respaldo de mantenimiento de ambos softwares activo hasta al menos el año 2025.

En primer lugar, comenzaremos con la instalación del SO en nuestro ordenador de escritorio.

ANEXO I. DESARROLLO DEL DEMOSTRADOR



Figura 25: DAFO software srsRAN.



Figura 26: DAFO software Open5Gs.



Figura 27: DAFO software OpenAirInterface.

ANEXO I. DESARROLLO DEL DEMOSTRADOR

Para llevar a cabo la instalación de Ubuntu, hay que utilizar una memoria USB booteable con la imagen del SO previamente descargada y almacenada en la memoria. Posteriormente lo configuramos con la configuración básica, es decir, con los programas mínimos, y seguidamente creamos el directorio del proyecto, en nuestro caso lo hemos llamado “Coche_autonomo“. Ese directorio va a ser de utilidad ya que es donde se va almacenar toda la información necesaria para la instalación del software srsRAN y programación de la SDR.

A continuación, comenzamos con el proceso de instalación y configuración del software SDR relacionado con la bladeRF. Para lograrlo, hemos seguido la documentación proporcionada por los desarrolladores del software srsRAN [29].

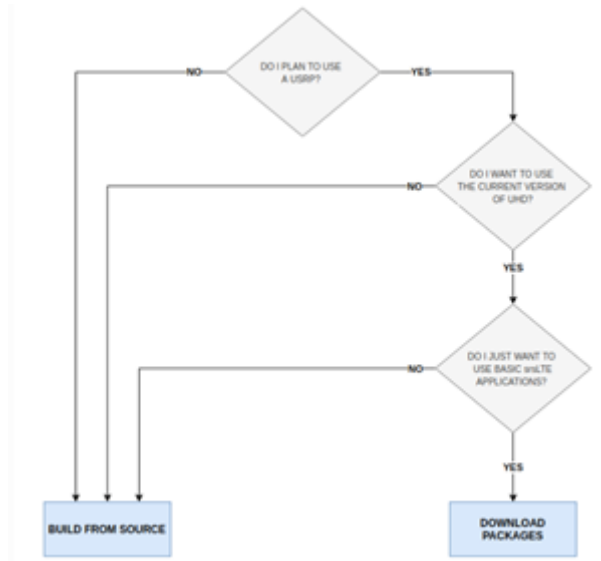


Figura 28: Árbol de decisión del tipo de instalación srsRAN [29].

El primer paso de todos es realizar la instalación en nuestro ordenador core todo el software de configuración inicial. Dadas las preguntas de la Figura 28, hemos elegido la opción de instalación de los paquetes de software directamente desde las fuentes de desarrolladores frente a la instalación básica de paquetes debido a que, para nuestro proyecto, no vamos a utilizar URSP (UE Router Selection Policy), la cual es una técnica empleada en 5G que proporciona información sobre qué sesión de PDU en una parte de la red debe utilizar un servicio o aplicación activa. Además, queremos modificar la configuración de la srsRAN_4G ya que no vamos a usar el software UE que se nos proporciona. Por lo tanto, para llevar a cabo la instalación correcta es necesario ejecutar los siguiente comando:

```
$ sudo apt-get install build-essential cmake libfftw3-dev libmbedtls-dev libboost-program-options-dev libconfig++-dev libsctp-dev.
```

Finalizada la instalación inicial, continuamos con la descarga e instalación de todos los RF-front-end drivers obligatorios para poder conectar la bladeRF al ordenador mediante el software srsRAN. Los drivers que son necesarios para el correcto funcionamiento son: UHD, SoapySDR, BladeRF y ZeroMQ. A mayores, nuestro sistema bladeRF necesita el plugin SopayBladeRF para

poder funcionar correctamente.

DAFO srsRAN

Debilidades

- **Dependencia de la comunidad de código abierto:** Falta de soporte técnico. Para la resolución de problemas hay que acudir a foros.
- **Complejo:** La curva de aprendizaje es dura, ya que la instalación, configuración y uso de srsRAN requieren conocimientos técnicos avanzados, lo cual puede ser una barrera para nuevos usuarios.
- **Escalabilidad y rendimientos limitados:** En comparación con soluciones comerciales, srsRAN tiene limitaciones en términos de rendimiento y escalabilidad. También puede haber bugs y errores no resueltos.

Amenazas

- **Cambios en los estándares de la industria:** La rápida evolución de las tecnologías de telecomunicaciones puede hacer que srsRAN quede obsoleto.
- **Competencia de soluciones comerciales:** La existencia de soluciones comerciales robustas y con soporte técnico dedicado puede ser una amenaza. También otros softwares de código abierto.
- **Seguridad y vulnerabilidades:** Puede ser vulnerable a ataques y exploits que, al ser un código abierto, son más fáciles de identificar.

Fortalezas

- **Software de código abierto y gratuito:** Supone un coste cero y permite que cualquier persona pueda acceder a su código fuente.
- **Compatibilidad y estandarizaciones:** Cumple con los estándares de 4G y 5G.
- **Flexibilidad y personalización:** Al ser un software de código abierto, srsRAN permite la personalización de la configuración de la red sin limitaciones software.

ANEXO I. DESARROLLO DEL DEMOSTRADOR

Oportunidades

- **Expansión del mercado de redes 5G:** Con la expansión actual de las redes 5G, existe una creciente demanda de soluciones que sean copompatibles con esta tecnología, dando posibilidades a los desarrolladores de emplear srsRAN en proyectos de investigación, desarrollo y pruebas de 5G.
- **Colaboraciones:** La colaboración con universidades y centros de investigación puede fomentar el desarrollo de nuevas características y mejoras.
- **Mejoras continuas:** Al ser un código abierto, le permite una evolución y mejora continua, con actualizaciones y nuevas funcionalidades basadas en las necesidades de la comunidad. También ofrece la posibilidad de ser integrado en nuevas tecnologías en un futuro.

A. UHD

Para instalar este primer driver en el core, hemos seguido la guía de instalación de la página del GitHub de UHD en Linux (Ubuntu). El primer paso es instalar todas las dependencias que necesita el driver para poder funcionar en Ubuntu, ejecutando el siguiente comando [30]:

```
$ sudo apt-get install autoconf automake build-essential ccache cmake cpufrequtils doxygen  
ethtool g++ git inetutils-tools libboost-all-dev libncurses5 libncurses5-dev libusb-1.0-0 libusb-1.0-  
0-dev libusb-dev python3-dev python3-mako python3-numpy python3-requests python3-scipy  
python3-setuptools python3-ruamel.yaml
```

El siguiente paso es clonar el repositorio git en nuestro directorio del proyecto de los desarrolladores del UHD:

```
$ git clone https://github.com/EttusResearch/uhd.git
```

Tras ejecutar dicho comando, se generará un directorio llamado *uhd*, en donde trabajaremos a partir de aquí. A continuación, generamos los Makefiles:

```
$ cd uhd/host  
$ mkdir build  
$ cd build  
$ cmake ../
```

Por último, construimos el software descargado, lo instalamos y creamos la ruta de librerías con los siguientes comandos:

```
$ make
$ make test # This step is optional
$ sudo make install
$ sudo ldconfig
```

B. SoapySDR

El segundo driver que necesitamos instalar en nuestro core es SoapySDR. SoapySDR es una biblioteca de software y una API (Interfaz de Programación de Aplicaciones) diseñada para proporcionar una interfaz unificada para trabajar con dispositivos de radio definido por software (SDR). Hemos seguido los siguientes pasos:

1. Conseguimos las dependencias necesarias (CMake y un compilador C++) utilizando python3:
\$ sudo apt-get install cmake g++ libpython3-dev python3-numpy swig
2. Conseguimos el código fuente a través de la herramienta Git:
\$ git clone https://github.com/pothosware/SoapySDR.git
3. Al realizar el clonado del repositorio, se crea un directorio llamado SoapySDR. A continuación entramos en el directorio creado y completamos la compilación e instalación:
\$ cd SoapySDR
\$ mkdir build
\$ cd build
\$ cmake ..
\$ make -j`nproc`
\$ sudo make install -j`nproc`
4. Por último, creamos la ruta de librerías del plugin:
\$ sudo ldconfig #needed on debian systems

C. BladeRF

Otro driver necesario es el BladeRF, debido a que vamos a utilizar una bladeRF ax9 como elemento hardware que actúa de antena para poder desplegar nuestro servicio. Es necesario su instalación para poder programarla y poder interactuar con ella. Para ello hemos realizado los siguientes pasos:

1. Activamos PPA y los ficheros con las cabeceras de la gnuradio a desarrollar:
\$ sudo add-apt-repository ppa:nuandllc/bladerf
\$ sudo apt-get update

ANEXO I. DESARROLLO DEL DEMOSTRADOR

```
$ sudo apt-get install bladerf
$ sudo apt-get install libbladerf-dev
```

2. Instalamos el firmware y la imagen FPGA necesaria para el buen funcionamiento de la bladeRF.

```
$ sudo apt-get install bladerf-firmware-fx3
$ sudo apt-get install bladerf-fpga-hostedxa9
```

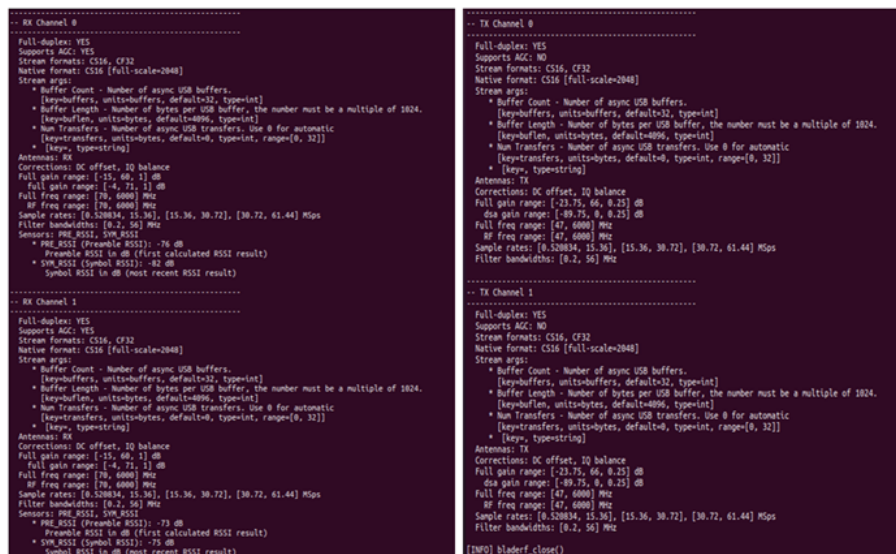
D. SoapyBladeRF

Una vez instalados todos los drivers, es necesario una última instalación, en este caso el plugin SoapySDR, el cual nos permite usar la Blade RF dentro de la API SoapySDR y el software soportado por SoapySDR. Para ello, simplemente hay que ejecutar los siguientes comandos:

```
$ git clone https://github.com/pothosware/SoapyBladeRF.git
$ cd SoapyBladeRF
$ mkdir build
$ cd build
$ cmake ..
$ make
$ sudo make install
```

Gracias a este plugin, ya podremos detectar la Blade RF desde nuestro ordenador al ejecutar este comando:

```
$ SoapySDRUtil -probe="driver=bladerf"
```



```
-- RX Channel 0
-----
Full-duplex: YES
Supports AGC: YES
Stream formats: CS16, CF32
Native format: CS16 [full-scale=2048]
Stream args:
  * Buffer Count - Number of async USB buffers.
    [keybuffers, units=buffers, default=32, type=int]
  * Buffer Length - Number of bytes per USB buffer, the number must be a multiple of 1024.
    [keybuflen, units=bytes, default=4096, type=int]
  * Num Transfers - Number of async USB transfers. Use 0 for automatic.
    [keytransfers, units=bytes, default=0, type=int, range=[0, 32]]
  * [key, type=string]
Antennas: RX
Corrections: DC offset, IQ balance
Full gain range: [-15, 60, 1] dB
Full gain range: [-4, 71, 1] dB
Full freq range: [70, 6000] MHz
RF freq range: [70, 6000] MHz
Sample rates: [0.520834, 15.36], [15.36, 30.72], [30.72, 61.44] Msps
Filter bandwidths: [0.2, 56] MHz
Sensors: PWR_RSSI, SNR_RSSI
  * PWR_RSSI (Preamble RSSI): -76 dB
  * Preamble RSSI in dB (first calculated RSSI result)
  * SNR_RSSI (Symbol RSSI): -82 dB
  * Symbol RSSI in dB (most recent RSSI result)
-----
-- RX Channel 1
-----
Full-duplex: YES
Supports AGC: YES
Stream formats: CS16, CF32
Native format: CS16 [full-scale=2048]
Stream args:
  * Buffer Count - Number of async USB buffers.
    [keybuffers, units=buffers, default=32, type=int]
  * Buffer Length - Number of bytes per USB buffer, the number must be a multiple of 1024.
    [keybuflen, units=bytes, default=4096, type=int]
  * Num Transfers - Number of async USB transfers. Use 0 for automatic.
    [keytransfers, units=bytes, default=0, type=int, range=[0, 32]]
  * [key, type=string]
Antennas: RX
Corrections: DC offset, IQ balance
Full gain range: [-15, 60, 1] dB
Full gain range: [-4, 71, 1] dB
Full freq range: [70, 6000] MHz
RF freq range: [70, 6000] MHz
Sample rates: [0.520834, 15.36], [15.36, 30.72], [30.72, 61.44] Msps
Filter bandwidths: [0.2, 56] MHz
Sensors: PWR_RSSI, SNR_RSSI
  * PWR_RSSI (Preamble RSSI): -73 dB
  * Preamble RSSI in dB (first calculated RSSI result)
  * SNR_RSSI (Symbol RSSI): -73 dB
  * Symbol RSSI in dB (most recent RSSI result)
-----
-- TX Channel 0
-----
Full-duplex: YES
Supports AGC: NO
Stream formats: CS16, CF32
Native format: CS16 [full-scale=2048]
Stream args:
  * Buffer Count - Number of async USB buffers.
    [keybuffers, units=buffers, default=32, type=int]
  * Buffer Length - Number of bytes per USB buffer, the number must be a multiple of 1024.
    [keybuflen, units=bytes, default=4096, type=int]
  * Num Transfers - Number of async USB transfers. Use 0 for automatic.
    [keytransfers, units=bytes, default=0, type=int, range=[0, 32]]
  * [key, type=string]
Antennas: TX
Corrections: DC offset, IQ balance
Full gain range: [-23.75, 66, 0.25] dB
Full gain range: [-89.75, 0, 0.25] dB
Full freq range: [47, 6000] MHz
RF freq range: [47, 6000] MHz
Sample rates: [0.520834, 15.36], [15.36, 30.72], [30.72, 61.44] Msps
Filter bandwidths: [0.2, 56] MHz
-----
-- TX Channel 1
-----
Full-duplex: YES
Supports AGC: NO
Stream formats: CS16, CF32
Native format: CS16 [full-scale=2048]
Stream args:
  * Buffer Count - Number of async USB buffers.
    [keybuffers, units=buffers, default=32, type=int]
  * Buffer Length - Number of bytes per USB buffer, the number must be a multiple of 1024.
    [keybuflen, units=bytes, default=4096, type=int]
  * Num Transfers - Number of async USB transfers. Use 0 for automatic.
    [keytransfers, units=bytes, default=0, type=int, range=[0, 32]]
  * [key, type=string]
Antennas: TX
Corrections: DC offset, IQ balance
Full gain range: [-23.75, 66, 0.25] dB
Full gain range: [-89.75, 0, 0.25] dB
Full freq range: [47, 6000] MHz
RF freq range: [47, 6000] MHz
Sample rates: [0.520834, 15.36], [15.36, 30.72], [30.72, 61.44] Msps
Filter bandwidths: [0.2, 56] MHz
-----
[INFO] bladerf_close()
```

Figura 29: Detección de la bladeRF.

E. ZeroMQ

El siguiente software a instalar y configurar es el software ZeroMQ, también conocido como ØMQ. Es una biblioteca de mensajería asíncrona de alto rendimiento que permite el intercambio de mensajes entre diferentes componentes de software del mismo sistema. Es ampliamente utilizado en aplicaciones que requieren una comunicación eficiente y de baja latencia entre procesos. Para poder instalarlo correctamente se han seguido los siguientes pasos:

1. Instalar las librerías básicas de desarrollador de ZeroMQ para Ubuntu:
\$ sudo apt-get install libzmq3-dev
2. Instalar la librería *libzmq*:
\$ git clone https://github.com/zeromq/libzmq.git
\$ cd libzmq
\$./autogen.sh
\$./configure
\$ make
\$ sudo make install
\$ sudo ldconfig
3. Instalar la librería *czmq*:
\$ git clone https://github.com/zeromq/czmq.git \$ cd czmq \$./autogen.sh \$./configure \$ make \$ sudo make install \$ sudo ldconfig

F. Software srsRAN 4G

Tras instalar todos los drivers necesarios para el control y manejo de la BladeRF, es necesario instalar el software srsRAN que hará posible crear y configurar la Red Definida por Software (SDR). Tal y como hemos visto, es un conjunto de herramientas de código abierto para redes de acceso por radio (RAN) diseñado para permitir la investigación, el desarrollo y la implementación de redes móviles. Para instalarlo en nuestro dispositivo, hemos ejecutado estos comandos para la descarga, compilación e instalación del software:

```
$ git clone https://github.com/srsRAN/srsRAN_4G.git
$ cd srsRAN_4G
$ mkdir build
$ cd build
$ cmake ../
$ make
$ make test
$ sudo make install
$ srsran_install_configs.sh user
```

ANEXO I. DESARROLLO DEL DEMOSTRADOR

Al finalizar la ejecución de los comandos anteriormente descritos, ya disponemos de la todo lo necesario para desplegar una SDR con la configuración inicial predeterminada (la tendremos que personalizar más adelante) y de avanzar en el desarrollo del demostrador.

Paso 2. Configuración de la SDR

La SDR que vamos a desplegar requiere de la configuración del eNodeB y del EPC ya que con la configuración predeterminada que nos ofrece srsRAN no es válida. Para empezar, tanto para el EPC como para el eNodeB se deben descargar los paquetes binarios para Ubuntu e instalar los ficheros de configuración de ambos:

```
$ sudo add-apt-repository ppa:srslte/releases
$ sudo apt-get update
$ apt-get install srsepc
$ sudo apt-get install srsenb
$ srsran_4g_install_configs.sh user
```

A continuación, comenzamos con la configuración del EPC, el cual se configura mediante la modificación de los ficheros *epc.conf* y del *user_db.csv* (ambos guardados en el directorio */root/.config/srsran/*) en primer lugar. Se trata de introducir y modificar los parámetros de configuración de la celda de radiación de la antena. Comenzamos con el fichero *epc.conf*, el cual contiene información general sobre los parámetros sobre los servicios del core MME, SGW, PGW y HSS. Esta información la extrajimos de la información de la SIM programable para darla de alta en nuestro sistema y, por tanto, permitir al dispositivo que la utilice conectarse a nuestra red.

Para permitir la conectividad del módem, es necesario configurar un APN (Access Point Name) en el modem que va a llevar la SIM, debido a que el APN actúa como una puerta de entrada que conecta el dispositivo móvil a la red de datos del proveedor de servicios móviles (en este caso, nuestra red). Lo hemos llamado APNgrupo8.

En el fichero *epc.conf* hemos modificado el valor del servicio MME del core, cambiando los valores de los parámetros mcc (901) y mnc (70) con los primeros números del IMSI de la SIM. Además, cambiamos el nombre del APN por el nombre del que hemos configurado anteriormente (APNgrupo8). El resultado final de la configuración del fichero *epc.conf* puede verse en la Figura 30.

Por otro lado, también modificamos el fichero *user_db.csv*, pero esta vez es para registrar en nuestro sistema el APN anteriormente configurado. Por lo tanto, hemos añadido una línea con los parámetros de identificación del dispositivo, separados los valores por comas. Los parámetros que se han introducido son:

ANEXO I. DESARROLLO DEL DEMOSTRADOR

```
[mme]
mme_code = 0x1a
mme_group = 0x0001
tac = 0x0007
ncc = 901
nnc = 70
mme_bind_addr = 127.0.0.100
apn = APNgrupo8
dns_addr = 8.8.8.8
encryption_algo = EEA0
integrity_algo = EIA1
paging_timer = 2
request_inetiv = false
lac = 0x0006

#####
# HSS configuration
#
# db_file:          Location of .csv file that stores UEs information.
#
#####
[hss]
db_file = user_db.csv
```

Figura 30: Configuración del fichero *epc.conf*.

- UE_name: Nombre legible para facilitar la lectura del logs del EPC.
- Auth: Algoritmo de autenticación (xor/mil). MILENAGE es el algoritmo utilizado en la mayoría de las redes, el algoritmo XOR se utiliza principalmente en equipos de prueba tarjetas USIM de prueba.
- IMSI del UE. Es un número único para identificar al usuario dentro de la red pública. En redes privadas puede no ser único. Los primeros 5 dígitos del IMSI son el MCC (3) y el MNC (2) y identifican al país y a la red de emisión de tarjetas dentro del país, respectivamente. En nuestro caso es: 901700000052128.
- Key: clave secreta que utilizan el HSS (Home Subscriber Server) y el UE para autenticarse mutuamente.
- Tipo OPC. Tipo de código del operador.
- Código OPC. Código del operador.
- AMF: campo de gestión de autenticación.
- SQN: Número de secuencia del UE para la actualización de la autenticación.
- QCI: Identificador de clase de QoS.
- IP Alloc: Dirección IP que se le va a asignar al usuario (puede ser dynamic). Define la estrategia de asignación de IP para el SPGW.

```
# .csv to store UE's information in HSS
# Kept in the following format: "Name,Auth,IMSI,Key,OP_Type,OP/OPc,AMF,SQN,QCI,IP_alloc"
#
# Name:      Human readable name to help distinguish UE's. Ignored by the HSS
# Auth:      Authentication algorithm used by the UE. Valid algorithms are XOR
#            (xor) and MILENAGE (mil)
# IMSI:      UE's IMSI value
# Key:       UE's key, where other keys are derived from. Stored in hexadecimal
# OP_Type:   Operator's code type, either OP or OPc
# OP/OPc:    Operator Code/Cyphered Operator Code, stored in hexadecimal
# AMF:       Authentication management field, stored in hexadecimal
# SQN:       UE's Sequence number for freshness of the authentication
# QCI:       QoS Class Identifier for the UE's default bearer.
# IP_alloc:  IP allocation strategy for the SPGW.
#            with 'dynamic' the SPGW will automatically allocate IPs
#            with a valid IPv4 (e.g. '172.16.0.2') the UE will have a statically assigned IP.
#
# Note: Lines starting by '#' are ignored and will be overwritten
ue2_mil,001010123456789,00112233445566778899aabbccddeeff,opc,63bfa50ee6523365fff14c1f45f88737d,0000,000000001234,7,dynamic
ue1_xor,001010123456789,00112233445566778899aabbccddeeff,opc,63bfa50ee6523365fff14c1f45f88737d,9001,000000001234,7,dynamic
APNgrupo8_mil,9017000000052128,7c9e0122bd85c05077fe535c82867bf,opc,acce99ce1b0f50eba85a9145f3cda4d,9000,000000001230,9,172.16.0.2
```

Figura 31: Configuración del fichero *user_db.csv*.

Una vez el EPC está configurado, procedemos a configurar la parte de la estación base, es decir, el eNodeB. Con respecto a esta parte, solamente es necesario configurar el fichero *enb.conf*

ANEXO I. DESARROLLO DEL DEMOSTRADOR

que se encuentra en la carpeta `/root/.config/srsran/`, por lo que resulta más sencillo y más rápido. Primero, debemos cambiar los valores del `mcc` y del `mnc` con los mismos valores que en el EPC (901 y 70 respectivamente). Seguidamente, debemos cambiar el número de PRBs (Physical Resource Blocks, `n_prb`) a 25 para que el ancho de banda de nuestro filtro sea de 5dBs (es el número mínimo de prb que necesita el módem para poder funcionar correctamente ya que, si este número es menor, el módem no detecta la señal de la antena). En la sección de `[enb_files]`, simplemente debemos completar la ruta de los ficheros para evitar los warnings en la ejecución del sistema. Por último, cambiamos el valor del parámetro `dl_eafcn` (código EARFCN para DL) a 3275, para que nuestra antena transmita datos a una frecuencia de 2672.5MHz y reciba las señales a una frecuencia de 2552.5MHz, la ganancia de transmisión a 95dB, y el nombre del dispositivo a `bladeRF`. El resultado final fichero de configuración `enb.conf` se puede observar en las Figuras 32 y 33.

```
#####
# srsENB configuration file
#####

# eNB configuration
#
# enb_id:          20-bit eNB identifier.
# mcc:            Mobile Country Code
# mnc:            Mobile Network Code
# mme_addr:       IP address of MME for S1 connection
# gtp_bind_addr:  Local IP address to bind for GTP connection
# gtp_advertise_addr: IP address of eNB to advertise for DL GTP-U Traffic
# sic_bind_addr:  Local IP address to bind for S1AP connection
# sic_bind_port:  Source port for S1AP connection (0 means any)
# n_prb:          Number of Physical Resource Blocks (6,15,25,50,75,100)
# tm:             Transmission mode 1-4 (TM1 default)
# nof_ports:      Number of Tx ports (1 port default, set to 2 for TM2/3/4)
#
#####

[enb]
enb_id = 0x198
mcc = 901
mnc = 70
mme_addr = 127.0.1.100
gtp_bind_addr = 127.0.1.1
sic_bind_addr = 127.0.1.1
sic_bind_port = 0
n_prb = 25
tm = 4
nof_ports = 2
```

Figura 32: Configuración del fichero `enb.conf` (parte 1).

```
#####
# eNB configuration files
#
# srb_config: SIB1, SIB2 and SIB3 configuration file
# note: When enabling MMS, use the srb_conf.mmsf configuration file which includes SIB13
# rr_config: Radio Resources configuration file
# rb_config: SRS/DRB configuration file
#####

[enb_files]
srb_config = /root/.config/srsran/srb.conf
rr_config = /root/.config/srsran/rr.conf
rb_config = /root/.config/srsran/rb.conf

#####
# RF configuration
#
# dl_eafcn: EARFCN code for DL (only valid if a single cell is configured in rr.conf)
# tx_gain: Transmit gain (dB)
# rx_gain: Optional receive gain (dB). If disabled, AGC is enabled
#
# Optional parameters:
# dl_freq: Override DL frequency corresponding to dl_eafcn
# ul_freq: Override UL frequency corresponding to dl_eafcn (must be set if dl_freq is set)
# device_name: bladeRF
# device_args: Supported options: "auto" (uses first driver found), "uhd", "bladeRF", "soapy", "zmq" or "sideiq"
#               Arguments for the device driver. Options are "auto" or any string.
#               Default for UHD: "recv_frame_size=9232,send_frame_size=9232"
#               Default for bladeRF: ""
# time_adv_nsamples: Transmission time advance (in number of samples) to compensate for RF delay
#                   from antenna to timestamp insertion.
#                   Default "auto". 8210 USRP: 100 samples, bladeRF: 27
#####

[rf]
dl_eafcn = 3275
tx_gain = 95
rx_gain = 40

device_name = bladeRF
```

Figura 33: Configuración del fichero `enb.conf` (parte 2).

Paso 3. Conexión con el coche y configuración

Para poder establecer conexión con el coche y posteriormente configurarlo primero debemos desplegar la SDR que hemos configurado anteriormente. Antes de inicializar el EPC y el eNodeB, debemos llevar a cabo el enmascarado IP de todo el tráfico proveniente de la red a desplegar. ¿Por qué? Porque el vehículo hace uso de conexión a un servidor cloud para su correcto funcionamiento y, por lo tanto, necesita tener conexión a internet. El enmascarado IP debe hacerse siempre antes de inicializar los elementos de la red con el siguiente comando:

```
$ sudo srsepc_if_masq enp0s25
```

Una vez se realiza el enmascarado IP ya podemos desplegar el EPC y seguidamente el eNodeB. El EPC se despliega con el siguiente comando:

```
$ sudo srsepc
```

A continuación, desplegamos el eNodeB, que se conectará de manera automática con el EPC si la configuración es correcta, ejecutando:

```
$ sudo srsenb
```

Si se ha desplegado correctamente el resultado por pantalla del EPC y del eNodeB deberían ser como se muestran en las Figuras 35 y 34 respectivamente.

```
grupo00@grupo00:~$ sudo srsenb
Active RF plugins: librsran_rf_uhd.so librsran_rf_soapy.so librsran_rf_blade.so librsran_rf_zmq.so
Inactive RF plugins:
--- Software Radio Systems LTE eNodeB ---

Couldn't open , trying /root/.config/srsran/enb.conf
Reading configuration file /root/.config/srsran/enb.conf...
WARNING: cpu0 scaling governor is not set to performance mode. Realtime processing could be compromised. Consider setting it to performance mode before running the application.

Built in Release mode using commit ec29b0c1f on branch master.

/home/grupo00/Cache_autonomo/srsRAN_4G/srsenb/src/enb_cfg_parser.cc:1881: Force DL EARFCN for cell PCI=1 to 3275
Opening 1 channels in RF device=bladeRF with args=default
Supported RF device list: UHD soapy bladeRF zmq file
Opening bladeRF...
Set RX sampling rate 1.92 Mhz, filter BW: 1.92 Mhz

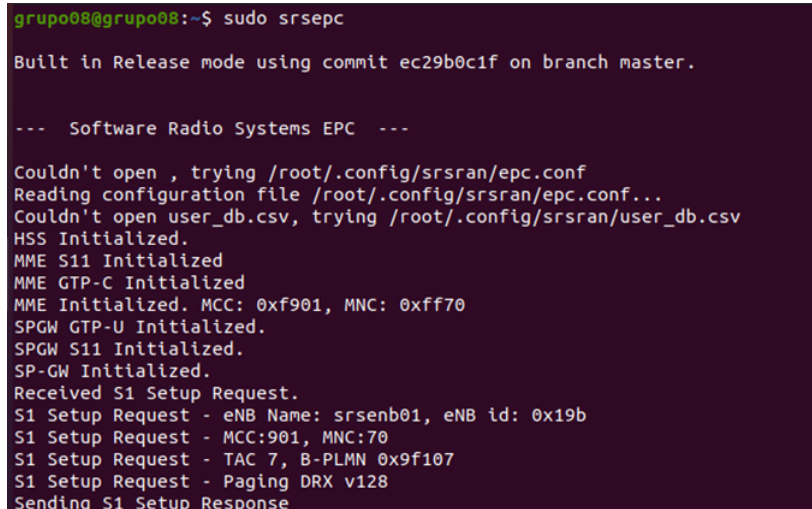
==== eNodeB started ====
Type <t> to view trace
Set RX sampling rate 11.52 Mhz, filter BW: 9.22 Mhz
Setting frequency: DL=2672.5 Mhz, UL=2552.5 Mhz for cc_idx=0 nof_prb=50
set TX frequency to 2672500000
set RX frequency to 2552500000
```

Figura 34: Funcionamiento eNodeB.

Por último, para que haya un suscriptor utilizando esa red hay que configurar el UE. En nuestro caso, ya hemos configurado anteriormente desde un ordenador el módem que va a utilizar la SIM para acceder a la red. Sin embargo, al conectar el modem al vehículo, se reconoce como un dispositivo de almacenamiento en vez de un dispositivo de conectividad. Por este motivo, cada vez que se conecte el módem al vehículo debemos ejecutar:

ANEXO I. DESARROLLO DEL DEMOSTRADOR

```
$ sudo usb_nodeswitch -v 12d1 -p 1f01 -J  
$ sudo systemctl restart ARTEMIS.service
```



```
grupo08@grupo08:~$ sudo srsepc  
Built in Release mode using commit ec29b0c1f on branch master.  
  
--- Software Radio Systems EPC ---  
  
Couldn't open , trying /root/.config/srsran/epc.conf  
Reading configuration file /root/.config/srsran/epc.conf...  
Couldn't open user_db.csv, trying /root/.config/srsran/user_db.csv  
HSS Initialized.  
MME S11 Initialized  
MME GTP-C Initialized  
MME Initialized. MCC: 0xf901, MNC: 0xff70  
SPGW GTP-U Initialized.  
SPGW S11 Initialized.  
SP-GW Initialized.  
Received S1 Setup Request.  
S1 Setup Request - eNB Name: srsenb01, eNB id: 0x19b  
S1 Setup Request - MCC:901, MNC:70  
S1 Setup Request - TAC 7, B-PLMN 0x9f107  
S1 Setup Request - Paging DRX v128  
Sending S1 Setup Response
```

Figura 35: Funcionamiento EPC.

Tras el despliegue correcto de la antena, nos ponemos inmediatamente a establecer la conexión entre el vehículo Amazon DeepRacer y la antena. Para llevarlo a cabo, es necesario instalar un servidor MQTT (Message Queuing Telemetry Transport), para tener en funcionamiento un servicio de control en la nube. MQTT es un protocolo de mensajería basado en suscripción diseñado para dispositivos con recursos limitados y redes de baja capacidad. Es ampliamente utilizado en el ámbito del IoT (Internet of Things) y en redes de comunicaciones para facilitar la comunicación entre dispositivos y sistemas. Para nuestro demostrador, el elegido ha sido el software mosquitto, ya que tiene una gran documentación disponible y es el más utilizado actualmente.

Para instalar mosquitto, simplemente debemos ejecutar los siguientes comandos desde la máquina donde queremos que corra el servidor mosquitto, en este caso el epc, instalando así todos los paquetes necesarios para su funcionamiento:

```
$ sudo apt-get update  
$ sudo apt-get install mosquitto
```

Una vez instalado deberemos ejecutar el mosquitto para que el servicio sea accesible desde la red. Para ello ejecutamos:

```
$ sudo systemctl start mosquitto
```

Una vez hemos configurado y puesto en marcha de manera exitosa el servidor MQTT, pasamos a la parte del cliente. En el vehículo hay que instalar el cliente MQTT y el software Artemis

ANEXO I. DESARROLLO DEL DEMOSTRADOR

(software que proporciona una plataforma de conectividad y gestión para dispositivos IoT, Internet of Things). Debido a que los vehículos fueron prestados, estos ya venían configurados con dichos softwares. Una vez comprobado, debemos indicar al cliente MQTT el puerto y la dirección IP del servidor MQTT al que debe conectarse para poderse comunicar con él, así como el servidor cloud. Por lo tanto, debemos realizar cambios en la configuración del cliente. Dicha configuración se encuentra dentro del directorio de ficheros *SoftARTEMIS/vehicle.conf*. La configuración de dicho fichero puede verse en la Figura 36. Una vez dichos cambios son guardados, debemos reiniciar el cliente, ejecutando para ello la siguiente instrucción:

```
$ sudo systemctl restart ARTEMIS.service
```

```
[mqtt]
vehicle_id = 3
mqtt_server_ip = 10.0.128.174
mqtt_server_port = 1883

[cloud_autonomous_driving]
cloud_server_ip = 10.0.128.174
cloud_server_port = 20003

[steering_calibration]
max = 1750000
mid = 1370000
min = 1230000
```

Figura 36: Fichero de configuración *vehicle.conf*.

Una vez tenemos el cliente configurado y conectado a la red, debemos esperar a que el cliente MQTT se conecte de manera automática (al ejecutarse el software Artemis) con el servidor MQTT. Cuando la luz del LED pase de color rojo a amarillo, significa que se ha establecido la conexión correctamente. Conectado el vehículo solamente falta la posibilidad de enviarle instrucciones desde nuestro epc. Se ha instalado la aplicación MQTT Explorer en el servidor para poder hacerlo.

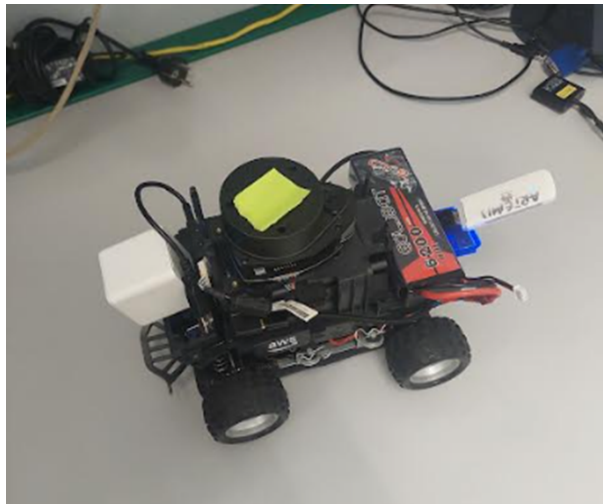


Figura 37: Vehículo del laboratorio conectado a la red.

ANEXO I. DESARROLLO DEL DEMOSTRADOR

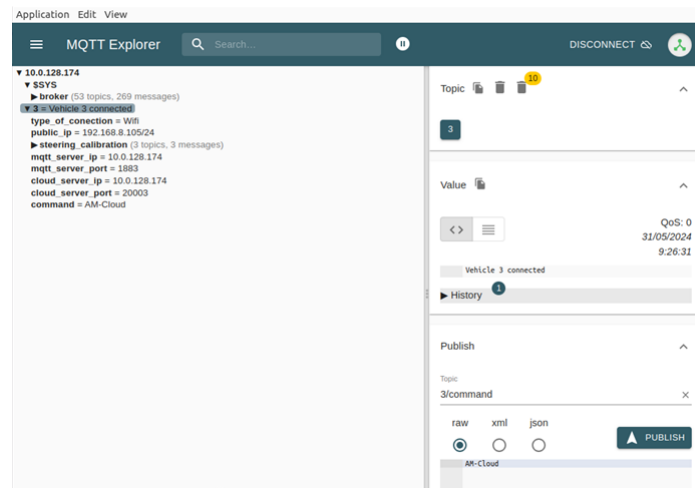


Figura 38: MQTT explorer.

Paso 4. Elección del algoritmo de IA y entrenamiento

A. Escoger algoritmo de identificación de señales

La detección de objetos es capaz de localizar varios objetos dentro de una imagen, mientras que la segmentación de imágenes permite comprender a nivel de píxel los elementos que se encuentran en la escena [31].

La detección de objetos es uno de los puntos fundamentales de la visión por ordenador y constituye la base de muchas otras tareas posteriores de ella. Su objetivo es identificar y localizar diversos objetos dentro de imágenes o videos, trazando una “caja delimitadora” (bounding box) alrededor de cada objeto, permitiendo determinar su posición y movimiento en la imagen. La detección de objetos se usa para el conteo o reconocimiento de personas mediante la detección de rostros, vehículos autónomos o detección de anomalías.

Las redes neuronales convolucionales (CNN) son la base de la detección de objetos, ya que pueden identificar una cantidad específica de objetos, si se entrenan mediante indicaciones explícitas sobre la clase de objeto y su posición en el plano bidimensional.

Hay que determinar como se quiere realizar la detección de objetos, ya que se puede hacer en una etapa o en dos. La detección de objetos en dos etapas, utiliza algoritmos para identificar las “bounding boxes” que podrían contener objetos y después, clasifica estas cajas según su tipo de límite. Por otro lado, la detección de objetos en una sola etapa, combina ambos pasos previamente descritos para lograr una detección eficiente y precisa, ya que está orientado a la detección de objetos en tiempo real.

Nuestro proyecto necesita la detección de objetos en tiempo real, ya que se trata de un vehículo autónomo, que debe reaccionar a las señales de tráfico que se le presenten en el menor tiempo posible, por eso nos centraremos en la detección de objetos en una sola etapa, más en

ANEXO I. DESARROLLO DEL DEMOSTRADOR

concreto, en la arquitectura de detección “YOLO”.

YOLO (You Only Look Once) es una red neuronal, que con simplemente una pasada a la red convolucional y, sin necesidad de iterar, consigue detectar todos los objetos que han sido etiquetados previamente con unas velocidades de detección muy elevadas (30 FPS). Para detectar los objetos, YOLO define una cuadrícula de tamaño fijo sobre la imagen, y en estas celdas predice todas las posibles “bounding boxes” que existan, calculando así el nivel de probabilidad de cada una de ellas.

B. Etiquetado de Imágenes

El software de etiquetado que hemos usado se llama “Roboflow“, ya que es compatible con YOLO y además ofrece herramientas para construir y desplegar modelos de visión por ordenador, facilitando la carga de imágenes, la compatibilidad con diversas bases de datos y la implementación rápida de modelos.

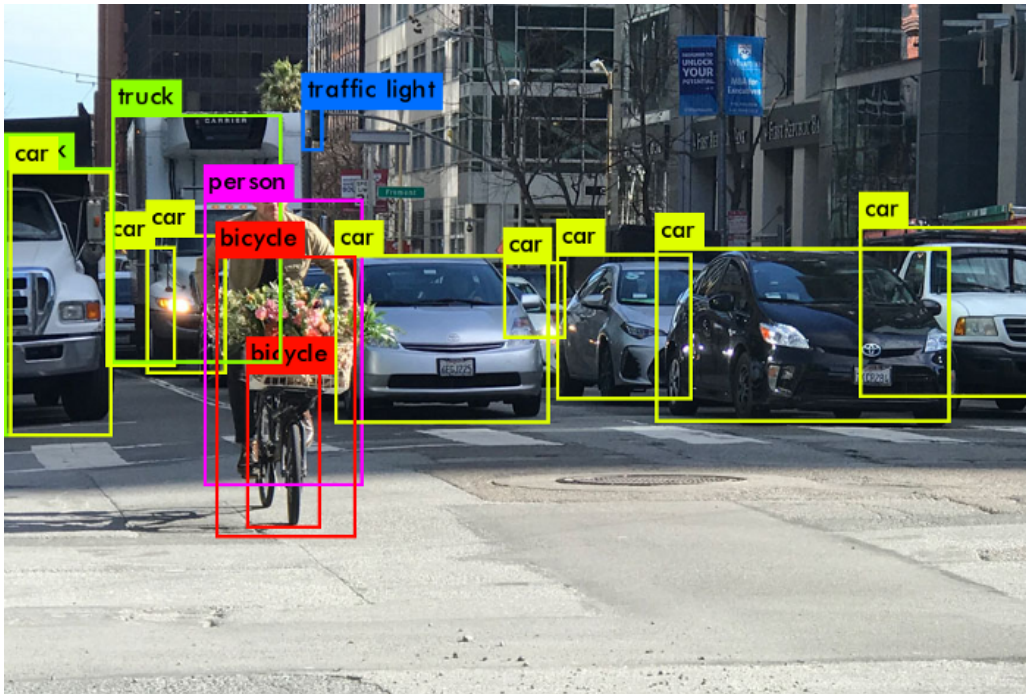


Figura 39: Detección de objetos con YOLO

C. Entrenamiento del modelo

Primero nos aseguramos de que tenemos acceso a la GPU, creamos una constante HOME para facilitarnos la gestión de conjuntos de datos, imágenes y modelos, usamos un fork del repositorio principal para la utilización del yolo versión 9, e instalamos el paquete roboflow, que utilizaremos para descargar nuestro conjunto de datos de Roboflow Universe [32].

ANEXO I. DESARROLLO DEL DEMOSTRADOR

```
!nvidia-smi

import os
HOME = os.getcwd()

!git clone https://github.com/SkalskiP/yolov9.git
%cd yolov9
!pip install -r requirements.txt -q
!pip install -q roboflow
```

Figura 40: 1ª Parte del Script

Descargamos los pesos de los distintos modelos de yolo versión 9, ya que todavía no están disponibles en el repositorio de YOLOv9 [32].

```
!wget -P {HOME}/weights -q https://github.com/WongKinYiu/yolov9/releases/download/v0.1/yolov9-c.pt
!wget -P {HOME}/weights -q https://github.com/WongKinYiu/yolov9/releases/download/v0.1/yolov9-e.pt
!wget -P {HOME}/weights -q https://github.com/WongKinYiu/yolov9/releases/download/v0.1/gelan-c.pt
!wget -P {HOME}/weights -q https://github.com/WongKinYiu/yolov9/releases/download/v0.1/gelan-e.pt

!ls -la {HOME}/weights
```

Figura 41: 2ª Parte del Script

Cargamos el conjunto de datos en formato YOLO disponibles en Roboflow Universe que hemos creado en Roboflow después de haber etiquetado todas las imágenes sacadas del coche [32].

```
[3] %cd {HOME}/yolov9
    from roboflow import Roboflow

    from roboflow import Roboflow
    rf = Roboflow(api_key="jRiy2jGK0gFa8DDeRazN")
    project = rf.workspace("tp-ii").project("coche-conectado-senales_juguete")
    version = project.version(2)
    dataset = version.download("yolov9")
```

Figura 42: 3ª Parte del Script

Entrenamiento del modelo pre-entrenado con los pesos descargados y el dataset cargado de Roboflow [32].

```
%cd {HOME}/yolov9

!python train.py \
--batch 16 --epochs 20 --img 640 --device 0 --min-items 0 --close-mosaic 15 \
--data {dataset.location}/data.yaml \
--weights {HOME}/weights/gelan-c.pt \
--cfg models/detect/gelan-c.yaml \
--hyp hyp.scratch-high.yaml
```

Figura 43: 4ª Parte del Script

Montamos nuestra unidad de Drive en nuestro Google Colab, para poder exportar el modelo entrenado y usarlo en nuestro proyecto para la detección de señales de tráfico [32].

```
[5] from google.colab import drive
drive.mount('/content/drive')
```

Figura 44: 5ª Parte del Script

D. Resultados del entrenamiento

La matriz de confusión es una herramienta útil para analizar el rendimiento de los algoritmos de aprendizaje automático, ya que compara las predicciones del modelo con los valores verdaderos (etiquetas reales). Las filas representan las etiquetas verdaderas y las columnas representan las etiquetas predichas.

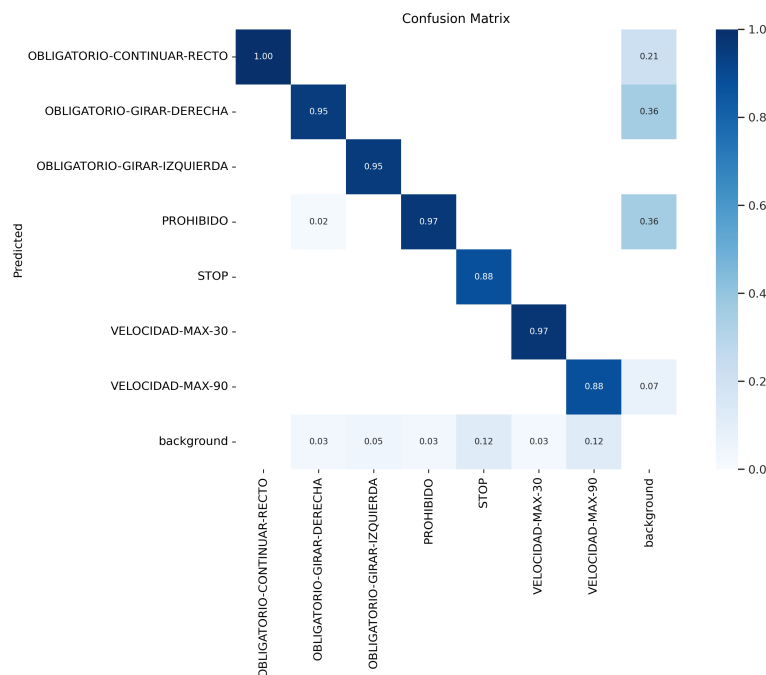


Figura 45: Matriz de confusión

Nuestra matriz de confusión muestra que el modelo tiene un alto rendimiento en la mayoría de las clases, con precisiones cercanas al 95-100 %, aunque se puede apreciar el principal fallo que puede tener nuestro modelo en un futuro.

Existen algunos casos de predicciones incorrectas dispersas, especialmente para el fondo, lo que indica que el modelo podría estar teniendo dificultades para diferenciar ciertas señales en algunas condiciones específicas. La solución sería incluir más ejemplos de señales que actualmente se confunden con el fondo en la base de datos para mejorar la capacidad del modelo de diferenciarlas.

E. DAFO Yolov9

Fortalezas

Información de Gradiente Programable (PGI): Esta característica permite a YOLOv9 preservar detalles cruciales a través del proceso de detección, lo que mejora la capacidad del modelo para aprender tanto de objetos grandes como pequeños. Yolov9 implementa el PGI para garantizar una generación de gradiente más fiable y, en consecuencia, una mejor convergencia y rendimiento del modelo, solucionando uno de los grandes retos de esta tecnología, que es el cuello de botella.

Precisión y eficiencia: YOLOv9 es conocido por su equilibrio entre precisión y demandas computacionales, lo que lo hace ideal para aplicaciones en tiempo real como la detección de señales de tráfico.

Arquitectura GELAN: La Red de Agregación de Capas Eficientes Generalizadas (GELAN) ayuda a YOLOv9 a mantener una alta eficiencia y precisión, lo que es esencial para la detección precisa de señales en diferentes condiciones de iluminación y desde varios ángulos.

Funciones Reversibles: Esta propiedad es crucial para las arquitecturas de aprendizaje profundo, ya que permite a la red conservar un flujo de información completo, posibilitando así actualizaciones más precisas de los parámetros del modelo. YOLOv9 incorpora funciones reversibles dentro de su arquitectura para mitigar el riesgo de degradación de la información, especialmente en las capas más profundas, garantizando la conservación de datos críticos para las tareas de detección de objetos.

Debilidades

Complejidad de implementación: A pesar de sus ventajas, YOLOv9 puede ser más complejo de implementar y afinar en comparación con modelos anteriores debido a sus características avanzadas, lo que le convierte en el más “pesado” y más lento de procesar. Esto puede ser un problema grave si no se tiene un ordenador adecuado, ya que se reflejará en una gran lentitud en la ejecución del modelo YOLOv9.

Requisitos de hardware: Para aprovechar plenamente las capacidades de YOLOv9, se pueden requerir recursos de hardware más potentes, lo que podría ser un desafío para algunos proyectos con limitaciones de presupuesto.

Oportunidades

Evolución: YOLOv9 es parte de una serie de modelos en constante evolución, lo que significa que se beneficiará de mejoras y optimizaciones continuas a medida que la comunidad de investigación avance.

Adaptabilidad: La capacidad de YOLOv9 para ajustarse a diferentes entornos y condiciones lo hace muy adecuado para el dinámico entorno de una autopista, donde las condiciones pueden cambiar rápidamente.

Amenazas

Modelos competidores: Siempre existe la posibilidad de que surjan nuevos modelos que puedan superar a YOLOv9 en términos de rendimiento o eficiencia y, tarde o temprano se quedará atrás con respecto otras redes neuronales u otras versiones de YOLO.

Leyes de circulación: Los cambios en las regulaciones de tráfico o en las normativas de vehículos autónomos podrían requerir adaptaciones en el modelo que podrían no ser inmediatamente factibles con YOLOv9.

En resumen, YOLOv9 fue elegido para nuestro proyecto debido a su destacada precisión y eficiencia en la detección de objetos en tiempo real, su capacidad para manejar información detallada a través de PGI y GELAN, y su potencial para adaptarse y mejorar con el desarrollo continuo. Estas características lo hacen particularmente adecuado para la detección de señales de tráfico en un entorno de autopista, donde la rapidez y la precisión son críticas. Sin embargo, es importante tener en cuenta las posibles debilidades y amenazas, como los requisitos de hardware, la complejidad de implementación y los posibles cambios en el panorama competitivo o regulatorio.

Paso 5. Implementación del algoritmo de IA en la red 4G

Lo primero que hacemos es importar todas las bibliotecas necesarias para trabajar con sockets, serialización, procesamiento de imágenes, control del coche autónomo y ejecución de comandos del sistema.

```
import socket
import struct
import pickle
import cv2
import artemis_autonomous_car
import time
import os
import subprocess
import time
import select
```

Figura 46: Importaciones y configuración inicial

Configuramos el servidor, estableciendo su dirección IP y el puerto por el que nos vamos a co-

ANEXO I. DESARROLLO DEL DEMOSTRADOR

municar, y creamos las funciones auxiliares que vamos a utilizar más adelante. `Send_control()` envía los comandos de control al coche empaquetando los datos de giro y aceleración. `Clean_buffer()` limpia el buffer del socket para asegurarnos de que se procesa la imagen más reciente.

```
server_address = ('10.0.128.174', 20003)
show_info=True

def send_control(control_giro, control_acelerador, address):
    sock.sendto(struct.pack('c', bytes('c', 'ascii'))+struct.pack('d', round(control_giro,3))+struct.pack('d', round(control_acelerador,3)), address)

def clean_buffer(sock, timeout=0.1):
    while True:
        try:
            data, addr = sock.recvfrom(99999)
        except BlockingIOError:
            break
```

Figura 47: Servidor y funciones auxiliares

En esta sección de código, se inicializan las variables, se configura el socket en modo no bloqueante, se crea una carpeta para almacenar las imágenes capturadas y se configura la ventana de OpenCV para mostrar las imágenes procesadas.

```
if __name__ == "__main__":
    cuenta = 0
    received_payload=b''
    auto_utils=artemis_autonomous_car.artemis_autonomous_car([2,3,2,3,2,2,2,2,0],0)#[2,3,1,3,2,2,0][2,3,2,2,2,2,0]
    #Generación de socket
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.bind(server_address)
    sock.setblocking(0)
    folder_path = 'imagenes_capturadas'
    cv2.namedWindow("Video", cv2.WINDOW_NORMAL)
    cv2.resizeWindow("Video", 640, 480)
```

Figura 48: Bloque principal del código

En el bucle principal del código, lo primero que se hace es limpiar el buffer del socket para eliminar todas las imágenes que se han ido acumulando durante la ejecución del modelo sobre una imagen y, luego intenta recibir datos para que la siguiente imagen que se procese sea la más reciente.

Si los datos recibidos son una imagen (`data_type == b'I'`), se decodifica la imagen, se guarda en el ordenador de manera ordenada y se procesa utilizando el modelo YOLOv9 (“python3 ficheros_IA/yolov9/detect.py –weights ficheros_IA/runs/train/exp/weights/best.pt –source imagenes_capturadas/image_name”). Guardamos la imagen ya procesada y mostramos los resultados en una ventana de OpenCV, para que se pueda ir viendo a tiempo real las imágenes grabadas por el coche y las mismas ya procesadas.

Podemos recibir otros tipos de datos del coche (`data_type == b'D'`, `data_type == b'L'`, `data_type == b'B'`), pero como solo nos interesan las imágenes, lo que hacemos es ignorarlos.

En conclusión, este código permite recibir imágenes de un coche autónomo a través de un socket UDP, procesarlas utilizando un modelo YOLOv9 para detección de objetos, y mostrar los resultados en tiempo real.

ANEXO I. DESARROLLO DEL DEMOSTRADOR

```
while True:
    clean_buffer(sock)
    data = None
    while data is None:
        try:
            data, address = sock.recvfrom(99999)
        except BlockingIOError:
            time.sleep(0.1)
    data_type = struct.unpack('c', bytes([data[0]]))[0]
    received_payload = bytes(data[1:])
    data = pickle.loads(received_payload, encoding='latin1')
    if data_type == b'I':
        img = cv2.imdecode(data, 1)
        control_giro, control_acelerador, trayectoria_not_found = auto_utils.proceso_fotograma(img, show_info)
        send_control(control_giro, control_acelerador, address)
        image_name = f"imagen_{cuenta}.png"
        cv2.imwrite(os.path.join(folder_path, image_name), img)
        command = 'python3 ficheros_IA/yolov9/detect.py --weights ficheros_IA/runs/train/exp/weights/best.pt --source imagenes_capturadas/' + str(image_name)
        subprocess.run(command, shell=True)
        cuenta += 1
    if cuenta == 1:
        ruta_name = "exp"
    else:
        ruta_name = f"exp_{cuenta}"
    ruta_imagen = "ficheros_IA/yolov9/runs/detect/" + str(ruta_name) + "/" + str(image_name)
    imagen_detectada = cv2.imread(ruta_imagen)
    cv2.imshow("Video", imagen_detectada)
    if data_type == b'D':
        pass
    if data_type == b'L':
        auto_utils.proceso_lidar(data, False)
    if data_type == b'B':
        auto_utils.set_battery_level(data)
```

Figura 49: Bucle principal de procesamiento

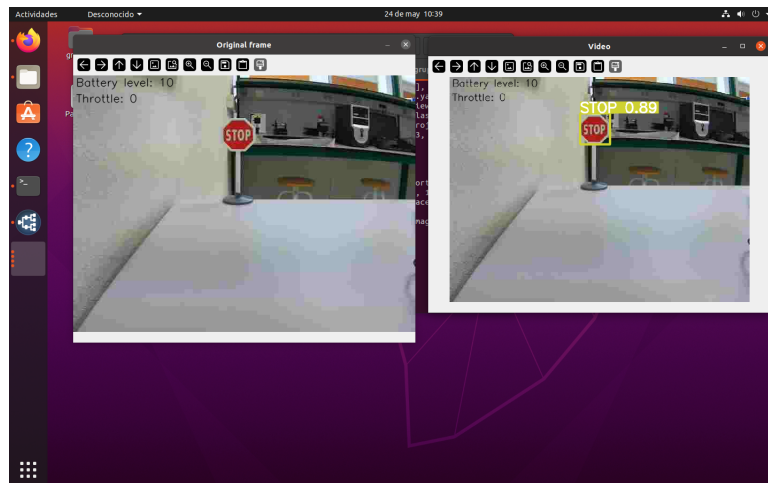


Figura 50: Resultado de la detección de señales

Paso 6. Funcionamiento del demostrador

Para hacer funcionar el demostrador, y que el coche reaccione a las señales detectadas hemos realizado modificaciones en el archivo *detect.py* que se usa para la ejecución del modelo, y en el script donde lanzamos el comando para la detección de las señales.

A. Archivo *detect.py*

Primero definimos una variable llamada “json_only” y la inicializamos con el valor False, para que por defecto esta opción esté desactivada, pero exista la opción de mostrar la salida en formato JSON. Después, agregamos un nuevo argumento “--json-only” de línea de comandos

ANEXO I. DESARROLLO DEL DEMOSTRADOR

al parser que el script aceptará. Si se especifica “--json-only” en la línea de comandos, el valor de este argumento será True, y simplemente su presencia activa el comportamiento. A este argumento le añadimos un mensaje de ayuda que describe su comportamiento, para mostrárselo al usuario en caso de que ejecute el script con el argumento “--help”.

```
json_only=False, #only print JSON output  
...  
parser.add_argument('--json-only', action='store_true', help='only print JSON output')
```

Figura 51: Resultados en formato json

Después, añadimos el fragmento de código encargado de procesar y almacenar los resultados de detecciones realizadas por el modelo de clasificación de señales de tráfico, iterando a través de una lista de detecciones “det” en orden inverso. Para cada detección, extraemos las coordenadas de la caja delimitadora “xyxy”, la confianza “conf” y la clase “cls”, y se crea un diccionario “detection” que contiene el id de la imagen (“image_id”), el tipo de señal de tráfico detectada (“class”), la confianza de la detección (“confidence”) y las coordenadas de la caja delimitadora (“bbox”). Este código es útil para estructurar los resultados de las detecciones en formato json de manera que sean fáciles de manejar y analizar posteriormente.

```
# Write results  
for *xyxy, conf, cls in reversed(det):  
    ##  
    detection = {  
        'image_id': p.stem,  
        'class': names[int(cls)], #names[c],  
        'confidence': float(conf), #conf.item(),  
        'bbox': [float(x) for x in xyxy] #float(coord) for coord in xyxy  
    }  
    all_detections.append(detection)
```

Figura 52: Escribir resultados de la detección

Por último, añadimos lo necesario para que el script actúe de la forma que queremos si se añade al comando la opción “--json-only”. Creamos una tupla “t” que contiene las velocidades por imagen procesada, y comprobamos si la variable “json_only” es True, para decidir si los resultados deben imprimirse solo en formato JSON. En el caso de que queramos imprimir los resultados solo en formato JSON, la lista de detecciones “all_detections” se convierte a una cadena JSON y se imprime.

```
# Print results  
t = tuple(x.t / seen * 1E3 for x in dt) # speeds per image  
if json_only:  
    print(json.dumps(all_detections))
```

Figura 53: Imprimir resultados de la detección

B. Mandar órdenes al coche

Primero definimos la función “send_control” para enviar comandos de control al vehículo a través de un socket, empaquetando un carácter “C” como un byte en formato ASCII, el giro del coche “control_giro” como un double y su aceleración “control_acelerador” como otro double, especificando la dirección “address” a la que se enviará la instrucción.

```
def send_control(control_giro, control_acelerador, address):
    sock.sendto(struct.pack('c', bytes('C', 'ascii'))+struct.pack('d', round(control_giro, 3))+struct.pack('d', round(control_acelerador, 3)), address)
```

Figura 54: Función para enviar los controles al coche

Después, añadimos el código que se encarga de ejecutar el script de detección de objetos de YOLOv9, procesar los resultados de detección y luego imprimir las señales de tráfico detectadas. Creamos y ejecutamos el comando de detección, y su salida, que está en formato JSON la convertimos en un objeto de Python, en este caso, una lista de detecciones. Por último, iteramos sobre cada detección en la lista de detecciones y extraemos la clase del objeto detectado de la detección actual y la imprimimos por pantalla.

```
command = 'python3 ficheros_IA/yolov9/detect.py --weights ficheros_IA/runs/train/exp/weights/best.pt --source imagenes_capturadas/' + str(image_name) + ' --json-only'
result = subprocess.run(command, shell=True, capture_output=True, text=True)
detections = json.loads(result.stdout)
for detection in detections:
    detected_class = detection['class']
    print(f"Clase detectada: {detected_class}")
```

Figura 55: Recogemos la señal detectada

Por último, añadimos el fragmento de código de lógica de control que ajusta los valores de “control_acelerador” y “control_giro” basándose en la clase del objeto detectado (“detected_class”). Cada clase de objeto representa una señal de tráfico específica, que dicta cómo debe comportarse el vehículo. Luego, enviamos estos valores a través de la función “send_control” para controlar el vehículo con dirección (“address”).

```
if detected_class=="VELOCIDAD-MAX-30":
    control_acelerador = 0.05
elif detected_class=="VELOCIDAD-MAX-90":
    control_acelerador = 0.25
elif detected_class=="OBLIGATORIO-CONTINUAR-RECTO":
    control_acelerador = 0.15
elif detected_class=="OBLIGATORIO-GIRAR-DERECHA":
    control_giro = -1
    control_acelerador = 0.15
elif detected_class=="OBLIGATORIO-GIRAR-IZQUIERDA":
    control_giro = 1
    control_acelerador = 0.15
elif detected_class=="PROHIBIDO":
    control_acelerador = 0
elif detected_class=="STOP":
    control_acelerador=1
elif detected_class == "no-detection":
    control_acelerador=0.15

cv2.waitKey(1)
send_control(control_giro, control_acelerador, address)
```

Figura 56: Establecemos los controles y lo enviamos al coche

Anexo II

Simulador Xirio-Online

En la sección 3.1 hemos indicado que para llevar a cabo la planificación y las simulaciones de la red 4G en el tramo de autovía A-62 vamos a emplear la herramienta Xirio-Online. Xirio ofrece una gran variedad de herramientas para diseñar y optimizar redes de radio, incluyendo la posibilidad de realizar simulaciones de cobertura, interferencias y rendimiento de antenas y estaciones base. Además, gracias a su cartografía de alta resolución, podemos tener en cuenta los factores relacionados con el terreno de manera integrada.

Crear nuevo estudio

Seleccione un tipo de estudio

 Enlace

 **Cobertura**

 Cobertura de interior

 Cobertura multitransmisor

 Red de transporte

Estudio de cobertura:

Este estudio representa valores de la señal impuesta por un transmisor, en términos de campo eléctrico o potencia, en todos los puntos dentro del área seleccionada por el usuario.

[Leer más](#)

Seleccione un servicio o tecnología

Categoría:

Servicio Móvil

Subcategoría:

LTE-A

Servicio:

LTE-A

Figura 57: Creación de una nueva cobertura individual.

Lo primero de todo es definir un estudio de cobertura individual. Un estudio en Xirio contiene todos los datos necesarios para llevar a cabo las simulaciones dentro de nuestro proyecto. Por lo tanto, si queremos llevar a cabo simulaciones de coberturas individuales de las antenas que vamos a desplegar en todo el tramo de autovía, debemos crear un nuevo estudio de cobertura seleccionando el tipo de servicio, telefonía móvil en nuestro caso, y la tecnología, LTE-A.

Cada estudio tiene las siguientes opciones de configuración:

- **Estudio de Cobertura:** Se indica el nombre de la cobertura a elegir, el servicio que se

ofrece (elegido en el paso anterior) y una banda de frecuencias a utilizar en dicho estudio.

- **Extremos:** Límites o puntos finales de la evaluación de la cobertura de la red. Se debe definir un sector el cual consiste en una subdivisión de la cobertura de una estación base.
- **Parámetros de cálculo:** Se definen las variables y configuraciones utilizadas para simular y evaluar la cobertura de la red.
- **Área de cálculo:** Superficie de evaluación de la cobertura.
- **Rangos:** Muestras de colores en función de la potencia de la señal recibida de la antena.

Propiedades del estudio de Cobertura

Estudio

Nombre: Cobertura 1

Grupo: ▼

Servicio: LTE-A

Banda: Banda de Frecuencias PTII ↑

Descripción: Estudio de cobertura...

Fecha de última puesta en servicio/apagado: 31

Estado:

Extremos

Sector: + ?

Parámetros del terminal: + ?

Parámetros de cálculo

Área del cálculo

Rangos

Figura 58: Menú de configuración de un nuevo estudio de cobertura individual.

A. Banda de frecuencias

Las bandas de frecuencia de un estudio de cobertura en Xirio pueden ser personalizables para cada uno. En nuestro caso y tal como nos marca el enunciado del proyecto, tenemos dos rangos de frecuencias de un ancho de banda de 10MHz cada uno, un rango para las comunicaciones entre las antenas y los UE (tramo superior) y otro para la comunicación entre los UEs y las antenas (tramo inferior). En el menú de Xirio, existe la posibilidad de dividir el rango de frecuencias disponibles en canales con el mismo ancho de banda. En nuestro caso, hemos cambiado dicho parámetro para posteriormente comparar los resultados de las coberturas en función del ancho de banda de cada canal en el que se subdivide el total. Hemos decidido esto para averiguar cuál es la mejor situación para este caso en específico. Los rangos de frecuencia que tenemos disponibles para utilizar y cuyos permisos hemos solicitado son, para el tramo inferior de 2550MHz a 2560MHz, y para el superior de 2660MHz a 2670MHz.

Propiedades de la Banda de Frecuencias

Banda ★

Nombre:

Descripción 1:

Descripción 2:

☐ **Color:**

Parámetros de la banda

Ancho de canal / Separación entre portadoras: MHz ▼

Ordinal del primer canal:

Tramo inferior:

Frecuencia inicial: MHz ▼

Frecuencia final: MHz ▼

Frecuencia primera portadora: MHz ▼

☒ **Tramo superior:**

Frecuencia inicial: MHz ▼

Frecuencia final: MHz ▼

Frecuencia primera portadora: MHz ▼

Canales prohibidos:

Canales prioritarios:

☐ Introduzca una lista de canales separados por comas y/o intervalos de canales (Ejemplo: 2, 2', 5-7, 12'-21').

Figura 59: Menú de configuración de la banda de frecuencias.

B. Extremos: Definición de los sectores

El sector contiene la configuración fundamental para la simulación de la cobertura. Lo primero que debemos hacer, es saber dónde situar los emplazamientos (estaciones base) en el mapa geográfico alrededor del tramo de autovía, indicando sus coordenadas geográficas. En nuestro caso, hemos decidido situar las estaciones base a una distancia de aproximadamente 3km para poder brindar una mayor calidad de servicio a nuestros usuarios, ya que cada uno necesita una tasa de transmisión de vídeo mínima de aproximadamente 5,44Mbps.

Una vez ya hemos situado el centro del sector (en los emplazamientos oportunos), debemos seleccionar la antena que vamos a utilizar en nuestra red real. Debido a nuestras necesidades de calidad de servicio, se barajaron diferentes tipos de antenas que podrían ser óptimas. En el mercado, una de las marcas más comerciales dedicadas al diseño y a la fabricación de antenas 4G LTE es Ubinquiti. Por lo tanto, barajamos todos los modelos de antenas que ofrecen en base a las dimensiones de las antenas, la ganancia, la polarización, la resistencia al viento... Dicha

ANEXO II. SIMULAROR XIRIO-ONLINE

comparativa puede verse en la Figura 61. Finalmente, nos decidimos por el modelo de antena AM-2G16-90.

Propiedades del sector

Sector

Nombre:

Emplazamiento

Emplazamiento:

Coordenadas

Latitud:

Longitud:

Parámetros de radio

Parámetros LTE

Figura 60: Menú de configuración de un sector.

	Antenna Characteristics			
Model	AM-9M13	AM-2G15-120	AM-2G16-90	AM-3G18-120
Dimensions* (mm)	1290 x 290 x 134	700 x 145 x 93	700 x 145 x 79	735 x 144 x 78
Weight**	12.5 kg	4.0 kg	3.9 kg	5.9 kg
Frequency Range	902 - 928 MHz	2.3 - 2.7 GHz	2.3 - 2.7 GHz	3.3 - 3.8 GHz
Gain	13.2 - 13.8 dBi	15.0 - 16.0 dBi	16.0 - 17.0 dBi	17.3 - 18.2 dBi
HPOL Beamwidth	109° (6 dB)	123° (6 dB)	91° (6 dB)	118° (6 dB)
VPOL Beamwidth	120° (6 dB)	118° (6 dB)	90° (6 dB)	121° (6 dB)
Electrical Beamwidth	15°	9°	9°	6°
Electrical Downtilt	N/A	4°	4°	3°
Max. VSWR	1.5:1	1.5:1	1.5:1	1.5:1
Wind Survivability	125 mph	125 mph	125 mph	125 mph
Wind Loading	95 lbf @ 100 mph	24 lbf @ 100 mph	19 lbf @ 100 mph	21 lbf @ 100 mph
Polarization	Dual-Linear	Dual-Linear	Dual-Linear	Dual-Linear
Cross-pol Isolation	30 dB Min.	28 dB Min.	28 dB Min.	28 dB Min.
ETSI Specification	N/A	EN 302 326 DN2	EN 302 326 DN2	EN 302 326 DN2
Mounting	Universal Pole Mount, RocketM Bracket, and Weatherproof RF Jumpers Included			

Figura 61: Tabla de especificaciones de los modelos de antenas Ubunquiti [33].

Para incluir dicha antena en nuestro sector primero se debe de crear, rellenando para ello todos los campos que la aplicación Xirio-Online necesita en base a sus especificaciones [34]. Todos los datos incluidos pueden verse en la Figura 62.

Definido el modelo de antena a utilizar, decidimos la altura a la que la vamos a situarla (20 metros en nuestro caso), su orientación respecto al Azimut (distinta en cada caso dependiendo de su zona geográfica) y tanto la inclinación mecánica como eléctrica. Por último, indicamos las frecuencias de transmisión y su polarización. El resto de la configuración la dejaremos por defecto ya que no necesitamos precisar mucho las simulaciones para el estudio que estamos llevando a cabo. Para guardar los cambios de la configuración del sector hay que pulsar *Aceptar*.

ANEXO II. SIMULAROR XIRIO-ONLINE

Propiedades de la Antena

Antena ★

Nombre: Ubunquiti AM-2G16-90

Tipo de antena: Estándar

Polaridad: ☒ Simple ☐ Doble

Peso: 3.9 Kg

Dimensión mayor: 0.7 m

Diagramas de radiación

	Frec. ini.	Frec. fin.	Tilt eléc.	Tipo
	2552.50 MHz	2672.50 MHz	4.00 °	Vertical

1 elementos (0 seleccionados)

Figura 62: Menú de configuración de la antena en Xirio-Online.

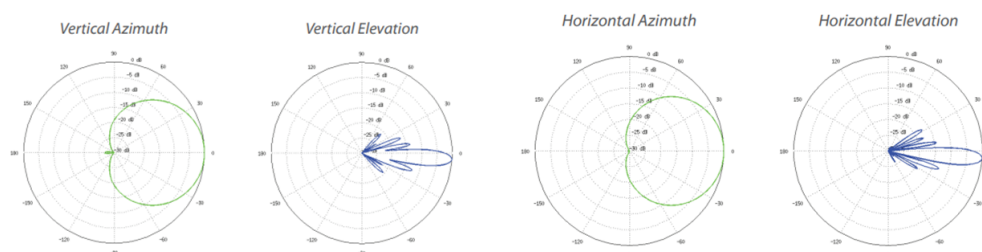


Figura 63: Azimut vertical y horizontal de la antena.

C. Extremos: Configuración del terminal

Para que funcione correctamente la simulación, aparte de configurar los terminales que actúan como elementos transmisores debemos configurar los dispositivos que actuarán como equipos de usuario que van a hacer uso de la red. Para ello, en la parte inmediatamente inferior a la configuración de los sectores encontramos la opción *Modificar parámetros del terminal*.

En la ventana de configuración del terminal, indicamos todos los elementos y propiedades que tendrán típicamente los UE. Para ello, hemos elegido una antena omnidireccional estándar de 2dBi de ganancia situada a 1,5 metros de altura (altura promedia de los turistas). Además, hemos configurado la frecuencia de transmisión (para utilizar la banda que va desde los 2550MHz a 2560MHz), la potencia de transmisión (típicamente alrededor de 23dBm), el factor de ruido a 7 y la sensibilidad de la señal RSRP a un valor de -92dBm (ver Figura 22).

Propiedades de parámetros de recepción del terminal

Propiedades de parámetros de recepción del terminal

Antena: 3G/4G/5G 2 dBi Omni

Altura antena: 1.5 m

Frecuencias de transmisión:

Frecuencias	Canal
2551.000 MHz	1

Polarización: Vertical

Feeder:

Longitud del feeder: 0 m

Pérdidas del feeder: 0.00 dB

Pérdidas pasivos: 3 dB

Potencia de transmisión: 23 dBm

Parámetros LTE

Figura 64: Menú de configuración del terminal.

D. Parámetros de cálculo

Los parámetros de cálculo son configuraciones y valores que determinan cómo se realiza el análisis y la simulación de la propagación de la señal. Xirio pone a nuestra disposición varios métodos de cálculo entre ellos los métodos Met526-15 y Met526-11, ambos empleados en todos los servicios radioeléctricos en entornos rurales y mixtos siempre que se disponga de cartografía de media o alta resolución, emplean el método determinístico basado en difracción y son válidos para frecuencias mayores de 30 MHz. Nosotros hemos empleado ambos métodos de cálculo para nuestras coberturas y así compararlas.

E. Área de cálculo y rangos de colores

El área de cálculo es aquella área en la que el simulador va a calcular las coberturas correspondientes. Para una mayor claridad en el resultado, hemos utilizado la herramienta que nos ofrece Xirio para ajustar automáticamente el área de cálculo. Debido al conocimiento que tenemos sobre las propiedades de las señales de banda 7, hemos configurado un máximo de 2km en dirección de la antena, 1k a los laterales y 1km hacia atrás.

Color	Rango	Descripción		
	[-105.00 , -96.00) dBm	Regular		
	[-96.00 , -88.00) dBm	Buena		
	[-88.00 , Infinity) dBm	Excelente		

Figura 65: Sistema de colores de las coberturas.

Por otro lado, Xirio nos permite elegir el color y los umbrales de las potencias de las señales

ANEXO II. SIMULAROR XIRIO-ONLINE

para que estas sean visibles en el mapa. Para una mayor calidad de análisis, hemos decidido escoger tres colores acordes a tres valores de señal (Excelente, Buena y deficiente).

F. Estudio multitransmisor: Simulación

En esta sección se recogen todos los resultados de las simulaciones llevadas a cabo con el objetivo de justificar todo el despliegue que se va a llevar a cabo.

Figura 66: Menú de configuración del estudio de cobertura multitrasmisor.

Una vez se han configurado todos los estudios de coberturas individuales una a una, calculando para ello el ángulo y la altura a la que debemos situar cada antena (Tabla 5), debemos crear un estudio de cobertura multitransmisor para poder simular el comportamiento que tienen tanto individual, como en conjunto.

En la creación de dicho estudio indicaremos el nombre del estudio y el servicio a desplegar (LTE-A). A continuación, en la sección *Coberturas de red* debemos incluir todas los estudios anteriormente configurados mediante la opción *Añade coberturas existentes*, después *Buscar* y seleccionamos todas aquellas que deseemos. Por último, definimos el área de cobertura, el cual abarca todo el área que ocupan los estudios individuales y realizamos el cálculo gratuito de baja resolución (gratuito).

El resultado de la simulación se muestra tres maneras distintas: señal RSRP, solapamiento y mejor servidor.

- Las señales RSRP sirven para evaluar la calidad de la señal y el rendimiento de la red, siendo en verde una calidad excelente, naranja una calidad buena y azul una calidad deficiente (Ver figura 65).
- Las señales de solapamiento se refieren a las áreas donde las señales de múltiples estaciones base se superponen. Sirven para identificar y optimizar las zonas de handover entre celdas, identificación y reducción de interferencias y mejora de la calidad de servicio (QoS).

ANEXO II. SIMULAROR XIRIO-ONLINE

Estación ID	Azimut - antena 1	Azimut - antena 2	Ubicación
01	221,3°	-	Km - 158
02	53,1°	217,4°	Km - 161
03	46,8°	212,5°	Km - 164
04	40,7°	234,6°	Km - 167
05	33,2°	195,1°	Km - 170
06	40,3°	204,7°	Km - 173
07	33,5°	192,6°	Km - 177
08	18,2°	254,2°	Km - 181
09	25,9°	210,1°	Km - 184
10	62,2°	233,4°	Km - 187
11	50,3°	213,8°	Km - 190
12	57,0°	200,5°	Km - 192
13	47,0°	194,3°	Km - 195
14	16,2°	-	Km - 198
15	36,7°	230,2°	Km - 201
16	20,0°	264,9°	Km - 205
17	72,4°	269,0°	Km - 207
18	42,4°	242,1°	Km - 210
19	44,8°	210,0°	Km - 214
20	356,6°	213,4°	Km - 218
21	33,5°	219,0°	Km - 220
22	69,5°	180,0°	Km - 225
23	48,9°	200,0°	Km - 227
24	46,1°	202,5°	Km - 229
25	73,0°	-	Km - 233

Tabla 5: Azimut estaciones base.

- Las señales de mejor servidor juegan un papel fundamental para identificar cuál es la mejor antena para servir a un usuario en una ubicación determinada dentro de nuestra área de cálculo. Nos permiten determinar el área de cobertura óptima que cada antena brinda y gestionar el tráfico y capacidad de la red.

G. Extra: Análisis de los UE

Analizada la cobertura de la red de acceso, vamos a proceder con el análisis y justificación de los UE cuando actúan como transmisores. Para ello, vamos a crear varios estudios de cobertura nueva de servicio móvil y tecnología LTE-A. El objetivo consiste en situar cada sector de cada estudio de cobertura en una ubicación diferente cada uno pero a lo largo de la carretera, para poder saber cuál es el rango de la señal RSRP de cada UE.

ANEXO II. SIMULAROR XIRIO-ONLINE

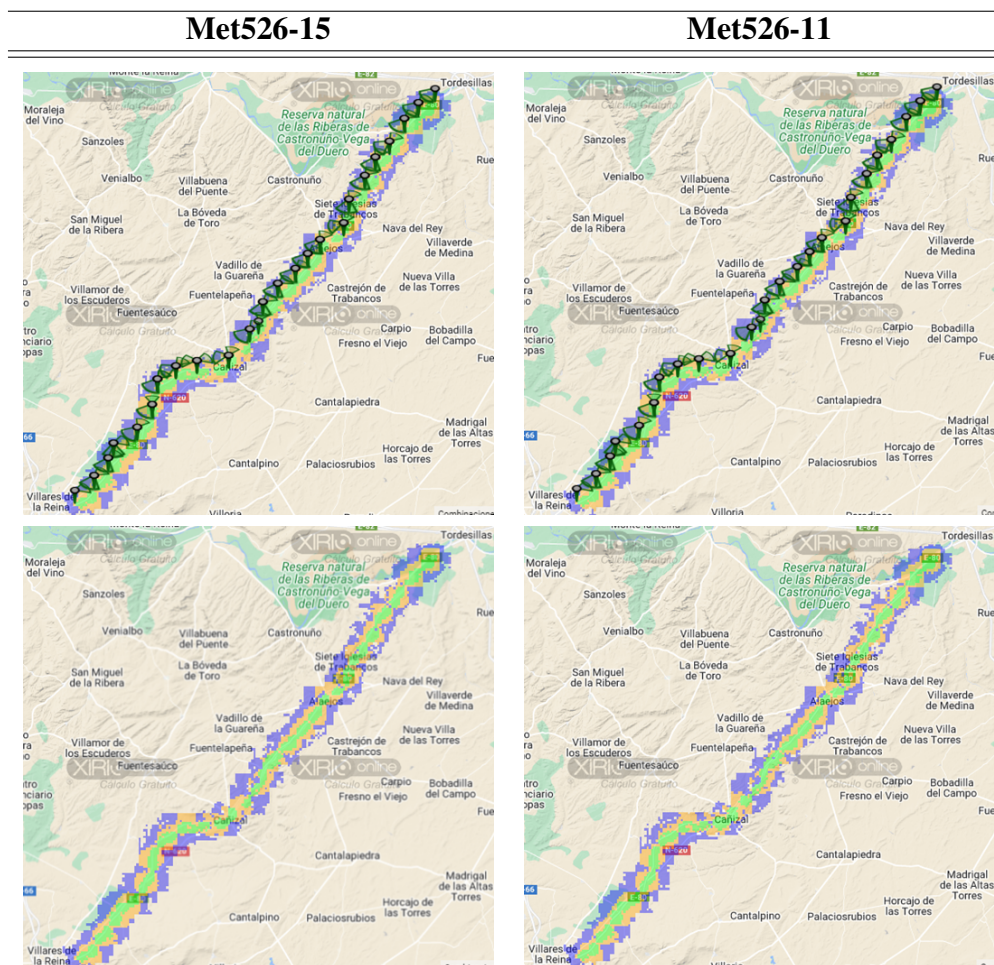


Tabla 6: Resultados RSRP usando 5MHz de ancho de banda.

Tras elegir el centro del sector, configuramos el resto del sector. Para ello, vamos a elegir primero la antena omnidireccional (la misma que en la configuración del terminal del apartado anterior) y una frecuencia de transmisión de 2555 MHz. Por último, situamos la altura de la antena y fijamos su valor a 1.5m.

A continuación, definimos el terminal, cuyos parámetros serán los mismos que los definidos anteriormente en cada antena, ya que vamos a simular que los receptores son las antenas de los eNodeB.

Por último, ajustamos el método de cálculo y el área de cálculo de cada estudio. Para finalizar, creamos un estudio multitransmisor que englobe el resto de los estudios individuales y realizamos el cálculo gratuito. Los resultados pueden verse a continuación.

ANEXO II. SIMULAROR XIRIO-ONLINE

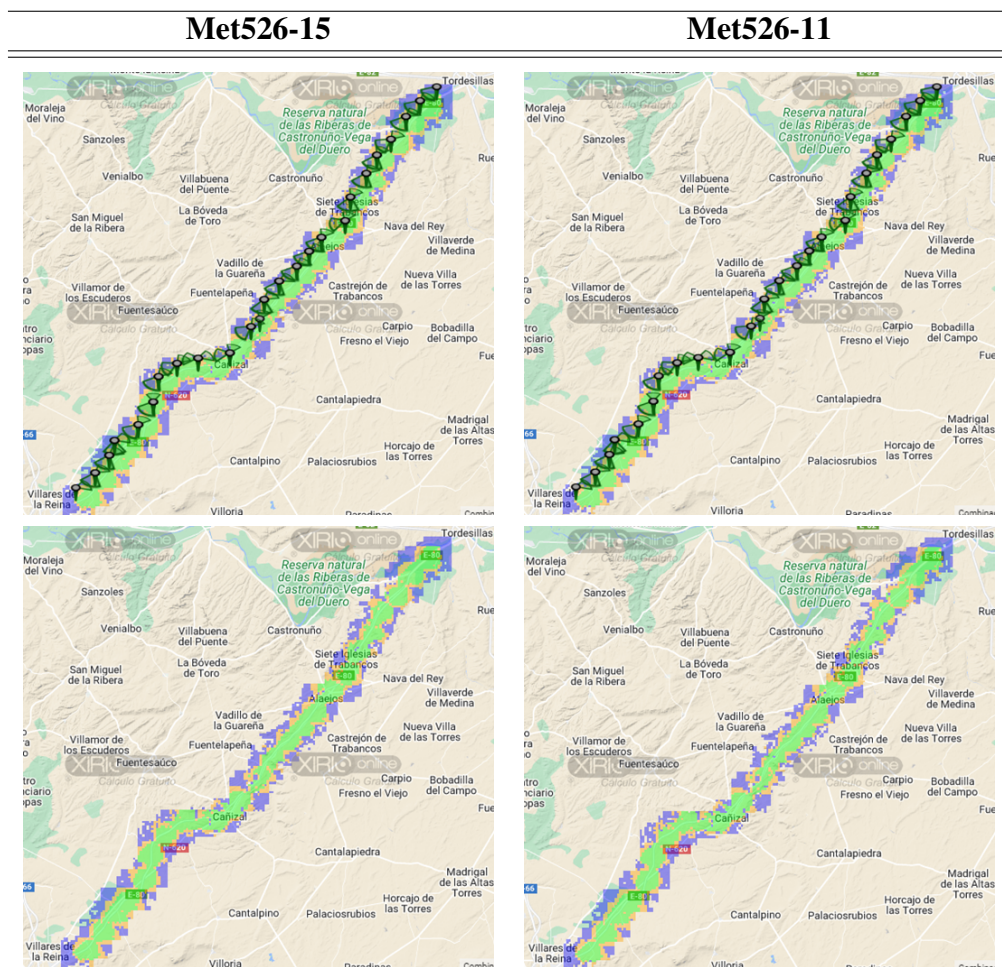


Tabla 7: Resultados RSRP usando 1.4MHz de ancho de banda.

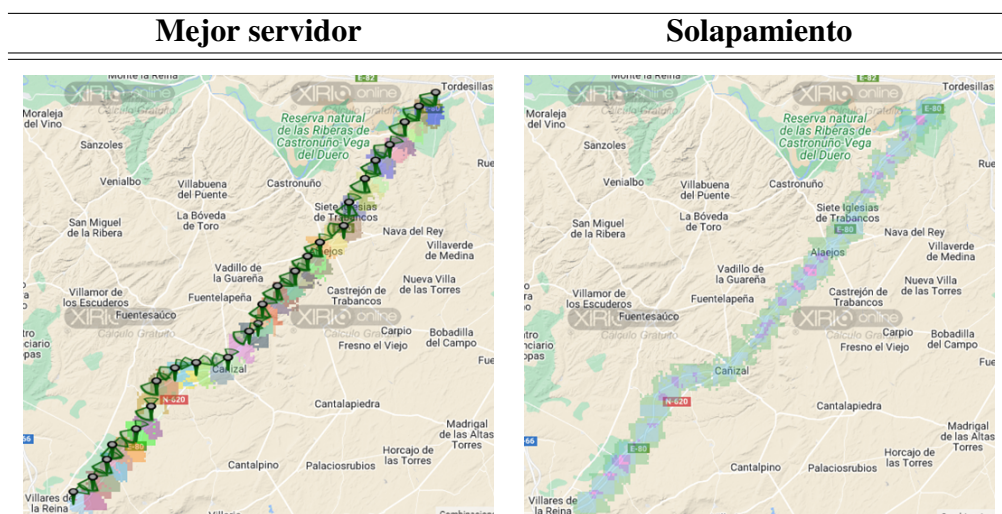


Tabla 8: Resultados mejor servidor y solapamiento con 5MHz de ancho de banda.

ANEXO II. SIMULAROR XIRIO-ONLINE

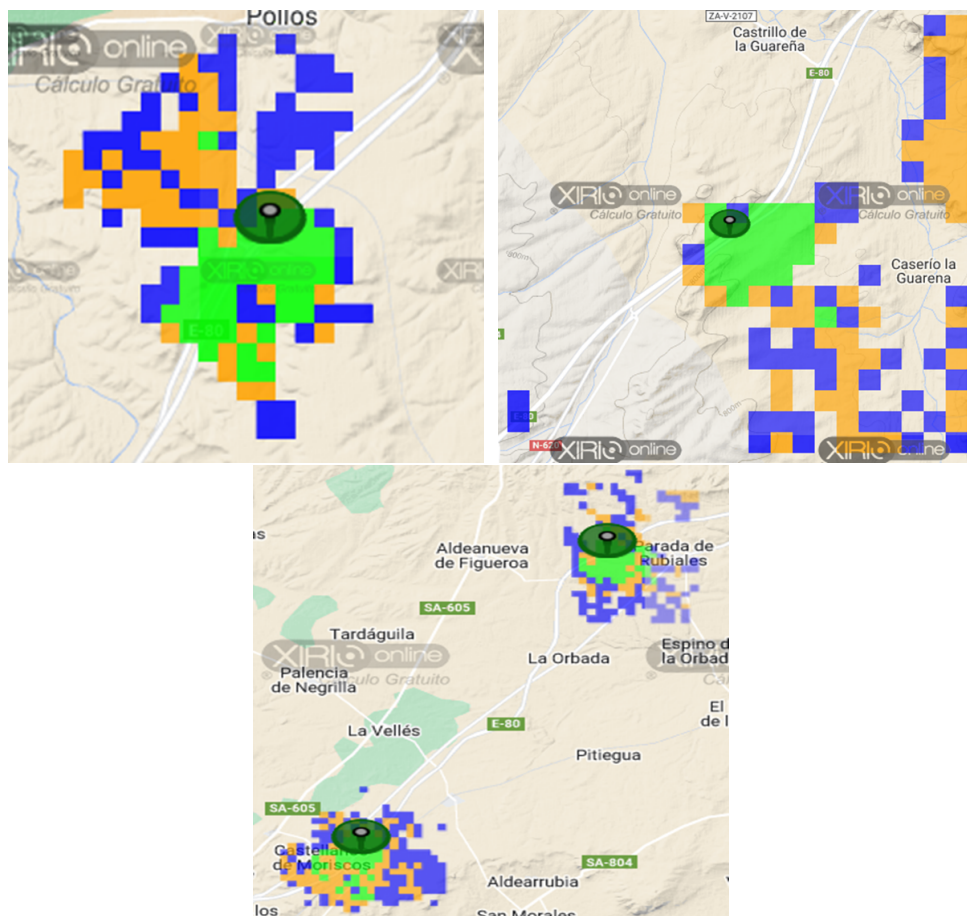


Figura 67: Alcance de las antenas de los UE en distintos puntos de la autovía.

Bibliografía

- [1] E. De Expertos En Ciencia Y Tecnología. «Evolución de la red de comunicación móvil, del 1G al 5G.» (6 de nov. de 2023), dirección: <https://www.universidadviu.com/es/actualidad/nuestros-expertos/evolucion-de-la-red-de-comunicacion-movil-del-1g-al-5g>.
- [2] S. (R. Systems), *srsRAN 4G Features — srsRAN 4G 23.11 documentation*, 2023. dirección: https://docs.srsran.com/projects/4g/en/latest/feature_list.html.
- [3] S. (R. Systems). «Introduction — srsRAN 4G 23.11 documentation.» (2023), dirección: https://docs.srsran.com/projects/4g/en/latest/usermanuals/source/srsepc/source/1_epc_intro.html#epc-intro.
- [4] J. Gualda Muñoz, «Estudio de la arquitectura de protocolos de LTE,» sep. de 2016. dirección: https://upcommons.upc.edu/bitstream/handle/2117/98231/pfc_build.pdf.
- [5] L. Iglesias Quiñones, «PLANIFICACIÓN Y OPTIMIZACIÓN DE UNA RED LTE CON LA HERRAMIENTA ATOLL,» feb. de 2016. dirección: <https://upcommons.upc.edu/bitstream/handle/2117/83015/PFC%20Luis%20Iglesias.pdf>.
- [6] M. para la Transformación Digital y de la Función Pública, *Infoantenas*. dirección: <https://geoportal.minetur.gob.es/VCTEL/vcne.do>.
- [7] Á. Alves, *Drones y 5G, la combinación perfecta. Lo probamos en un caso de uso real con ADIF*, nov. de 2020. dirección: <https://www.linkedin.com/in/angelalves/?originalSubdomain=es>.
- [8] M. Patlayenko, O. Osharovska y V. Solodka, «Comparison of LTE Coverage Areas in Three Frequency Bands,» *IEEE*, sep. de 2021. DOI: 10.1109/aict52120.2021.9628960. dirección: <https://doi.org/10.1109/aict52120.2021.9628960>.
- [9] J. T. J. Penttinen, M. Zarri y D. Kim, *Open RAN explained*. John Wiley Sons, ago. de 2024.
- [10] A. M. Abdalla, I. Elfergani, J. Rodriguez y A. Teixeira, *Optical and Wireless Convergence for 5G Networks*. Wiley-IEEE Press, ago. de 2019.

- [11] L. Huawei Technologies Co., *CPRI*. dirección: <https://forum.huawei.com/enterprise/es/%C2%BFqu%C3%A9-es-el-cpri/thread/667221913707102208-667212889045479424>.
- [12] ¿Qué es una red óptica pasiva (PON)? | VIAVI Solutions Inc. dirección: <https://www.viavisolutions.com/es-es/que-es-una-red-optica-pasiva-pon>.
- [13] *BBU3900*. dirección: <https://forum.huawei.com/enterprise/en/unable-to-login-to-wmpt/thread/667262512707551232-667213872962088960>.
- [14] Vikancha-Msft, *Serie NCas T4 v3 - Azure Virtual Machines*, jun. de 2023. dirección: <https://learn.microsoft.com/es-es/azure/virtual-machines/nct4-v3-series>.
- [15] 3GPP, *LTE; Evolved Universal Terrestrial Radio Access (E-UTRA); User Equipment (UE) Radio Transmission and Reception (3GPP TS 36.101 Version 14.5.0 Release 14)*, release 14. 3GPP.
- [16] M. de Transportes y Movilidad Sostenible, *Estimación tráfico RCE*, 2023. dirección: <https://www.transportes.gob.es/carreteras/trafico-velocidades-accidentes-y-tramos-de-concentracion-de-accidentes/estimacion-trafico-rce-datos-provisionales/estimacion-trafico-de-la-rce-datos-provisionales>.
- [17] F. J. Alonso, «Retos tecnológicos en el desarrollo e implantación del vehículo autónomo y conectado,» *Carreteras: Revista técnica de la Asociación Española de la Carretera*, n.º 216, págs. 8-16, ene. de 2017. dirección: <https://dialnet.unirioja.es/servlet/articulo?codigo=6313820>.
- [18] E. R. Moraguez. «Inteligencia Artificial en Vehículos Autónomos: El Futuro de la Movilidad | LovTechnology.» (19 de jun. de 2023), dirección: <https://lovtechnology.com/inteligencia-artificial-en-vehiculos-autonomos-el-futuro-de-la-movilidad/>.
- [19] H. De La Horra. «7 aplicaciones de la Inteligencia Artificial en los vehículos.» (4 de sep. de 2023), dirección: https://www.webfleet.com/es_es/webfleet/blog/7-aplicaciones-reales-de-la-inteligencia-artificial-en-los-vehiculos/.
- [20] Induportals Media Publishing. «Un marco ético para los vehículos autónomos.» (22 de jun. de 2023), dirección: <https://www.auto-innovaciones.com/news/70281-un-marco-%C3%A9tico-para-los-veh%C3%ADculos-aut%C3%B3nomos>.
- [21] U. «Models Supported by Ultralytics.» (25 de mayo de 2024), dirección: <https://docs.ultralytics.com/models/>.
- [22] A. Ghosh y A. Ghosh. «YOLOv9: Advancing the YOLO Legacy.» (13 de mayo de 2024), dirección: <https://learnopencv.com/yolov9-advancing-the-yolo-legacy/>.
- [23] U. «YOLOV9.» (22 de mayo de 2024), dirección: <https://docs.ultralytics.com/models/yolov9/>.

- [24] *Torre De Comunicacioacute;n Autoportante Para Telecomunicaciones,9,10,12,15,18,20,22,25,28,30,32,33 Y 62 M - Buy Self Supporting Lattice Tower Types Of Communication Towers Antenna Mast And Communication Tower Product on Alibaba.com.* dirección: https://spanish.alibaba.com/p-detail/9-1600632088524.html?spm=a2700.galleryofferlist.normal_offer.d_title.6a3a2b60XbJMDS.
- [25] *Ubiquiti.* dirección: https://www.senetic.es/product/AM-2G16-90?gad_source=1&gclid=Cj0KCQjwsPCyBhD4ARIsAPaaRf3i9VwQS2fmaWJ7g-oMjlkhdM8kSZr7aBfPOmL8jUKExyeVMazZo8oaAiIpEALw_wcB.
- [26] *Huawei-dbs Rru Original,Huawei Rru5301,2600mhz,02312lrx,Huawei Rru 5301 - Buy Rru5301,Huawei Rru 5301,Huawei Rru Product on Alibaba.com.* dirección: <https://spanish.alibaba.com/product-detail/Original-Huawei-DBS-RRU-Huawei-RRU5301-1600814511085.html>.
- [27] A. Simmons, *How Much Does it Cost to Build a Cell Tower?* Ene. de 2024. dirección: <https://dgtlinfra.com/how-much-does-it-cost-to-build-a-cell-tower/#:~:text=On%5C%20average%5C%2C%5C%20the%5C%20total%5C%20cost, and%5C%20height%5C%20of%5C%20the%5C%20tower.>
- [28] *IPLOOK Compact EPC Platform-IKEPC 520(id:11498410) Product details - View IPLOOK Compact EPC Platform-IKEPC 520 from IPLOOK Technologies Co., Ltd - EC21.* dirección: https://iplook.en.ec21.com/IPLOOK_Compact_EPC_Platform_IKEPC--11493241_11498410.html.
- [29] S. (R. Systems), *srsRAN 4G 23.11 Documentation*, 2023. dirección: <https://docs.srsran.com/projects/4g/en/latest/index.html>.
- [30] URSP, *USRP Hardware Driver and USRP Manual: Building and Installing UHD from source.* dirección: https://files.ettus.com/manual/page_build_guide.html.
- [31] L. Garrido Rey, «RECONOCIMIENTO DE SEÑALES DE TRÁFICO MEDIANTE VISIÓN ARTIFICIAL y REDES NEURONALES,» jun. de 2021. dirección: https://www.uv.es/lapeva/Thesis/TFG_2021_Laura_Garrido_compressed.pdf.
- [32] GaryI, *YOLOV9ObjectDetectionFootballPlayers.* dirección: https://github.com/GaryI53/YOLOV9_ObjectDetection_FootballPlayers/blob/main/train_yolov9_object_detection_on_custom_dataset.ipynb.
- [33] U. Networks, «airMAX Sector Antennas Datasheet,» *Ubiquiti Networks*,
- [34] U. Networks, *Guía de inicio rápido de AM-2G16-90*, 2024. dirección: https://dl.ui.com/qsg/AM-2G16-90/AM-2G16-90_ES.html.